

现代控制理论与人工智能

从状态空间到选择性、可编辑与混合记忆

目录

课程总主线：把“序列建模”统一成有限状态记忆问题	7
1 第一章：有限状态如何表示历史——SSM 的基本语言	10
1.1 第 1 页：先看全局——为什么 AI 重新研究状态空间模型	10
1.2 第 2 页：什么是状态——先固定后面所有章节都要使用的符号	10
1.3 第 3 页：先看状态为什么真的含有历史——连续时间精确解	11
1.4 第 4 页：从连续时间进入离散序列——只保留主线需要的离散化	12
1.5 第 5 页：递推展开——当前状态就是全部历史输入的矩阵加权和	12
1.6 第 6 页：读出矩阵 C 为什么在这里出现——从状态自然得到 SSM 卷积核	13
1.7 第 7 页：同一个 SSM 的三种视图——训练和推理为什么可以采用不同算法	13
1.8 第 8 页：第一章真正的终点——为什么不能随便学习一个 A	14
2 第二章：有限状态如何记得更长——HiPPO 与结构化状态空间模型 S4	16
2.1 第 9 页：先给结论——S4 到底优化了什么	16
2.2 第 10 页：HiPPO 先解决“状态应该保存什么”——把历史看成函数	17
2.3 第 11 页：测度决定“记哪一段历史”——先解释 LegT、LegS 的命名与动机	18
2.4 第 12 页：LegS 的正交基——状态系数到底对应什么	19
2.5 第 13 页：为什么投影系数可以在线更新——从积分定义到线性常微分方程 (ODE)	19
2.6 第 14 页：LegS 为什么有意义——尺度等变、长历史和可控时间尺度	20
2.7 第 15 页：原始 LegS 是时变系统——S4 为什么却使用固定 A, B	20
2.8 第 16 页：TOSSM 的投影视角——把状态看成一组卷积基系数	21
2.9 第 17 页：从 HiPPO 进入 S4——现在问题只剩“如何快速生成卷积核”	22
2.10 第 18 页：先学最简单的工具——状态换基不改变输入输出算子	23
2.11 第 19 页：为什么不能直接把 HiPPO 对角化——数值条件数才是障碍	23
2.12 第 20 页：NPLR——HiPPO 只差一个低秩修正就接近正规矩阵	24
2.13 第 21 页：从 NPLR 到 DPLR——把整个 SSM 一起换到对角基	25

2.14	第 22 页：DPLR 并没有让矩阵幂自动变简单——为什么还需要生成函数	25
2.15	第 23 页：为什么在单位根上评价生成函数——快速傅里叶变换（FFT）在哪一步出现	26
2.16	第 24 页：把双线性离散化代回 resolvent——连续参数为什么又出现	26
2.17	第 25 页：Woodbury 恒等式——只解决一个问题：怎样求“对角 + 低秩”的逆	27
2.18	第 26 页：柯西核——S4 最终把长序列问题化成结构化有理函数求值	28
2.19	第 27 页：把 S4 Algorithm 1 用已经学过的符号重新写一遍	28
2.20	第 28 页：S4 为什么同时适合训练和推理——复杂度放回三种视图看	29
2.21	第 29 页：实验告诉我们——真正重要的不只是 NPLR，而是 HiPPO 初始化	29
2.22	第 30 页：S4 层不是只有一个线性 SSM——完整 Deep SSM 还需要非线性和残差	30
2.23	第 31 页：S4D / DSS 为什么会出现——S4 成功以后，哪些结构其实可以删掉	31
2.24	第 33 页：从 S4 到 S5——为什么还要从许多 SISO SSM 改成一个 MIMO SSM	32
2.25	第 35 页：S4、S4D、S5 应该怎样放在同一条学习线上	33
2.26	第 36 页：第二章收束——S4 已经解决“长记忆”，但还没有解决“按内容选择记忆”	33
3	第三章：有限状态如何按内容选择——Mamba 与选择性状态空间模型	35
3.1	第 1 页：先看 Mamba 到底解决什么——S4 已经能记很长，为什么还不够？	35
3.2	第 2 页：先看失败任务——为什么固定卷积核无法解决选择性复制（Selective Copying）	36
3.3	第 3 页：Mamba 的总体方案——不换掉 SSM，只让三个关键参数随输入变化	36
3.4	第 4 页：先固定 Mamba 的张量与符号——避免后面 B 、 D 、状态维度混在一起	37
3.5	第 5 页：选择性不是一句“门控”——从连续参数严格离散到 $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t$	38
3.6	第 6 页：为什么 Δ_t 是最关键的选择参数——从 ZOH 严格推出 RNN gate	39
3.7	第 7 页： $\mathbf{B}_t$ 和 $\mathbf{C}_t$ 分别解决什么——把“写入”和“读出”从固定矩阵中解耦出来	40
3.8	第 8 页：为什么 Mamba 失去了固定卷积——时变系统的“核”依赖整条路径	41
3.9	第 9 页：选择性扫描的数学核心——仿射状态更新具有结合律，可以做并行前缀扫描	41
3.10	第 10 页：数学上能扫描还不够——Mamba 为什么必须做硬件感知的融合与重计算	42
3.11	第 11 页：Mamba 模块（Mamba Block）——selective SSM 如何成为完整神经网络层	43
3.12	第 12 页：把 S4 与 S6 放到同一张算法表——真正改变的是“谁开始依赖 token”	44
3.13	第 13 页：实验怎样验证“selectivity”而不是只验证“架构更大”	45
3.14	第 14 页：第三章收束——Mamba 已经解决 content-awareness，但计算和状态容量留下两个新问题	46
4	第四章：选择性状态如何高效计算——Mamba-2 与结构化状态空间对偶性	47
4.1	第 1 页：先看 Mamba-2 要优化什么——不是重新发明 selectivity，而是重构计算视图	47
4.2	第 2 页：第一步——把时变 SSM 完整展开成一个下三角矩阵乘法	47
4.3	第 3 页：把 $\mathbf{M}$ 写出来——学生先从一个 $T = 4$ 的显式矩阵看结构	48

4.4	第 4 页：再给严格定义——什么是 N -半可分矩阵，为什么 SSM 天然属于它	49
4.5	第 5 页：最小例子——1-半可分矩阵就是一维标量 recurrence	49
4.6	第 6 页：从另一侧看——Linear Attention 其实也是“矩阵状态 + recurrence”	50
4.7	第 7 页：结构化掩码注意力（Structured Masked Attention）——把“因果全 1 mask”推广成可快速乘的结构矩阵	51
4.8	第 8 页：SSD 核心恒等式——Scalar-Identity SSM 与 1-SS Structured Attention 是同一个函数	51
4.9	第 9 页：为什么叫“对偶性”——同一个函数既有线性 recurrence，也有二次 attention 计算	53
4.10	第 10 页：为什么还要设计新算法——纯递推和纯二次矩阵计算都只利用了一半结构	53
4.11	第 11 页：SSD 分块算法（SSD Block Algorithm）——对角块做局部 attention，非对角块通过状态传递	54
4.12	第 12 页：非对角低秩块的三步——右因子（Right Factor）、中心因子（Center Factor）、左因子（Left Factor）	55
4.13	第 13 页：复杂度为什么同时像 SSM 和 Attention——把训练、推理、内存放放到一张表	55
4.14	第 14 页：Mamba-2 Block 不只是换内核——参数生成从 sequential 改成 parallel	56
4.15	第 16 页：Mamba-2 到底改了些什么——把“数学层”“算法层”“网络层”分开总结	57
4.16	第 17 页：实验与下一步——Mamba-2 提高了效率和 state capacity，但“可编辑记忆”问题仍未结束	58
5	第五章：有限状态如何精确改写——DeltaNet、门控 DeltaNet 与 KDA	60
5.1	第 1 页：先看这一章解决什么——Mamba-2 已经能高效选择和衰减，为什么还需要 Delta Rule?	60
5.2	第 2 页：先把 Linear Attention 看成“矩阵关联记忆”——后面的 Delta Rule 才有对象可以修改	61
5.3	第 3 页：为什么纯加法会产生记忆冲突（memory collision）——先用同一个 key 覆盖的最小例子算清楚	61
5.4	第 4 页：Delta Rule 从一个在线回归目标直接推出来——不是经验公式	62
5.5	第 5 页：同一个更新再看一遍——读取-删除-写入（Read-Remove-Write）解释比梯度公式更直观	63
5.6	第 6 页：为什么 $\mathbf{I} - \beta \mathbf{k} \mathbf{k}^\top$ 很重要——DeltaNet 的“删除”本质是一个广义豪斯霍尔德变换	64
5.7	第 7 页：DeltaNet 为什么一开始难以扩展——表达力提高以后，序列并行性变差了	64
5.8	第 8 页：紧凑 WY 表示（Compact WY）的作用——把一串 rank-one 状态转移压成两个“瘦矩阵”	65
5.9	第 9 页：从序列单元递推到分块递推——DeltaNet 怎样重新获得张量核心（Tensor Core）友好的矩阵乘	66
5.10	第 10 页：DeltaNet 不只是一个公式——完整 block 仍包含卷积、归一化、门控和投影	66
5.11	第 11 页：先看 DeltaNet 得到了什么——关联回忆更强，但“长期遗忘”仍是空缺	67

5.12	第 12 页：门控 Delta 规则（Gated Delta Rule）——先衰减旧状态，再对当前 key 做 delta correction	68
5.13	第 13 页：在线学习视角把 LA、Mamba-2、DeltaNet、GDN 放进同一张表	68
5.14	第 14 页：S-NIAH 把“gate”和“delta”的互补性量化出来——不是只看语言建模 perplexity	69
5.15	第 15 页：GDN 的分块算法如何从 DeltaNet 延续下来——只增加衰减感知重标度（decay-aware rescaling）	69
5.16	第 16 页：Gated DeltaNet 的网络结构——recurrent primitive 只是 block 中的核心 mixer .	70
5.17	第 17 页：为什么 GDN 之后还要 KDA——标量 α_t 对整个 key 子空间同时遗忘，仍然太粗	71
5.18	第 20 页：Kimi Linear 的完整架构——KDA 本身很强，但作者仍然保留了少量完整多头潜在注意力（Multi-Head Latent Attention, MLA）	71
5.19	第 21 页：KDA 的合成任务说明了什么——有限状态可以被做得更聪明，但不是无限容量	72
6	第六章：有限状态的边界——精确回忆、状态跟踪与混合注意力	74
6.1	第 1 页：先建立两个彼此不同的能力轴——精确回忆与状态跟踪	74
6.2	第 2 页：精确回忆首先是一个“状态容量”问题——MQAR 直接测有限状态能装多少键值对	75
6.3	第 3 页：BASED 的 Pareto frontier——Attention 记得最准，但代价是状态随上下文增长 .	76
6.4	第 4 页：BASED 已经给出最小 Hybrid 原型——全局压缩记忆 + 很小的局部精确窗口 . .	76
6.5	第 5 页：状态跟踪是另一类边界——“有递推状态”不自动等于“能做真正的顺序状态计算”	77
6.6	第 6 页：先把 TC^0 与 NC^1 的含义讲清——这里关心的是“顺序组合复杂度”	78
6.7	第 7 页：实验验证——S4/Mamba/Transformer 需要随序列增长不断加深，RNN 与更强 input-dependent SSM 不需要	79
6.8	第 8 页：把第五章和理论边界连起来——负特征值为什么突然变得重要	80
6.9	第 9 页：现在可以形成一张真正有用的能力矩阵——Transformer 与 GDN 各强一边 . . .	80
6.10	第 10 页：State-Based Recall——为什么单纯“追踪状态”或单纯“检索”都不够	81
6.11	第 11 页：OLMo 的合成实验把这两个轴分别测出来——Hybrid 是唯一在组合任务上保持稳健的	82
6.12	第 12 页：Hybrid 还有第二个动机——状态大小和 KV cache 的数量级差异巨大	82
6.13	第 15 页：Kimi Linear 与 Qwen3-Next 说明 3:1 已成为一种常见工程折中，但不是理论常数	83
6.14	第 16 页：OLMo Hybrid 的重点不只是“省 cache”——它把 Hybrid 放到表达力与缩放规律（scaling law）两个层面验证	84
6.15	第 17 页：Hybrid 的理论图景——为什么 OLMo 说它可能比两种纯架构都更有表达力 . .	85
6.16	第 19 页：如何选择 Hybrid——先判断任务需要什么信息，再决定哪些信息压缩、哪些信息保留显式索引	85
6.17	第 20 页：第五、六章收束——状态更新越来越聪明，但最终必须承认“有限压缩”和“显式检索”是两种不同资源	86

主线中场总结：三次状态升级	88
7 第七章：状态模型何时适用——任务记忆结构、扫描路径与应用边界	89
7.1 第 1 页：应用章节先解决一个总问题——把 Mamba 放进新模态，真正需要重新设计的是什么？	89
7.2 第 2 页：先看最自然的场景——音频为什么几乎不需要重新定义扫描顺序？	90
7.3 第 3 页：多变量时间序列比音频多一个难题——“变量”本身没有天然的线性顺序	90
7.4 第 4 页：MambaTS 先给整体方案——先解释 VST、VAST 与 TMB 各自解决什么	91
7.5 第 6 页：MambaTS 的复杂度为什么有吸引力——跨变量建模不一定要 K^2	92
7.6 第 7 页：从时间序列进入图像前，先看 S4ND——多维 SSM 可以从“多时间轴”而不是“flatten”出发	93
7.7 第 8 页：视觉 Mamba 的根本问题——若采用一维递推，必须说明二维网格怎样变成序列	94
7.8 第 9 页：Vision Mamba (Vim)——最直接的修正是“前向 + 后向”双向状态传播	95
7.9 第 10 页：VMamba——双向还不够，二维图像最好沿多条空间路径同时扫描	95
7.10 第 11 页：MambaVision——视觉上的经验结论逐渐变成 Hybrid：低层局部结构与高层全局关系不必由同一种 mixer 负责	96
7.11 第 12 页：MambaOut 是必须保留的反例——“序列长”与“任务是否需要 causal state”是两个不同问题	97
7.12 第 13 页：把视觉路线放到一张表——S4ND、Vim、VMamba、MambaVision、MambaOut 实际回答的是不同问题	99
7.13 第 14 页：视频把问题再放大一层——token 数从 HW 变成 THW ，同时需要空间与时间两类上下文	99
7.14 第 15 页：VideoMamba——三维块嵌入（3D patch embedding）后，用双向 Mamba 做长时空序列建模	100
7.15 第 17 页：机器人场景里，“状态”终于重新对应到物理交互历史——但还必须区分推理（reasoning）与 action	100
7.16 第 19 页：MaIL——更贴近“序列策略”的问题：Mamba 直接参数化动作序列的去噪器	101
7.17 第 20 页：SmolVLA Mamba expert——一个非常好的“任务结构”案例：Mamba 只替换 action 块内的 causal temporal mixer	102
7.18 第 21 页：执行 horizon 实验说明了什么——Mamba 的优势在“长开环块协调”时才明显出现	103
7.19 第 22 页：第七章收束——不要按模态记模型名，要按“状态沿什么结构传播”来记	104
8 第八章：从公式到代码——最小 Mamba 的可验证实践	106
8.1 第 1 页：实践部分的目标不是“调用 mamba-ssm 包”——而是亲手验证前三章的三条数学结论	106

8.2	第 2 页：第一步先手写 fixed diagonal SSM——把状态模态和稳定性做成显式参数	106
8.3	第 3 页：严格写出 ZOH——实践里不要偷偷用 $\bar{\mathbf{B}} \approx \Delta \mathbf{B}$ 而不说明	107
8.4	第 4 页：先验证递推与卷积——这一步能一次排掉大量实现错误 (bug)	107
8.5	第 5 页：从固定 SSM 到 selective SSM——只改三件事，不要一开始就把整个 Mamba block 搬进来	108
8.6	第 6 页：张量形状必须先固定——很多“公式看懂但代码写错”都来自广播维度不清 . .	109
8.7	第 7 页：教学版选择性扫描先用逐步循环 (for-loop) ——先求正确，再求并行	109
8.8	第 8 页：再封装最小 Mamba Block——局部卷积和 gate 的功能要与 selective state 分开解释	110
8.9	第 9 页：主实践任务为什么选 Selective Copying——它直接隔离“按内容写入”的能力 .	110
8.10	第 10 页：实验不能只比较最终 accuracy——必须把“模型改动”对应到可观测内部量 .	111
8.11	第 11 页：把 Δ_t 画出来——它最适合用来解释“状态时间常数”	112
8.12	第 12 页：长度外推要区分两种失败——记忆衰减失败与状态容量失败不是一回事	112
8.13	第 13 页：从 for-loop 到并行扫描——实践里只需要证明结合律，不必一上来写 CUDA .	112
8.14	第 15 页：可选挑战任务——把第五章 Delta Rule 接回实践，做“同一 key 被新 value 覆盖”	113
8.15	第 16 页：建议最终作业结构——学生交付的不是一个 accuracy, 而是一套“公式-代码-证据”闭环	114
8.16	第 17 页：第七、八章最终收束——应用和实践都回到同一个判断框架	114
A	扩展阅读：被弱化但仍值得保留的技术细节	116
A. S4	简化分支中的额外细节	116
B.	Mamba-2 的 head-sharing 设计语言	117
C.	KDA 与动态 DPLR/chunkwise 的进一步细节	118
D.	Hybrid 大模型的补充案例与组织形式	120
E.	应用中的额外扫描与架构案例	123
F.	NPU/Ascend 迁移的设备特定建议	125
	术语速查：第一次阅读时不需要死记英文	127
	全课程收束：最后只保留五句话	127

课程总主线：把“序列建模”统一成有限状态记忆问题

整门课不再按论文名称串联，而统一从一个更一般的问题出发。给定序列输入 $\mathbf{x}_1, \dots, \mathbf{x}_T$ ，模型需要把过去的信息变成当前可用的记忆。无论状态是向量还是矩阵，都可以抽象成

$$\mathcal{M}_t = \Phi_t(\mathcal{M}_{t-1}, \mathbf{x}_t), \quad \mathbf{y}_t = \Psi_t(\mathcal{M}_t, \mathbf{x}_t), \quad (1)$$

其中 $\mathcal{M}_t$ 是有限状态记忆。前四章的 $\mathcal{M}_t$ 主要是向量状态 $\mathbf{h}_t \in \mathbb{R}^N$ ；第五章以后把它扩展为矩阵关联记忆 $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ 。矩阵状态并没有离开“状态空间”范式，因为把 $\mathbf{S}_t$ 向量化以后仍然只是一个固定维度的递推状态。

真正贯穿全课的不是某个模型名，而是四个逐层升级的问题：

1. **状态里应该保存什么？**——第一、二章从普通 SSM 走到 HiPPO/S4，解决长历史表示与长核计算；
2. **状态应该怎样更新？**——第三、五章分别引入内容感知选择（content-aware selection）与 error-corrective overwrite；
3. **这些更强的更新怎样高效训练？**——S4 用结构化长卷积，Mamba 用并行扫描，Mamba-2 用分块矩阵计算，Delta/GDN 用分块低秩表示——它们都在解决同一类“表达力增强后如何重新并行化”的矛盾；
4. **有限状态最终做不到什么？**——第六章把精确回忆与状态跟踪分开，说明为什么混合架构（Hybrid）Attention 不是“倒退”，而是两种记忆资源的分工。

表 1: 八章内容在同一条“有限状态记忆”主线中的位置。

章节	核心对象	主要改动	下一步仍未解决的问题
1	$\mathbf{h}_t$	建立连续/离散 SSM、递推与卷积等价	怎样让固定维状态真正记得很长?
2	$\mathbf{h}_t$	HiPPO 规定“记什么”, S4 结构化 A 并快速生成 K_j	固定的线性时不变 (LTI) 核不理解序列单元内容
3	$\mathbf{h}_t$	$\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ 输入相关, 形成选择性 SSM (selective SSM)	时变递推 (recurrence) 的并行训练更复杂
4	$\mathbf{h}_t / \mathbf{M}$	SSD: 递推 $\Leftrightarrow$ 半可分矩阵 (semiseparable matrix) $\Leftrightarrow$ 结构化注意力	状态会写、会忘, 但还不善于定点覆写
5	$\mathbf{S}_t$	Delta/GDN/KDA: 预测误差校正、遗忘与细粒度衰减	固定容量仍限制精确回忆 (exact recall)
6	状态压缩 + 显式检索	区分精确回忆与状态跟踪 (state tracking) 两种边界, 形成混合架构 (Hybrid)	如何按任务结构分配两类记忆?
7	任务相关状态	把序列单元 (token)、扫描路径、状态含义与混合需求放回具体模态	怎样把这些结论变成可验证实现?
8	代码中的状态	固定 SSM $\rightarrow$ 选择性扫描 $\rightarrow$ 最小 Mamba $\rightarrow$ 覆写挑战	完成“公式-代码-证据”闭环

统一符号：后文不再重复更换核心记号

符号	含义	全文约定
$\mathbf{x}_t$	第 t 个序列单元 (token) / 输入	单输入单输出 (SISO) 推导时可退化为标量 x_t ; 深度模型中为向量
$\mathbf{h}_t \in \mathbb{R}^N$	向量状态	第一至四章的主要有限状态记忆 (finite-state memory)
$\mathbf{A}$	连续时间状态转移矩阵	S4/Mamba 中共享的动态模态; 离散后写成 $\bar{\mathbf{A}}$ 或 $\bar{\mathbf{A}}_t$
$\bar{\mathbf{A}}, \bar{\mathbf{A}}_t$	离散状态转移	固定 SSM 中固定; Mamba 中随 Δ_t 改变; SSD 特例可写 $\bar{\mathbf{A}}_t = a_t \mathbf{I}$
$\mathbf{B}, \mathbf{B}_t$	连续时间写入向量	固定 SSM 中固定; Mamba 中由当前序列单元生成
$\bar{\mathbf{B}}, \bar{\mathbf{B}}_t$	离散写入向量	由 $(\mathbf{A}, \mathbf{B}_t, \Delta_t)$ 经 ZOH 得到
$\mathbf{C}, \mathbf{C}_t$	读出向量	固定 SSM 中固定; Mamba 中由当前序列单元生成

符号	含义	全文约定
Δ, Δ_t	离散步长/时间尺度	Mamba 中输入相关，控制状态保留与 reset 速度
K_j	LTI SSM 卷积核	$K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}$ ；只在固定参数情形下是距离核
$M_{t,i}$	一般序列混合矩阵元素	时变 SSM 中 $M_{t,i} = \mathbf{C}_t^\top (\prod_{r=i+1}^t \bar{\mathbf{A}}_r) \bar{\mathbf{B}}_i$
$\mathbf{S}_t$	矩阵关联记忆	第五、六章用来统一线性注意力 (Linear Attention)、DeltaNet、GDN、KDA
$\mathbf{k}_t, \mathbf{q}_t, \mathbf{v}_t$	键/查询/值 (key/查询 (query) /值 (value))	统一取 $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ ，读出 $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$
D_{skip}	SSM 直接/跳跃项 (direct/skip term)	为避免与 Mamba 通道数 D 混淆，第一章将原记号 D 解释为 D_{skip}
D	selective SSM 通道数	Mamba 章节中每个通道维护一个 N 维状态

术语提示：全文采用“中文术语为主、首次出现给出英文”的写法。例如：卷积核 (kernel)、递推 (recurrence)、扫描 (scan)、半可分矩阵 (semiseparable matrix)、联想记忆 (associative memory)、精确回忆 (exact recall)、状态跟踪 (state tracking)。NPLR、DPLR、SSD、KDA 等已经成为模型/参数化名称的缩写保留不译，但首次出现会解释其完整含义。

状态表示 → 内容感知更新 → 高效结构化计算
→ 可编辑记忆 → 混合记忆

(2)

SSM/HiPPO/S4 Mamba Mamba-2/SSD Delta/GDN/KDA 状态 + 注意力

后续每一章只是在式(1)的一个组成部分上继续加约束或放宽约束。这样读者不需要记住一串模型名，只需要持续追问：**状态里装什么、怎样更新、怎样并行算、什么时候压缩不够。**

建议分页位置

1 第一章：有限状态如何表示历史——SSM 的基本语言

本章只解决一个问题：状态为什么能代表过去？连续状态空间模型怎样变成离散序列模型与卷积核？

本章给出的答案：把历史通过状态递推压缩为有限维充分表示；在线性时不变（linear time-invariant, LTI）情形下，递推可严格展开为卷积。

1.1 第 1 页：先看全局——为什么 AI 重新研究状态空间模型

这一章先不从离散化公式开始，而是先回答学生最容易问的一个问题：**已经有 RNN 和 Attention，为什么还要重新研究状态空间模型（State Space Model, SSM）？**

对长度为 L 的输入序列 $u_{1:L} = (u_1, \dots, u_L)$ ，序列模型至少要解决两件事：第一，如何把过去的信息传到当前；第二，如何让训练和推理都能承受很长的序列。三类模型的基本思路可以先并列来看：

表 3：三种序列记忆路线的基本区别。这里的目的是不是给出优劣排名，而是先建立“历史如何被保存”的整体认识。

模型	历史如何保存	典型计算形式	主要矛盾
RNN	把历史压缩进固定维状态 $\mathbf{h}_t$	$\mathbf{h}_t = f(\mathbf{h}_{t-1}, u_t)$	长期传播依赖反复递推，梯度和信息都可能衰减或爆炸
Attention	显式保留过去 token 的 key/value	$\text{softmax}(QK^\top)V$	精确检索强，但训练计算和自回归键值缓存（KV cache）随长度增长
SSM	用结构化状态 $\mathbf{h}_t$ 压缩历史	$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}u_t$	需要同时解决长记忆、稳定训练和高效计算

这一章的目标可以浓缩为一个问题：

怎样让固定维状态 $\mathbf{h}_t$ 真正成为一段长历史的有效压缩？

后续 HiPPO、S4、Mamba 都是在改变“状态如何保存历史”这件事，只是它们解决的层次不同。

来源：LSSL; S4 Sec. 2.

[建议分页位置](#)

1.2 第 2 页：什么是状态——先固定后面所有章节都要使用的符号

为了保证后面公式不再反复换记号，本课程统一采用下面的 SISO 连续时间 SSM：

$$\dot{\mathbf{h}}(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}u(t), \quad y(t) = \mathbf{C}\mathbf{h}(t) + D_{\text{skip}}u(t), \quad (3)$$

其中

$$\mathbf{h}(t) \in \mathbb{R}^N, \quad \mathbf{A} \in \mathbb{R}^{N \times N}, \quad \mathbf{B} \in \mathbb{R}^{N \times 1}, \quad \mathbf{C} \in \mathbb{R}^{1 \times N}.$$

这里 $u(t)$ 是输入， $y(t)$ 是输出， $\mathbf{h}(t)$ 是内部状态。四个参数的作用以后不再重新定义：

A：状态如何演化， **B**：输入如何写入， **C**：状态如何读出， D_{skip} ：输入到输出的直接通道。

“状态”的价值可以用最简单的二阶物理系统理解。对于

$$m\ddot{q} + c\dot{q} + kq = u,$$

若只知道当前位置 $q(t_0)$ ，并不能预测下一时刻，因为速度 $\dot{q}(t_0)$ 同样决定未来。令

$$\mathbf{h}(t) = \begin{bmatrix} q(t) \\ \dot{q}(t) \end{bmatrix},$$

则

$$\dot{\mathbf{h}}(t) = \underbrace{\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}}_{\mathbf{A}} \mathbf{h}(t) + \underbrace{\begin{bmatrix} 0 \\ 1/m \end{bmatrix}}_{\mathbf{B}} u(t), \quad y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \mathbf{h}(t).$$

因此，“状态”并不是神经网络中随意命名的 hidden vector，而是把当前输出中没有直接显示、但预测未来仍需要的信息集中保存起来的内部变量。

从这一页起，后面的所有 Deep SSM 都沿用 **A** 表示状态演化、**B** 表示写入、**C** 表示读出。这样后面出现 $\mathbf{B}_t, \mathbf{C}_t$ 时，学生可以直接理解为“写入和读出开始依赖当前 token”。

来源：S4 Eq. (1); LSSL Fig. 1.

[建议分页位置](#)

1.3 第 3 页：先看状态为什么真的含有历史——连续时间精确解

式(3)的状态方程是线性常微分方程。对任意初始时刻 t_0 ，其精确解为

$$\mathbf{h}(t) = e^{\mathbf{A}(t-t_0)}\mathbf{h}(t_0) + \int_{t_0}^t e^{\mathbf{A}(t-\tau)}\mathbf{B}u(\tau) d\tau. \quad (4)$$

这个公式已经把“状态保存历史”的机制完全写出来了：

- $e^{\mathbf{A}(t-t_0)}\mathbf{h}(t_0)$ 是旧状态传播到当前的残留；
- 对任意过去时刻 τ ，输入 $u(\tau)$ 先经 **B** 写入，再由 $e^{\mathbf{A}(t-\tau)}$ 传播到当前；
- 因此 **A** 决定“多远以前的信息还能留下多少”，**B** 决定“新输入进入哪些状态方向”。

若 **A** 可对角化， $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ ，则

$$e^{\mathbf{A}t} = \mathbf{V}e^{\mathbf{\Lambda}t}\mathbf{V}^{-1} = \mathbf{V} \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_N t})\mathbf{V}^{-1}.$$

每个特征值 λ_n 对应一个连续时间模态： $\Re(\lambda_n)$ 决定衰减速度， $\Im(\lambda_n)$ 决定振荡频率。因此，状态维度 N 可以理解成一组并行的“记忆模态”。

来源：经典线性系统解；S4 Sec. 2.1.

[建议分页位置](#)

1.4 第 4 页：从连续时间进入离散序列——只保留主线需要的离散化

实际 AI 输入是 $u_0, u_1, \dots, u_{L-1}$ 。因此连续系统必须变成离散递推。统一把离散系统写成

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k, \quad y_k = \mathbf{C}\mathbf{h}_k + D_{\text{skip}}u_k. \quad (5)$$

这里 $\bar{\mathbf{A}}, \bar{\mathbf{B}}$ 是离散参数；后面凡是带横线的矩阵，都表示离散时间参数。

如果采用零阶保持 (zero-order hold, ZOH)，有

基础知识补充：什么是零阶保持

离散序列 u_k 若来自对连续输入的采样，必须说明两个采样点之间的输入如何处理。**零阶保持 (ZOH)** 假设在区间 $[k\Delta, (k+1)\Delta)$ 内输入保持为常数 $u(t) = u_k$ 。因此离散化不是把 $\dot{h}$ 简单替换成差分，而是把这一小段时间内的连续动力学精确积分。后面的 Mamba 也会再次使用这一假设，只是把步长 Δ 改成输入相关的 Δ_{t_0} 。

$$\bar{\mathbf{A}} = e^{\Delta\mathbf{A}}, \quad \bar{\mathbf{B}} = \int_0^\Delta e^{\mathbf{A}\tau}\mathbf{B} d\tau. \quad (6)$$

若 $\mathbf{A}$ 可逆，进一步有

$$\bar{\mathbf{B}} = \mathbf{A}^{-1}(e^{\Delta\mathbf{A}} - \mathbf{I})\mathbf{B}.$$

S4 主论文采用双线性变换 (bilinear transform)：

$$\bar{\mathbf{A}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2}\mathbf{A}), \quad \bar{\mathbf{B}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B}. \quad (7)$$

这一章不展开 Euler，因为后续算法真正会再次使用的是式(7)：S4 的生成函数会把离散预解矩阵重新写回连续时间的 $\mathbf{A}$ 。

离散化以后，连续极点和离散记忆系数直接联系起来。对 ZOH，

$$\mu_n = e^{\Delta\lambda_n},$$

所以 $|\mu_n|$ 越接近 1，第 n 个状态模态衰减越慢； $|\mu_n| > 1$ 则会不稳定。这一事实后面会直接变成“为什么不能随便学习 $\mathbf{A}$ ”。

来源：S4 Eq. (3); S5 Eq. (6).

[建议分页位置](#)

1.5 第 5 页：递推展开——当前状态就是全部历史输入的矩阵加权和

现在从式(5)直接展开，不再引入任何新记号：

$$\begin{aligned} \mathbf{h}_k &= \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k \\ &= \bar{\mathbf{A}}(\bar{\mathbf{A}}\mathbf{h}_{k-2} + \bar{\mathbf{B}}u_{k-1}) + \bar{\mathbf{B}}u_k \\ &= \bar{\mathbf{A}}^2\mathbf{h}_{k-2} + \bar{\mathbf{A}}\bar{\mathbf{B}}u_{k-1} + \bar{\mathbf{B}}u_k. \end{aligned}$$

继续递推得到

$$\mathbf{h}_k = \bar{\mathbf{A}}^{k+1}\mathbf{h}_{-1} + \sum_{j=0}^k \bar{\mathbf{A}}^j\bar{\mathbf{B}}u_{k-j}. \quad (8)$$

若取 $\mathbf{h}_{-1} = 0$ ，则

$$\mathbf{h}_k = \bar{\mathbf{B}}u_k + \bar{\mathbf{A}}\bar{\mathbf{B}}u_{k-1} + \bar{\mathbf{A}}^2\bar{\mathbf{B}}u_{k-2} + \cdots + \bar{\mathbf{A}}^k\bar{\mathbf{B}}u_0.$$

这里每一项的结构完全一致：

$$\underbrace{u_{k-j}}_{\text{历史输入}} \xrightarrow{\bar{\mathbf{B}}} \underbrace{\bar{\mathbf{B}}u_{k-j}}_{\text{写入状态}} \xrightarrow{\bar{\mathbf{A}}^j} \underbrace{\bar{\mathbf{A}}^j\bar{\mathbf{B}}u_{k-j}}_{\text{传播 } j \text{ 步后的残留}}.$$

因此，在进入 HiPPO 前必须先建立一个非常具体的认识：**状态不是抽象的“记忆”，而是所有历史输入经过不同次数状态转移以后叠加起来的结果。**

来源：S4 Sec. 2.4 的离散展开。

[建议分页位置](#)

1.6 第 6 页：读出矩阵 $\mathbf{C}$ 为什么在这里出现——从状态自然得到 SSM 卷积核

前一页只讨论内部状态。现在回到从第 2 页就已经定义过的读出方程

$$y_k = \mathbf{C}\mathbf{h}_k + D_{\text{skip}}u_k.$$

把式(8)代入，在零初始状态下得到

$$\begin{aligned} y_k &= \mathbf{C} \sum_{j=0}^k \bar{\mathbf{A}}^j \bar{\mathbf{B}} u_{k-j} + D_{\text{skip}} u_k \\ &= \underbrace{\mathbf{C}\bar{\mathbf{B}}}_{K_0} u_k + \underbrace{\mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}}_{K_1} u_{k-1} + \underbrace{\mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}}_{K_2} u_{k-2} + \cdots + D_{\text{skip}} u_k. \end{aligned}$$

于是定义

$$K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}, \quad j = 0, 1, \dots, L-1, \quad (9)$$

整个核写成

$$\mathbf{K}_L(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}) = (\mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}}). \quad (10)$$

因此

$$y = \mathbf{K} * u + D_{\text{skip}}u. \quad (11)$$

式(9)是后面 S4、S4D、Mamba-2 反复出现的共同对象。区别只在于：**谁来设计 $\mathbf{A}$ ，怎样快速计算 $\bar{\mathbf{A}}^j$ ，以及参数是否随序列单元 (token) 改变。**

来源：S4 Eqs. (4)–(5).

[建议分页位置](#)

1.7 第 7 页：同一个 SSM 的三种视图——训练和推理为什么可以采用不同算法

式(3)、式(5)和式(11)并不是三个模型，而是同一个线性系统的三种计算视图：

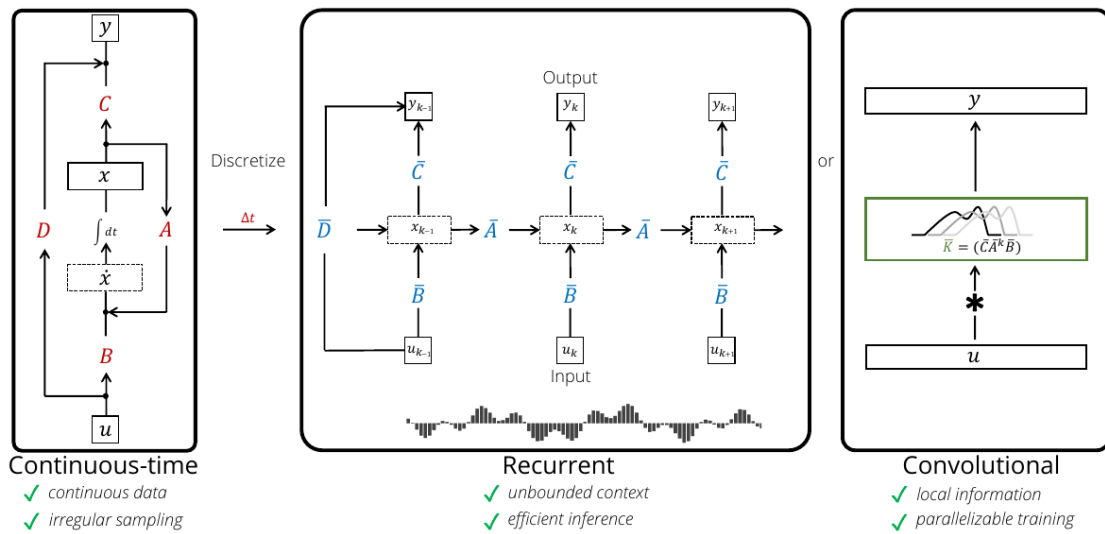


图 1: LSSL 给出的三种 SSM 视图。图中英文标签保留原论文, 中文含义分别是连续动力学、离散递推和卷积计算。

- **连续时间视图:** 参数 (A, B, C) 有明确动力学含义, 也便于处理采样率变化;
- **递推视图:** 每一步只保存 N 维状态, 适合自回归推理;
- **卷积视图:** 整段序列可并行处理, 适合训练。

真正的难点因此不是“SSM 能不能写成卷积”, 而是: 给定长度 L , 怎样高效得到

$$K_0, K_1, \dots, K_{L-1} = \mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}}.$$

如果逐项做矩阵乘, 复杂度会很高。S4 的核心技术就是围绕这个问题展开。

来源: LSSL Fig. 1; S4 Sec. 2.

[建议分页位置](#)

1.8 第 8 页：第一章真正的终点——为什么不能随便学习一个 A

到这里, 状态、离散化、历史展开和卷积核已经连成一条完整链:

$$(\mathbf{A}, \mathbf{B}, \mathbf{C}) \xrightarrow{\text{discretize}} (\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}) \xrightarrow{\text{unroll}} \mathbf{h}_k \xrightarrow{\text{readout}} K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}.$$

于是 Deep SSM 面临的核心问题变得非常具体: 怎样选 A 才能让 K_j 同时“记得长、算得快、训练得稳”?

表 4: 设计长序列 SSM 时，状态矩阵必须同时满足的四个要求。

要求	数学表现	直接问题
长记忆	$\ \bar{\mathbf{A}}^j\ $ 在较大 j 时仍保留有效模态	若谱半径太小，远期输入迅速消失
稳定性	递推不能产生无界放大	若谱半径太大，状态和梯度会爆炸
可训练性	参数化对优化器和有限精度数值友好	理论可对角化不代表数值上能稳定计算
高效性	$K_0, \dots, K_{L-1}$ 能比 $O(N^2L)$ 更快生成	否则卷积视图失去训练优势

这也是第二章的学习顺序：先看 HiPPO 如何回答“状态应该保存什么”，再看 S4 如何回答“这些状态如何快速生成长卷积核”。

来源：S4 Secs. 2–3; S4 Fig. 3–4.

2 第二章：有限状态如何记得更长——HiPPO 与结构化状态空间模型 S4

本章只解决一个问题：固定维状态怎样在稳定、可训练的前提下保存很长的历史，并高效生成长卷积核？

本章给出的答案：HiPPO 先规定“状态应该记什么”；S4 再把这个记忆矩阵变换到数值稳定、适合低秩修正和频域快速求值的结构中，从而解决“怎样算得快”。

2.1 第 9 页：先给结论——S4 到底优化了什么

在进入 HiPPO 的定理和 S4 的线性代数之前，先把算法的最终目标说清楚。S4 不是为了改变式(5)的基本状态方程，而是为了让这个状态方程在长序列上真正可用。

普通 SSM 已经有

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k, \quad K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}.$$

S4 的技术链可以先概括为四步：

表 5: S4 的四个核心问题与对应工具。后面的数学推导都围绕这张表展开。

问题	S4/HiPPO 的处理	后续数学工具
状态应该记什么	用 HiPPO 把状态定义成历史函数在正交基上的投影系数	希尔伯特空间 (Hilbert space)、正交投影、勒让德多项式基 (Legendre basis)
如何得到长记忆 A	使用缩放勒让德方案 HiPPO-LegS 所诱导的结构化矩阵	LegS 状态方程、尺度等变与时间尺度
如何让 A 可稳定计算	把 HiPPO 矩阵写成“正规矩阵 + 低秩修正”	酉相似变换、正规矩阵加低秩修正 (Normal Plus Low-Rank, NPLR)、对角矩阵加低秩修正 (Diagonal Plus Low-Rank, DPLR)
如何快速生成长卷积核	不直接算 $\bar{\mathbf{A}}^j$ ，改算生成函数 (generating function)，再利用 Woodbury 恒等式与柯西核	预解矩阵 (resolvent)、快速傅里叶变换 (FFT)、Woodbury 恒等式、柯西矩阵 (Cauchy matrix)

最终目标可以写成：

HiPPO 历史表示 $\rightarrow$ 正规/对角 + 低秩结构化参数化 $\rightarrow$ 快速生成 $\mathbf{K}_L \rightarrow$ S4 层.

这一页先给学生地图，后面才逐步解释每一箭头为什么成立。

来源：HiPPO; How to Train Your HiPPO; S4 Sec. 3.

建议分页位置

2.2 第 10 页：HiPPO 先解决“状态应该保存什么”——把历史看成函数

对当前时刻 t ，过去不是一个有限向量，而是一个整个函数

$$f_{\leq t} : \tau \mapsto f(\tau), \quad 0 \leq \tau \leq t.$$

模型只能保存 N 维状态 $\mathbf{c}(t) \in \mathbb{R}^N$ 。HiPPO 的基本思想不是让神经网络自由决定 $\mathbf{c}(t)$ 的含义，而是先规定： $\mathbf{c}(t)$ 就是历史函数在一组正交基上的最优投影系数。

基础知识补充：内积、正交与希尔伯特空间

在有限维向量空间中，内积 $\langle \mathbf{a}, \mathbf{b} \rangle$ 衡量两个方向的“相似程度”；若内积为零，就称两者正交。函数空间也可以定义内积，例如

$$\langle f, g \rangle_{\mu} = \int f(\tau)g(\tau) d\mu(\tau).$$

带有完备内积结构的函数空间称为**希尔伯特空间 (Hilbert space)**。因此“把历史函数投影到一组正交基上”与在线性代数中把向量分解到正交坐标轴上是同一件事，只是对象从有限维向量换成了函数。

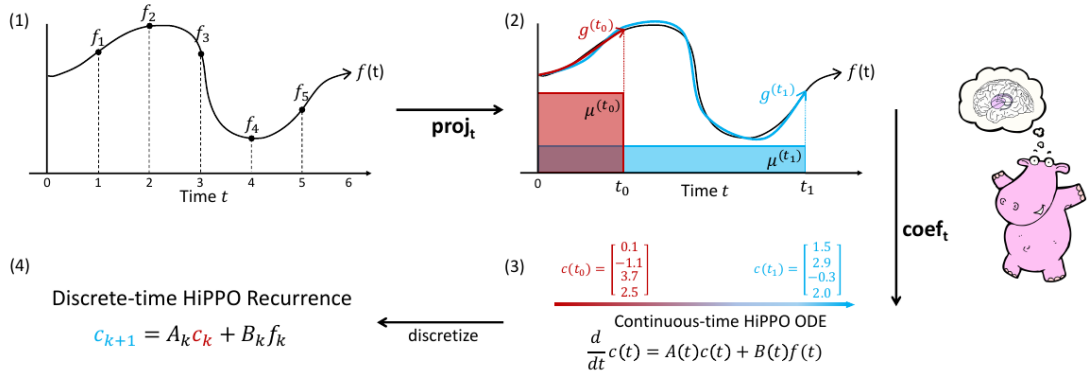


图 2: HiPPO 的总体流程：历史函数 f 在每个时刻被投影为有限个系数 $\mathbf{c}(t)$ ，这些系数满足线性常微分方程 (ordinary differential equation, ODE)，离散化后得到在线递推。原图来自 HiPPO Fig. 1，图中英文只保留流程标记，具体数学定义在正文中重新给出。

设 $\mu^{(t)}$ 是对过去区间赋权的测度，定义内积

$$\langle f, g \rangle_{\mu^{(t)}} = \int f(\tau)g(\tau) d\mu^{(t)}(\tau). \quad (12)$$

若 $\{g_n^{(t)}\}_{n=0}^{N-1}$ 在该内积下正交归一，则最优投影为

$$f_{\leq t}(\tau) \approx \sum_{n=0}^{N-1} c_n(t)g_n^{(t)}(\tau), \quad c_n(t) = \langle f_{\leq t}, g_n^{(t)} \rangle_{\mu^{(t)}}. \quad (13)$$

因此，状态维数 N 在 HiPPO 中有了明确含义：它是保留多少个历史基函数系数，而不只是一个任意 hidden size。

来源：HiPPO Definition 1 and Fig. 1.

建议分页位置

2.3 第 11 页：测度决定“记哪一段历史”——先解释 LegT、LegS 的命名与动机

式(13)中真正决定记忆策略的是 $\mu^{(t)}$ 。如果给过去不同位置不同权重，得到的状态含义也不同。

术语提示：HiPPO 中的“Leg”指 **勒让德多项式 (Legendre polynomial)**。LegT 中的 T 表示 **平移 (translated)**：用一个固定宽度窗口随时间向前移动；LegS 中的 S 表示 **缩放 (scaled)**：把不断增长的区间 $[0, t]$ 缩放到统一标准区间。因而两者不是两个凭空出现的模型名，而是“同一种勒让德基 + 两种不同历史坐标变换”。

基础知识补充：勒让德多项式为什么会出现

勒让德多项式 $P_0, P_1, \dots$ 是 $[-1, 1]$ 上的一组经典正交多项式，满足

$$\int_{-1}^1 P_n(z) P_m(z) dz = \frac{2}{2n+1} \delta_{nm}.$$

因此它们很适合充当“历史函数的坐标轴”：低阶项描述整体趋势，高阶项逐渐补充更细的形状信息。HiPPO 并不是因为 Legendre 名字特殊才选择它，而是利用了正交多项式能够系统逼近函数、且投影系数可递推更新的性质。

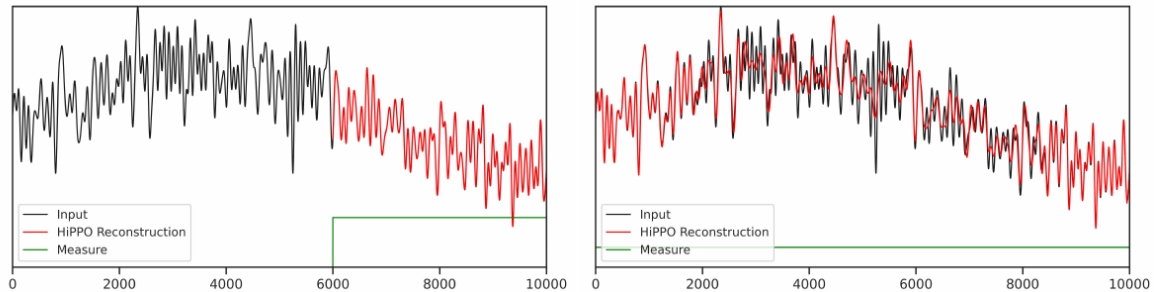


图 3: HiPPO 的在线重构直觉。左图对应固定长度窗口，右图对应整个历史的尺度化表示。红线是由有限维状态重构的历史，绿色表示测度的支持/权重。图源：How to Train Your HiPPO Fig. 2（仅保留曲线部分）。

两种最重要的 Legendre 方案可以先这样区分：

表 6: LegT 与 LegS 的教学对照。

方法	测度直觉	状态含义
LegT	固定宽度滑动窗口	重点表示最近一段固定时间范围
LegS	对 $[0, t]$ 整段历史做尺度归一	随时间增长仍表示完整历史，具有尺度等变性

HiPPO-LegS 采用

$$d\mu^{(t)}(\tau) = \frac{1}{t} \mathbf{1}_{[0,t]}(\tau) d\tau. \quad (14)$$

这个 $1/t$ 不是装饰：它把不断增长的区间 $[0, t]$ 归一化到同一尺度，使得“历史有多长”不会改变基函数的基本形态。

来源：HiPPO Sec. 3; How to Train Your HiPPO Fig. 2.

建议分页位置

2.4 第 12 页：LegS 的正交基——状态系数到底对应什么

把 $[0, t]$ 映射到标准区间 $[-1, 1]$:

$$z = \frac{2\tau}{t} - 1.$$

令 $P_n(z)$ 为第 n 阶勒让德多项式，则可取归一化基

$$g_n^{(t)}(\tau) = \sqrt{2n+1} P_n\left(\frac{2\tau}{t} - 1\right). \quad (15)$$

在测度(14)下满足

$$\int_0^t g_n^{(t)}(\tau) g_m^{(t)}(\tau) \frac{d\tau}{t} = \delta_{nm}.$$

于是每个状态分量就是

$$c_n(t) = \frac{1}{t} \int_0^t f(\tau) g_n^{(t)}(\tau) d\tau. \quad (16)$$

这时可以对状态维数给出更具体的解释： $c_0(t)$ 更接近全局平均/低阶趋势，随后更高阶的 $c_n(t)$ 逐步表示更细致的历史变化。增加 N 并不是“凭空增加神经元”，而是提高历史函数的近似阶数。

从投影角度看， $\mathbf{c}(t)$ 还是最小二乘问题的最优解：

$$\mathbf{c}(t) = \arg \min_{\tilde{\mathbf{c}} \in \mathbb{R}^N} \left\| f_{\leq t} - \sum_{n=0}^{N-1} \tilde{c}_n g_n^{(t)} \right\|_{L^2(\mu^{(t)})}^2. \quad (17)$$

因此 HiPPO 的“memory”不是一个模糊概念，而是一个明确的在线函数逼近问题。

来源：HiPPO Secs. 2–3.

建议分页位置

2.5 第 13 页：为什么投影系数可以在线更新——从积分定义到线性常微分方程（ODE）

如果每来一个新样本都重新计算式(16)，HiPPO 就没有在线价值。关键结果是：对适当的测度与多项式基，投影系数本身满足一阶线性常微分方程。

以 LegS 为例，HiPPO 原论文把更新写为

$$\frac{d}{dt} \mathbf{c}(t) = -\frac{1}{t} \mathbf{A}_H \mathbf{c}(t) + \frac{1}{t} \mathbf{B}_H f(t), \quad (18)$$

其中按 $n, k = 0, 1, \dots$ 编号，

$$(\mathbf{A}_H)_{nk} = \begin{cases} \sqrt{(2n+1)(2k+1)}, & n > k, \\ n+1, & n = k, \\ 0, & n < k, \end{cases} \quad (\mathbf{B}_H)_n = \sqrt{2n+1}. \quad (19)$$

注意这里采用 HiPPO 原论文的符号： $\mathbf{A}_H$ 的元素写成正数，而真正的状态转移项是 $-\mathbf{A}_H/t$ 。为了与 S4 的连续状态方程(3)统一，后面定义

$$\boxed{\mathbf{A}_{\text{LegS}} = -\mathbf{A}_H, \quad \mathbf{B}_{\text{LegS}} = \mathbf{B}_H,} \quad (20)$$

于是 LegS 可写成

$$\dot{\mathbf{c}}(t) = \frac{1}{t} \mathbf{A}_{\text{LegS}} \mathbf{c}(t) + \frac{1}{t} \mathbf{B}_{\text{LegS}} f(t). \quad (21)$$

这个符号桥接非常重要：后面 S4 Eq. (2) 直接给出的就是 $\mathbf{A}_{\text{LegS}}$ 的负值形式，不应再把两套 $\mathbf{A}$ 混在一起。

来源：HiPPO Theorem 2; S4 Eq. (2).

[建议分页位置](#)

2.6 第 14 页：LegS 为什么有意义——尺度等变、长历史和可控时间尺度

HiPPO-LegS 的核心并不只是式(19)这一张矩阵，而是它对应的记忆性质。设时间缩放后的输入为 $h(t) = f(\alpha t)$ ，HiPPO 证明 LegS 投影满足尺度等变：

$$\text{HiPPO}(h)(t) = \text{HiPPO}(f)(\alpha t). \quad (22)$$

这意味着输入变快或变慢时，状态不是完全失效，而是相应地沿时间轴缩放。

从 Deep SSM 的视角看，离散步长 Δ 还可直接理解为时间尺度控制。对一般连续 SSM

$$\dot{\mathbf{h}} = \mathbf{A}\mathbf{h} + \mathbf{B}u,$$

若同时缩放 $(\mathbf{A}, \mathbf{B}) \mapsto (c\mathbf{A}, c\mathbf{B})$ ，系统演化速度也缩放 c 倍。因而离散化步长 Δ 与矩阵尺度只通过 $(\Delta\mathbf{A}, \Delta\mathbf{B})$ 共同出现。How to Train Your HiPPO 进一步把 $1/\Delta$ 解释为模型典型依赖范围。

对于后面的 S4，这一解释直接导向多时间尺度初始化：让不同通道的 Δ 覆盖一个对数范围，等价于让不同状态通道覆盖从短期到长期的历史。

到这里 HiPPO 已经回答“怎样让有限维状态有原则地保存长历史”。但它还没有回答另一个工程问题：如何用现代硬件高效训练这个状态空间层。

来源：HiPPO Prop. 3; How to Train Your HiPPO Sec. 3.3.

[建议分页位置](#)

2.7 第 15 页：原始 LegS 是时变系统——S4 为什么却使用固定 $\mathbf{A}, \mathbf{B}$

这是学习 S4 时最容易被跳过、但又最影响理解的一点。原始 HiPPO-LegS 是

$$\dot{\mathbf{c}}(t) = \frac{1}{t} \mathbf{A}_{\text{LegS}} \mathbf{c}(t) + \frac{1}{t} \mathbf{B}_{\text{LegS}} f(t),$$

显然是时间变化系统（time-varying system）。S4 却直接把相同的矩阵当作固定 LTI SSM：

$$\dot{\mathbf{h}}(t) = \mathbf{A}_{\text{LegS}} \mathbf{h}(t) + \mathbf{B}_{\text{LegS}} u(t). \quad (23)$$

为什么这样做仍然有“HiPPO memory”的含义？How to Train Your HiPPO 的作用就在于重新解释这件事。

它把一类 LTI SSM

$$\dot{\mathbf{h}} = \mathbf{A}\mathbf{h} + \mathbf{B}u$$

的状态写成卷积形式

$$h_n(t) = \int_{-\infty}^t K_n(t-s)u(s) ds, \quad K_n(\tau) = \mathbf{e}_n^\top e^{\mathbf{A}\tau} \mathbf{B}. \quad (24)$$

如果这些 K_n 恰好构成某个加权希尔伯特空间中的正交基，那么 $h_n(t)$ 就仍然可以解释成过去输入在这些基函数上的投影系数。这类系统被论文称为时间不变正交状态空间模型（time-invariant orthogonal SSM, TOSSM）。

因此，S4 中“固定 HiPPO-LegS”的合理性不是简单把 $1/t$ 删除，而是可以通过另一套时间不变的正交基/测度来重新解释。

来源：How to Train Your HiPPO Secs. 3.1–3.4.

[建议分页位置](#)

2.8 第 16 页：TOSSM 的投影视角——把状态看成一组卷积基系数

设一组因果核满足

$$K_n(\tau) = p_n(\tau) \omega(\tau), \quad \tau \geq 0, \quad (25)$$

其中 $\{p_n\}$ 在权重 ω 下正交。则状态分量

$$h_n(t) = \int_0^\infty K_n(\tau)u(t-\tau) d\tau$$

正是历史 $u(t-\tau)$ 在基 p_n 上的投影系数。

对于固定 HiPPO-LegS，How to Train Your HiPPO 给出的时间不变解释对应

$$K_n(\tau) = L_n(e^{-\tau})e^{-\tau}, \quad p_n(\tau) = L_n(e^{-\tau}), \quad \omega(\tau) = e^{-\tau}, \quad (26)$$

其中 L_n 是适当归一化的 Legendre 多项式。这个结果揭示了一个重要直觉：固定的 LegS SSM 不再对线性时间轴上的 $[0, t]$ 做均匀投影，而是在**指数时间扭曲**下组织过去，因此天然更强调较近历史，同时仍覆盖无限时间范围。

论文把几种 TOSSM 总结为下表。这里额外出现两个只用于对照的缩写：**FouT** 表示平移傅里叶方案（Fourier translated），**LagT** 表示平移拉盖尔方案（Laguerre translated）。它们不是后续 S4 主线必须掌握的模型，只用来说明“不同正交基 + 不同测度”可以产生不同时间记忆结构。

表 7: 时间不变正交状态空间模型 (TOSSM) 的代表性基与测度。依据 How to Train Your HiPPO Table 1 重新整理。

方法	SSM 核 $e^{t\mathbf{A}}\mathbf{B}$ 的基函数形态	测度 $\omega(t)$	典型时间尺度
LegS	$L_n(e^{-t})e^{-t}$	e^{-t}	1
FouT	$(\cos, \sin)(2\pi nt)\mathbf{1}_{[0,1]}$	$\mathbf{1}_{[0,1]}$	1/2
LegT	$L_n(t)\mathbf{1}_{[0,1]}$	$\mathbf{1}_{[0,1]}$	1/2
LagT	$\text{Lag}_n(t)e^{-t/2}$	$\mathbf{1}_{[0,\infty)}$	∞

这一步完成以后，“HiPPO memory”与“固定 LTI SSM”才真正接上。

来源：How to Train Your HiPPO Table 1 and Sec. 3.

[建议分页位置](#)

2.9 第 17 页：从 HiPPO 进入 S4——现在问题只剩“如何快速生成卷积核”

到这里，状态基的意义已经解决。S4 不再需要重新发明一个记忆理论，而是直接拿结构化 HiPPO 矩阵来构造 Deep SSM。问题转化为：

$$\mathbf{K}_L = (\mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}}), \quad (27)$$

怎样在 L 很大时高效求出？

若直接递推矩阵幂，计算量约为

$$\bar{\mathbf{A}}^j \bar{\mathbf{B}}: O(N^2 L),$$

而且要存储中间状态时还会带来 $O(NL)$ 空间。S4 的技术路线因此不是“再改一次状态定义”，而是围绕式(27)重构计算：

change basis $\rightarrow$ NPLR/DPLR $\rightarrow$ 生成函数 $\rightarrow$ Woodbury 恒等式 $\rightarrow$ 柯西核 $\rightarrow$ iFFT.

后面的每一个数学工具都必须能在这条链里找到自己的位置。

来源：S4 Sec. 3 and Algorithm 1.

[建议分页位置](#)

2.10 第 18 页：先学最简单的工具——状态换基不改变输入输出算子

基础知识补充：共轭转置、酉矩阵与相似变换

后面 S4 会频繁出现上标 $*$ 。对复矩阵 $\mathbf{V}$ ， $\mathbf{V}^*$ 表示共轭转置 (conjugate transpose)。若

$$\mathbf{V}^* \mathbf{V} = \mathbf{V} \mathbf{V}^* = \mathbf{I},$$

则称 $\mathbf{V}$ 为酉矩阵 (unitary matrix)；在实数情形它就是正交矩阵。用酉矩阵换基

$$\tilde{\mathbf{A}} = \mathbf{V}^* \mathbf{A} \mathbf{V}$$

不会改变特征值，而且 $\mathbf{V}$ 的条件数为 1，因此数值上非常稳定。所谓“相似变换”就是在不同坐标系中描述同一个线性算子。

设可逆矩阵 $\mathbf{V}$ ，定义新状态

$$\tilde{\mathbf{h}} = \mathbf{V}^{-1} \mathbf{h}.$$

把连续 SSM

$$\dot{\mathbf{h}} = \mathbf{A} \mathbf{h} + \mathbf{B} u, \quad y = \mathbf{C} \mathbf{h}$$

左乘 $\mathbf{V}^{-1}$ ，得到

$$\dot{\tilde{\mathbf{h}}} = (\mathbf{V}^{-1} \mathbf{A} \mathbf{V}) \tilde{\mathbf{h}} + (\mathbf{V}^{-1} \mathbf{B}) u, \quad y = (\mathbf{C} \mathbf{V}) \tilde{\mathbf{h}}. \quad (28)$$

因此参数变换为

$$\tilde{\mathbf{A}} = \mathbf{V}^{-1} \mathbf{A} \mathbf{V}, \quad \tilde{\mathbf{B}} = \mathbf{V}^{-1} \mathbf{B}, \quad \tilde{\mathbf{C}} = \mathbf{C} \mathbf{V}. \quad (29)$$

输入到输出的算子完全不变。S4 把这个基本事实作为后续结构化的入口：如果能找到一个数值稳定的基，使 $\mathbf{A}$ 在该基下非常简单，就能在不改变模型表达的情况下改变计算难度。

最理想情况是

$$\tilde{\mathbf{A}} = \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N),$$

此时

$$\tilde{\mathbf{A}}^j \tilde{\mathbf{B}} = \text{diag}(\lambda_1^j, \dots, \lambda_N^j) \tilde{\mathbf{B}},$$

长卷积核就退化为 Vandermonde 型求值。

来源：S4 Lemma 3.1.

[建议分页位置](#)

2.11 第 19 页：为什么不能直接把 HiPPO 对角化——数值条件数才是障碍

理论上，矩阵只要可对角化，就能写成 $\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^{-1}$ 。但 S4 强调：表达上等价不代表数值上可用。

对于 HiPPO-LegS 矩阵，S4 Lemma 3.2 给出了显式对角化矩阵，其元素会随状态维度快速增大；论文指出其中元素规模可达到指数级，导致 $\mathbf{V}$ 极度病态。即使形式上

$$\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^{-1},$$

计算

$$\tilde{\mathbf{B}} = \mathbf{V}^{-1} \mathbf{B}, \quad \tilde{\mathbf{C}} = \mathbf{C} \mathbf{V}$$

也可能在有限精度下产生巨大误差。

可以用条件数表达这种风险：

$$\kappa(\mathbf{V}) = \|\mathbf{V}\| \|\mathbf{V}^{-1}\| \gg 1 \implies \text{small numerical error } \delta \text{ may be amplified to } O(\kappa(\mathbf{V})\delta). \quad (30)$$

所以 S4 的关键不是“HiPPO 可以对角化”，而是寻找一个由良条件（最好是酉）矩阵完成的结构化变换。

来源：S4 Sec. 3.1 and Lemma 3.2.

建议分页位置

2.12 第 20 页：NPLR——HiPPO 只差一个低秩修正就接近正规矩阵

基础知识补充：正规矩阵为什么比一般矩阵好处理

正规矩阵 $\mathbf{A}_N$ 满足 $\mathbf{A}_N \mathbf{A}_N^* = \mathbf{A}_N^* \mathbf{A}_N$ 。谱定理保证它一定可以被酉矩阵对角化：

$$\mathbf{A}_N = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^*, \quad \mathbf{V}^* \mathbf{V} = \mathbf{I}.$$

这比“一般可对角化”更强：酉换基不会放大数值误差，因此是 S4 想要的稳定计算基础。NPLR 的目的就是把原本不够好处理的 HiPPO 矩阵拆成“一个正规矩阵 + 一个很低秩的修正”。

正规矩阵（normal matrix）满足

$$\mathbf{A} \mathbf{A}^* = \mathbf{A}^* \mathbf{A},$$

由谱定理可被酉矩阵对角化：

$$\mathbf{A}_N = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^*, \quad \mathbf{V}^* \mathbf{V} = \mathbf{I}.$$

因为酉矩阵条件数为 1，它恰好避免上一页的病态换基问题。

S4 的核心观察是：HiPPO 矩阵虽然本身不是正规矩阵，但可以写成

$$\mathbf{A} = \mathbf{A}_N - \mathbf{P} \mathbf{Q}^*, \quad \text{rank}(\mathbf{P} \mathbf{Q}^*) = r \ll N. \quad (31)$$

对 HiPPO-LegS/LegT/LagT，论文证明 $r = 1$ 或 2。把 $\mathbf{A}_N = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^*$ 代入：

$$\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^* - \mathbf{P} \mathbf{Q}^* = \mathbf{V} \left(\mathbf{\Lambda} - (\mathbf{V}^* \mathbf{P})(\mathbf{V}^* \mathbf{Q})^* \right) \mathbf{V}^*. \quad (32)$$

这就是“正规矩阵加低秩修正”（Normal Plus Low-Rank, NPLR）的含义。

它同时保留了两部分：

$$\underbrace{\mathbf{A}_N}_{\text{酉对角化, 数值稳定}} + \underbrace{(-\mathbf{P} \mathbf{Q}^*)}_{\text{低秩修正, 保留 HiPPO 的关键耦合}}.$$

来源：S4 Theorem 1.

建议分页位置

2.13 第 21 页：从 NPLR 到 DPLR——把整个 SSM 一起换到对角基

利用式(29)，令 $\tilde{\mathbf{h}} = \mathbf{V}^* \mathbf{h}$ ，同时定义

$$\tilde{\mathbf{P}} = \mathbf{V}^* \mathbf{P}, \quad \tilde{\mathbf{Q}} = \mathbf{V}^* \mathbf{Q}, \quad \tilde{\mathbf{B}} = \mathbf{V}^* \mathbf{B}, \quad \tilde{\mathbf{C}} = \mathbf{C} \mathbf{V}.$$

则状态矩阵变成

$$\tilde{\mathbf{A}} = \mathbf{\Lambda} - \tilde{\mathbf{P}} \tilde{\mathbf{Q}}^*. \quad (33)$$

这就是“对角矩阵加低秩修正”（Diagonal Plus Low-Rank, DPLR）形式。

注意，换基不是只改 $\mathbf{A}$ 。为了保持输入输出算子完全一致， $\mathbf{B}$ 和 $\mathbf{C}$ 必须同步变成 $\tilde{\mathbf{B}}, \tilde{\mathbf{C}}$ 。这也是课堂推导中最容易丢记号的一步：

$$(\mathbf{A}, \mathbf{B}, \mathbf{C}) \sim (\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{C}}) = (\mathbf{\Lambda} - \tilde{\mathbf{P}} \tilde{\mathbf{Q}}^*, \mathbf{V}^* \mathbf{B}, \mathbf{C} \mathbf{V}).$$

从这一页开始，为简化记号，我们直接在 DPLR 对角基下工作，并重新把带波浪号的变量记回 $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{P}, \mathbf{Q}$ ；即后文默认

$$\mathbf{A} = \mathbf{\Lambda} - \mathbf{P} \mathbf{Q}^*. \quad (34)$$

来源：S4 Eq. (6) and Sec. 3.3.

[建议分页位置](#)

2.14 第 22 页：DPLR 并没有让矩阵幂自动变简单——为什么还需要生成函数

即使 $\mathbf{A} = \mathbf{\Lambda} - \mathbf{P} \mathbf{Q}^*$ ，仍然不能写成

$$\mathbf{A}^j = \mathbf{\Lambda}^j - \mathbf{P} (\mathbf{Q}^*)^j,$$

因为矩阵幂不会对加法分配。直接计算

$$K_j = \mathbf{C} \bar{\mathbf{A}}^j \bar{\mathbf{B}}$$

仍然会被反复矩阵乘困住。

S4 的下一步不是继续处理 $\mathbf{A}^j$ ，而是把整串卷积核一次性变成一个生成函数（generating function）：

$$\hat{K}_L(z) = \sum_{j=0}^{L-1} K_j z^j = \sum_{j=0}^{L-1} \mathbf{C} \bar{\mathbf{A}}^j \bar{\mathbf{B}} z^j. \quad (35)$$

若暂时忽略长度截断，几何级数给出

$$\sum_{j=0}^{\infty} (z \bar{\mathbf{A}})^j = (\mathbf{I} - z \bar{\mathbf{A}})^{-1}, \quad (36)$$

于是无限生成函数为

$$\hat{K}(z) = \mathbf{C} (\mathbf{I} - z \bar{\mathbf{A}})^{-1} \bar{\mathbf{B}}. \quad (37)$$

这里发生了关键转变：原来的难点是矩阵幂，现在变成预解矩阵（resolvent）。低秩修正最适合在“求逆”问题里用 Woodbury 处理。

来源：S4 Algorithm 1 and Appendix C.

[建议分页位置](#)

2.15 第 23 页：为什么在单位根上评价生成函数——快速傅里叶变换（FFT）在哪一步出现

基础知识补充：生成函数、 z 变换与离散傅里叶变换

若一串系数为 $K_0, K_1, \dots$ ，把它们打包成

$$\hat{K}(z) = \sum_{j \geq 0} K_j z^j$$

就得到**生成函数（generating function）**，在离散系统里也可看作 z 域表示。对于长度 L 的有限序列，在 L 个单位根 $z = \omega_m$ 上取值恰好就是离散傅里叶变换（DFT）的采样；因此有了频域值后，可用逆快速傅里叶变换（iFFT）一次恢复全部 K_j 。这一技巧的本质是“同时算整串核”，而不是逐个计算矩阵幂。

长度为 L 的卷积核 $K_0, \dots, K_{L-1}$ 与它在 L 次单位根上的离散傅里叶变换是一一对应的。令

$$\omega_m = e^{-2\pi i m/L}, \quad m = 0, \dots, L-1,$$

则

$$\hat{K}_m = \sum_{j=0}^{L-1} K_j \omega_m^j = \hat{K}_L(\omega_m). \quad (38)$$

知道全部频域值以后，

$$K_j = \frac{1}{L} \sum_{m=0}^{L-1} \hat{K}_m \omega_m^{-j}, \quad (39)$$

即可通过 iFFT 一次性恢复整段卷积核。

因此 S4 的计算顺序是

$$\text{不逐项算 } K_j \implies \text{先并行算 } \hat{K}_L(\omega_m) \implies \text{iFFT.}$$

问题又进一步缩小为：怎样快速计算每个 ω_m 对应的预解矩阵

$$\mathbf{C}(\mathbf{I} - \omega_m \bar{\mathbf{A}})^{-1} \bar{\mathbf{B}}?$$

下一页会把双线性离散化代回去，把这个离散预解矩阵重新写成连续 DPLR 矩阵的逆。

来源：S4 Algorithm 1 Steps 1,4,5.

[建议分页位置](#)

2.16 第 24 页：把双线性离散化代回 resolvent——连续参数为什么又出现

由式(7)

$$\bar{\mathbf{A}} = (\mathbf{I} - \frac{\Delta}{2} \mathbf{A})^{-1} (\mathbf{I} + \frac{\Delta}{2} \mathbf{A}).$$

考虑

$$\mathbf{I} - z \bar{\mathbf{A}} = \mathbf{I} - z (\mathbf{I} - \frac{\Delta}{2} \mathbf{A})^{-1} (\mathbf{I} + \frac{\Delta}{2} \mathbf{A}).$$

左乘 $(\mathbf{I} - \frac{\Delta}{2} \mathbf{A})$ 可得

$$(\mathbf{I} - \frac{\Delta}{2} \mathbf{A})(\mathbf{I} - z \bar{\mathbf{A}}) = (1 - z) \mathbf{I} - \frac{\Delta}{2} (1 + z) \mathbf{A}.$$

因此

$$(\mathbf{I} - z\bar{\mathbf{A}})^{-1} = \left[(1 - z)\mathbf{I} - \frac{\Delta}{2}(1 + z)\mathbf{A} \right]^{-1} (\mathbf{I} - \frac{\Delta}{2}\mathbf{A}). \quad (40)$$

配合

$$\bar{\mathbf{B}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B},$$

中间项消去，得到

$$\boxed{(\mathbf{I} - z\bar{\mathbf{A}})^{-1}\bar{\mathbf{B}} = \frac{2}{1 + z} \left(\frac{2}{\Delta} \frac{1 - z}{1 + z} \mathbf{I} - \mathbf{A} \right)^{-1} \mathbf{B}.} \quad (41)$$

定义

$$s(z) = \frac{2}{\Delta} \frac{1 - z}{1 + z},$$

则核心计算变成

$$(s(z)\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}. \quad (42)$$

现在 $\mathbf{A} = \mathbf{\Lambda} - \mathbf{PQ}^*$ ，所以 $s\mathbf{I} - \mathbf{A}$ 正好是“对角 + 低秩”。Woodbury 在这里出现不是额外技巧，而是被推导自然逼出来的。

来源：S4 Algorithm 1 Step 2 and Appendix C.

[建议分页位置](#)

2.17 第 25 页：Woodbury 恒等式——只解决一个问题：怎样求“对角 + 低秩”的逆

设

$$\mathbf{D}_z = s(z)\mathbf{I} - \mathbf{\Lambda},$$

因为 $\mathbf{A} = \mathbf{\Lambda} - \mathbf{PQ}^*$ ，有

$$s(z)\mathbf{I} - \mathbf{A} = \mathbf{D}_z + \mathbf{PQ}^*.$$

Woodbury 恒等式（Woodbury identity）为

$$(\mathbf{D} + \mathbf{UV}^*)^{-1} = \mathbf{D}^{-1} - \mathbf{D}^{-1}\mathbf{U}(\mathbf{I} + \mathbf{V}^*\mathbf{D}^{-1}\mathbf{U})^{-1}\mathbf{V}^*\mathbf{D}^{-1}. \quad (43)$$

代入 $\mathbf{D} = \mathbf{D}_z$, $\mathbf{U} = \mathbf{P}$, $\mathbf{V} = \mathbf{Q}$:

$$(s\mathbf{I} - \mathbf{A})^{-1} = \mathbf{D}_z^{-1} - \mathbf{D}_z^{-1}\mathbf{P}(\mathbf{I} + \mathbf{Q}^*\mathbf{D}_z^{-1}\mathbf{P})^{-1}\mathbf{Q}^*\mathbf{D}_z^{-1}. \quad (44)$$

真正的优势来自 $r \ll N$:

- $\mathbf{D}_z^{-1}$ 是对角矩阵，逐元素求逆；
- $\mathbf{Q}^*\mathbf{D}_z^{-1}\mathbf{P}$ 只有 $r \times r$ ；
- HiPPO 的 r 只有 1 或 2，因此低秩校正成本很低。

若把最终标量输出 $\mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}$ 展开，所有大规模项都退化成

$$\mathbf{CD}_z^{-1}\mathbf{B}, \quad \mathbf{CD}_z^{-1}\mathbf{P}, \quad \mathbf{Q}^*\mathbf{D}_z^{-1}\mathbf{B}, \quad \mathbf{Q}^*\mathbf{D}_z^{-1}\mathbf{P}.$$

下一页只需要解释这些“对角逆加权和”为何就是柯西核。

来源：S4 Algorithm 1 Step 3.

[建议分页位置](#)

2.18 第 26 页：柯西核——S4 最终把长序列问题化成结构化有理函数求值

因为

$$\mathbf{D}_z^{-1} = \text{diag} \left(\frac{1}{s(z) - \lambda_1}, \dots, \frac{1}{s(z) - \lambda_N} \right),$$

任意向量 $\mathbf{a}, \mathbf{b} \in \mathbb{C}^N$ 都有

$$\mathbf{a}^* \mathbf{D}_z^{-1} \mathbf{b} = \sum_{n=1}^N \frac{a_n^* b_n}{s(z) - \lambda_n}. \quad (45)$$

对所有频率点 $z = \omega_m$ 同时计算时，出现矩阵

$$\mathcal{C}_{mn} = \frac{1}{s(\omega_m) - \lambda_n}, \quad (46)$$

它就是柯西矩阵 (Cauchy matrix) 的标准形式 (差一个简单变量变换)。因此 S4 最终的核心并不是“高速算矩阵幂”，而是

长卷积核 $\rightarrow$ 预解矩阵 $\rightarrow$ DPLR inverse $\rightarrow$ 几次柯西矩阵-向量乘。

S4 Theorem 3 表明，长度 L 的卷积核可以归约为 4 次柯西矩阵-向量乘，在忽略对数因子的记号下达到近似 $\tilde{O}(N + L)$ 的卷积核生成复杂度和 $O(N + L)$ 空间。这才是 NPLR/DPLR 参数化真正服务的目标。

来源：S4 Theorem 3 and Appendix C.

[建议分页位置](#)

2.19 第 27 页：把 S4 Algorithm 1 用已经学过的符号重新写一遍

此时算法不应再作为一张“突然出现的伪代码”，因为每一步已经有数学来源。下面按教学顺序重写 S4 卷积核生成流程。

步骤 1: 连续参数置于 DPLR 基下：

$$\mathbf{A} = \mathbf{\Lambda} - \mathbf{P}\mathbf{Q}^*, \quad (\mathbf{B}, \mathbf{C}) \text{ 已同步换基.}$$

步骤 2: 选择离散步长 Δ ，但不显式形成所有 $\bar{\mathbf{A}}^j$ 。双线性变换只用于把离散预解矩阵化成式(41)。

步骤 3: 在单位根 $\omega_m = e^{-2\pi im/L}$ 上构造

$$s_m = \frac{2}{\Delta} \frac{1 - \omega_m}{1 + \omega_m}.$$

步骤 4: 通过柯西求值并行计算四类项

$$\mathbf{C}\mathbf{D}_m^{-1}\mathbf{B}, \quad \mathbf{C}\mathbf{D}_m^{-1}\mathbf{P}, \quad \mathbf{Q}^*\mathbf{D}_m^{-1}\mathbf{B}, \quad \mathbf{Q}^*\mathbf{D}_m^{-1}\mathbf{P},$$

其中

$$\mathbf{D}_m = s_m \mathbf{I} - \mathbf{\Lambda}.$$

步骤 5: 使用 Woodbury 合并低秩校正：

$$\mathbf{C}(s_m \mathbf{I} - \mathbf{A})^{-1} \mathbf{B} = \mathbf{C}\mathbf{D}_m^{-1} \mathbf{B} - \mathbf{C}\mathbf{D}_m^{-1} \mathbf{P}(\mathbf{I} + \mathbf{Q}^* \mathbf{D}_m^{-1} \mathbf{P})^{-1} \mathbf{Q}^* \mathbf{D}_m^{-1} \mathbf{B}.$$

步骤 6: 恢复双线性变换前面的系数, 得到频域核 $\hat{K}_m$, 再 iFFT:

$$(K_0, \dots, K_{L-1}) = \text{iFFT}(\hat{K}_0, \dots, \hat{K}_{L-1}).$$

算法真正需要学生记住的不是每个实现常数, 而是这一条因果链:

matrix powers are hard $\Rightarrow$ 低秩修正使矩阵求逆可化简 $\Rightarrow$ 柯西结构可快速求值 $\Rightarrow$ iFFT 恢复整段卷积核.

来源: 根据 S4 Algorithm 1 重新翻译和展开。

[建议分页位置](#)

2.20 第 28 页: S4 为什么同时适合训练和推理——复杂度放回三种视图看

S4 的优势来自“同一个参数对象, 采用不同计算视图”。论文给出的复杂度比较可以整理为:

表 8: S4 论文 Table 1 的核心结论 (省略批大小等次要符号, 强调教学要点)。

方法	训练并行	长序列训练	单步推理	主要特点
普通递推	否	受 sequential recurrence 限制	高效	推理自然, 但训练不友好
Attention	是	$O(L^2)$ 级 token mixing	需随历史增长	精确检索强
卷积	是	FFT/卷积友好	每步需处理长核	训练强、生成弱
S4	是	先快速生成 K 后卷积	切换回状态递推	同时保留卷积训练与递推推理

具体地, 训练阶段使用

$$y = \mathbf{K} * u,$$

推理阶段使用

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k, \quad y_k = \mathbf{C}\mathbf{h}_k.$$

参数没有变化, 只是算法实现不同。这也是 S4 之后 Mamba/Mamba-2 一直保留的思想: 模型定义与硬件计算形式可以分离设计。

来源: S4 Table 1 and Theorems 2–3.

[建议分页位置](#)

2.21 第 29 页: 实验告诉我们——真正重要的不只是 NPLR, 而是 HiPPO 初始化

如果把 S4 只讲成“ $\mathbf{A} = \text{diagonal plus low-rank}$ ”, 很容易误解成“只要加一个低秩结构就行”。S4 的消融实验明确表明, 初始化本身非常重要。

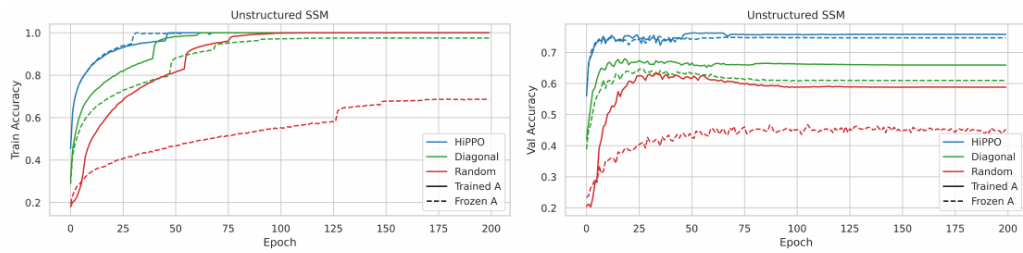


Figure 3: CIFAR-10 classification with unconstrained, real-valued SSMs with various initializations. (Left) Train accuracy. (Right) Validation accuracy.

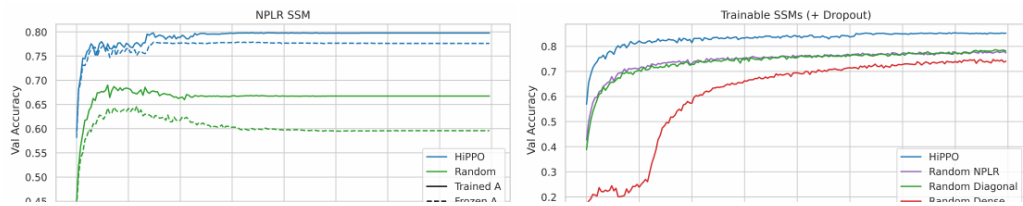


图 4: S4 论文 Fig. 3-4 的主要消融曲线。蓝色 HiPPO 初始化明显优于随机初始化；这说明性能提升不能只归因于 NPLR 参数化，HiPPO 提供的长历史结构是关键组成。

在 unconstrained SSM 中，随机 A 与 HiPPO A 的训练/验证性能就已经有明显差异；在 NPLR 参数化下，随机 NPLR 仍明显不如 HiPPO 初始化。可以用下面的逻辑解释：

$$\underbrace{\text{HiPPO initialization}}_{\text{给出有意义的历史基}} + \underbrace{\text{NPLR/DPLR}}_{\text{让这个基数值稳定、可快速计算}} = \text{S4 的核心组合.}$$

因此，课堂上不应把 NPLR 当成独立魔法；它服务于“保留并高效计算 HiPPO 记忆结构”。

来源：S4 Figs. 3-4.

[建议分页位置](#)

2.22 第 30 页：S4 层不是只有一个线性 SSM——完整 Deep SSM 还需要非线性和残差

前面的推导只说明 SSM token 混合器（mixer）本身。真正堆成深层网络时，一个典型 S4 layer 还会包含归一化、非线性、残差以及通道混合。可把一个简化层写成

$$\mathbf{z} = \text{SSM}(\mathbf{u}; \mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta), \quad (47)$$

$$\mathbf{z}' = \sigma(\mathbf{z}), \quad (48)$$

$$\mathbf{u}' = \mathbf{u} + W_{\text{mix}} \mathbf{z}'. \quad (49)$$

这里真正承担长历史 token mixing 的仍是

$$\text{SSM}(\mathbf{u}) = \mathbf{K} * \mathbf{u}.$$

因此要区分两层含义：

- S4 的理论核心：怎样参数化 A 并快速生成 K；

- **S4 的网络封装**：怎样用 nonlinear/residual/mixing 把多个 SSM 层堆成深层模型。

这一区分后面看 Mamba block 时仍然非常重要：Mamba 的 selective SSM 是核心记忆算子，但完整 block 还包含 projection、causal convolution 和 gate。

来源：S4 Sec. 3.4.

[建议分页位置](#)

2.23 第 31 页：S4D / DSS 为什么会出现——S4 成功以后，哪些结构其实可以删掉

S4 的 DPLR 结构较复杂。后续 DSS 和 S4D 追问：如果把低秩修正去掉，只保留对角状态矩阵，能不能仍然得到强性能？

对于对角连续矩阵

$$\mathbf{A} = \text{diag}(\lambda_1, \dots, \lambda_N),$$

离散后同样是对角：

$$\bar{\mathbf{A}} = \text{diag}(\bar{\lambda}_1, \dots, \bar{\lambda}_N).$$

因此卷积核直接化为

$$K_j = \sum_{n=1}^N C_n B_n \bar{\lambda}_n^j, \quad (50)$$

整段卷积核是一个范德蒙德（Vandermonde）矩阵乘法：

$$\begin{bmatrix} K_0 \\ K_1 \\ \vdots \\ K_{L-1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & \dots & 1 \\ \bar{\lambda}_1 & \dots & \bar{\lambda}_N \\ \vdots & & \vdots \\ \bar{\lambda}_1^{L-1} & \dots & \bar{\lambda}_N^{L-1} \end{bmatrix}}_{\text{Vandermonde matrix}} \begin{bmatrix} C_1 B_1 \\ \vdots \\ C_N B_N \end{bmatrix}. \quad (51)$$

S4D 把“结构、离散化、约束、 $\mathbf{B}$ 是否训练、初始化”这些因素拆开比较：

表 9: S4D Table 1 的参数化选择，重新翻译整理。

方法	$\mathbf{A}$ 结构	核计算	离散化	$\Re(\mathbf{A})$ 约束	初始化
S4	DPLR	柯西结构	双线性变换	exp	HiPPO
DSS	diagonal	softmax	ZOH	无	HiPPO-D
S4D	diagonal	范德蒙德结构	任意	exp/ReLU	多种

来源：S4D Sec. 3.4 and Table 1.

[建议分页位置](#)

2.24 第 33 页：从 S4 到 S5——为什么还要从许多 SISO SSM 改成一个 MIMO SSM

基础知识补充：SISO 与 MIMO

单输入单输出（single-input single-output, SISO）系统每次只有一个标量输入和一个标量输出；多输入多输出（multiple-input multiple-output, MIMO）系统同时处理多个输入/输出通道。S4 常把多个通道视为许多并行 SISO SSM，再在层间混合；S5 则直接把这些通道放进一个 MIMO 状态空间模型中。

标准 S4 层对 H 个特征通道常可看成 H 个相互独立的 SISO SSM，再通过后续混合层（mixing layer）混合通道。若每个 SSM 状态维数为 N ，总隐状态规模约为 HN 。

S5 的出发点是直接使用一个 MIMO SSM：

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t), \quad (52)$$

其中

$$\mathbf{u}(t), \mathbf{y}(t) \in \mathbb{R}^H, \quad \mathbf{x}(t) \in \mathbb{R}^P, \quad \mathbf{B} \in \mathbb{R}^{P \times H}, \quad \mathbf{C} \in \mathbb{R}^{H \times P}.$$

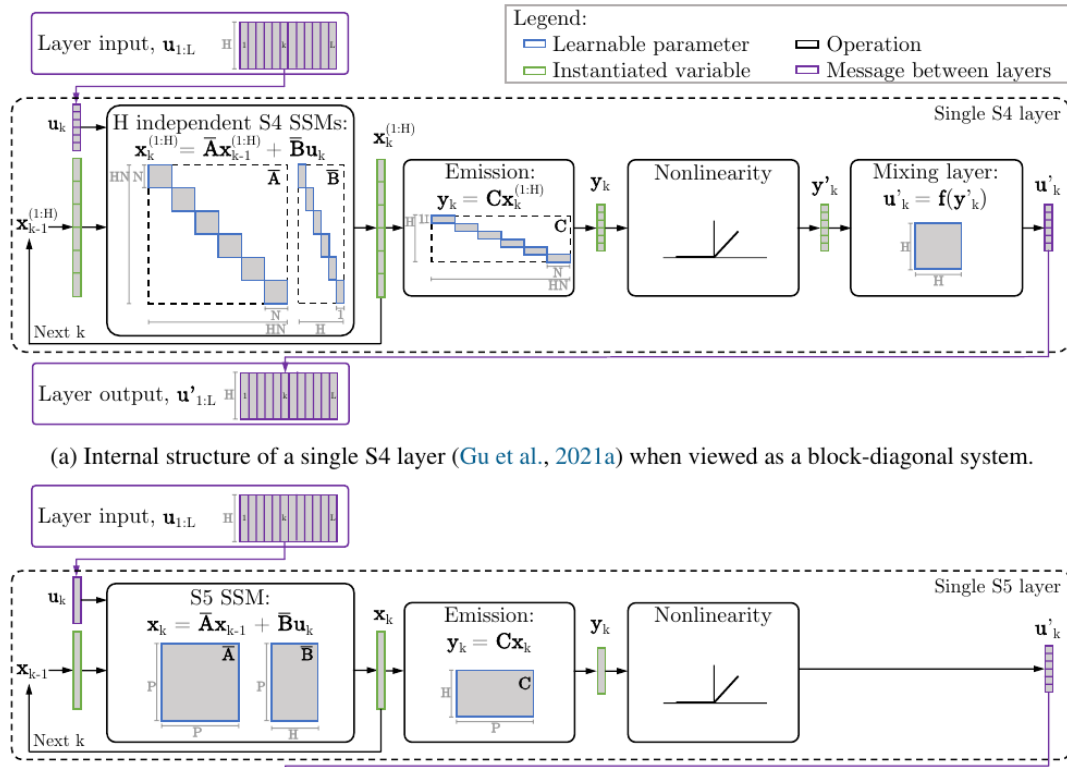


图 5: S4 与 S5 的结构差异（原图来自 S5 Fig. 2）：S4 可视为多个独立 SISO SSM 的块对角（block-diagonal）系统；S5 直接使用一个更紧凑的 MIMO SSM。

S5 的主要结构收益是：如果 $P \ll HN$ ，就能在保留多输入耦合的同时，使时间递推更适合并行扫描。

来源：S5 Sec. 3.1 and Fig. 2.

建议分页位置

2.25 第 35 页：S4、S4D、S5 应该怎样放在同一条学习线上

学完这些模型后，不应该记成三个互不相关的论文名字。它们的关系可以从“状态矩阵结构”和“训练计算方式”两条轴理解：

表 10: S4 家族的教学对照。

模型	状态结构	长序列计算	主要意义
S4	DPLR	柯西结构 + FFT 生成卷积核；推理回到递推	把 HiPPO 长记忆做成可训练、可并行的 Deep SSM
S4D/DSS	diagonal	范德蒙德 / soft-max 等对角核计算	说明很多性能来自良好谱结构与初始化，低秩项可进一步简化
S5	diagonal MIMO	并行扫描（并行扫描）直接计算递推	把多个 SISO 合为紧凑 MIMO，并强化 recurrent / scan 视图

三者共同保持的核心仍然是线性时不变状态空间：

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k, \quad y_k = \mathbf{C}\mathbf{h}_k.$$

也就是说， $\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}$ 对所有 token 都固定。正是这个共同点，构成下一章 Mamba 的切入口。

来源：S4; S4D; S5.

[建议分页位置](#)

2.26 第 36 页：第二章收束——S4 已经解决“长记忆”，但还没有解决“按内容选择记忆”

把前两章压缩成一条连续公式链：

$$\dot{\mathbf{h}} = \mathbf{A}\mathbf{h} + \mathbf{B}u \xrightarrow{\text{discretize}} \mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}u_k \xrightarrow{\text{unroll/readout}} K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}.$$

HiPPO 回答

状态 $\mathbf{h}$ 应该表示什么历史信息？

S4 回答

怎样把这种长记忆状态结构做成可训练、可快速生成 kernel 的 Deep SSM？

S4D/S5 则说明这种结构还能进一步简化或推广。

但它们仍然有一个共同的 LTI 限制：

$$K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}} \quad \text{只依赖距离 } j, \quad \text{不依赖具体 token } u_{k-j}. \quad (53)$$

因此，相同距离的两个 token 会经过同一组动力学权重。如果文本里一个 token 是噪声、另一个是关键实体，固定 SSM 本身没有直接的“看到内容后改变写入/读出规则”的机制。

下一章 Mamba 的问题就可以自然提出：

能否让 $\mathbf{B}, \mathbf{C}, \Delta$ 随当前输入变化，从 fixed long memory 变成 selective memory?

这时才进入 $\mathbf{B}_t, \mathbf{C}_t, \Delta_t$ ，前后所有符号都已经明确来源。

来源：S4 的 LTI kernel；Mamba 的 selective SSM 动机。

3 第三章：有限状态如何按内容选择——Mamba 与选择性状态空间模型

本章只解决一个问题：当 token 的重要性取决于内容而非距离时，状态更新怎样变成 content-aware?

本章给出的答案：让 $\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ 由当前输入产生，使保留、写入、读出都随 token 改变。

3.1 第 1 页：先看 Mamba 到底解决什么——S4 已经能记很长，为什么还不够？

上一章的终点是一个已经可以高效训练的结构化 SSM。统一符号下，离散递推为

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \quad y_t = \mathbf{C}\mathbf{h}_t, \quad (54)$$

而卷积核为

$$K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}, \quad y_t = \sum_{j=0}^t K_j x_{t-j}. \quad (55)$$

S4 的核心贡献是让这串很长的 K_j 能够稳定地产生并高效计算。但是，式(55)也暴露出它的根本限制：

K_j 只依赖距离 j ，不依赖第 $t-j$ 个 token 的内容。

这意味着同样距离的两个 token，在 LTI SSM 看来只能通过同一个核权重进入当前输出。对连续信号，这种“只看时间差”的归纳偏置很自然；但对文本、代码、离散符号而言，重要性往往由内容决定。例如：实体名应当被记住，填充词可以忽略；文档边界出现时旧状态应当清空；同一个 query 出现时应当从上下文中读取与它相关的 value。

因此第三章只围绕一个问题展开：

怎样让状态空间模型根据当前输入，动态决定“写什么、忘多少、读什么”？

Mamba 的答案不是抛弃 SSM，而是在上一章的连续时间参数化上加入输入相关的选择机制（选择机制（selection mechanism）），再设计新的高效计算方式。

表 11: 从 S4 到 Mamba，问题发生了什么变化。

	S4 / S4D	Mamba / S6
状态演化	固定 $\bar{\mathbf{A}}$	$\bar{\mathbf{A}}_t$ 由输入相关步长 Δ_t 间接改变
写入	固定 $\bar{\mathbf{B}}$	$\mathbf{B}_t$ 随 token 变化
读出	固定 $\mathbf{C}$	$\mathbf{C}_t$ 随 token 变化
计算	递推与卷积等价	时变递推，固定 convolution 不再成立
核心能力	长记忆、稳定长核	内容相关的长记忆与过滤

来源：Mamba Sec. 3.1–3.2；S4 Sec. 2.

建议分页位置

3.2 第 2 页：先看失败任务——为什么固定卷积核无法解决选择性复制 (Selective Copying)

在讲 Mamba 的公式之前，先用论文中的两个合成任务看清楚“选择”为什么必要。普通 Copying 任务中，需要记住的 token 与输出位置之间间隔固定。此时即使模型完全不看 token 内容，只学习一个固定延迟，也可以完成任务，因此 LTI recurrence 或全局卷积都能解。

Selective Copying 则把关键 token 的位置随机化。模型必须做到两步：

1. 看到无关 token 时，不让它污染已有状态；
2. 看到关键 token 时，把它写入状态，并跨越任意数量的噪声保存到输出位置。

从固定卷积角度，若

$$y_t = \sum_{j=0}^t K_j x_{t-j},$$

则 K_j 只知道“离当前多远”，不知道 x_{t-j} 是关键 token 还是噪声。随机间隔破坏了“距离就是语义”的捷径。

归纳头 (Induction Heads) 任务更进一步。若上下文曾出现键值对 $A \rightarrow B$ ，后面再次出现 A 时模型应输出 B 。这要求模型不仅保存某些 token，还要根据当前 query 决定什么时候读出关联信息。

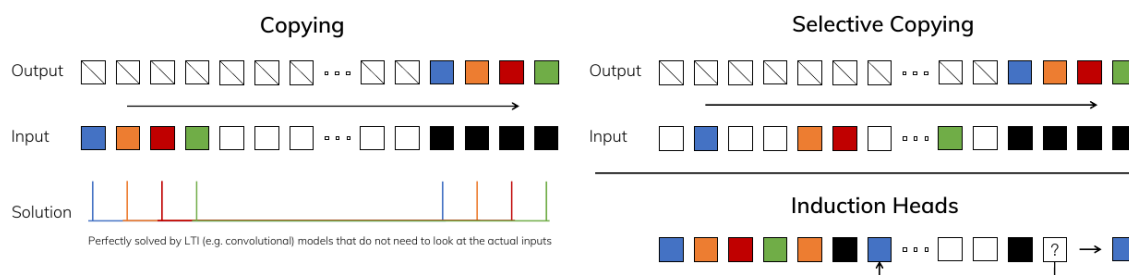


图 6: Mamba 原论文 Figure 2 的任务示意。左：固定间隔 Copying；右上：随机间隔 Selective Copying；右下：Induction Heads。课堂中应先这张图建立“固定 LTI 核缺乏 content-awareness”的直觉，再进入选择性公式。

把这两个任务抽象成状态压缩问题：Attention 通过显式保存整个历史来避免压缩损失；递推模型必须把历史压进有限维状态。于是高效递推模型的关键不是“状态越小越好”，而是

有限状态中只保留未来真正需要的信息。

这正是 Mamba 所说的 selectivity。

来源: Mamba Fig. 2, Sec. 3.1, Sec. 4.1.

建议分页位置

3.3 第 3 页: Mamba 的总体方案——不换掉 SSM, 只让三个关键参数随输入变化

为了与前两章符号保持一致，本章仍从连续 SSM 开始：

$$\dot{\mathbf{h}}(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}(t)x(t), \quad y(t) = \mathbf{C}(t)\mathbf{h}(t). \quad (56)$$

与 S4 最大的区别不是把所有矩阵都做成任意时变系统，而是保留一个共享的结构化 $\mathbf{A}$ ，同时让

$$\Delta_t, \mathbf{B}_t, \mathbf{C}_t$$

由当前输入产生。直观上：

- Δ_t ：改变离散时间尺度，控制旧状态保留多久；
- $\mathbf{B}_t$ ：控制当前 token 如何写入状态；
- $\mathbf{C}_t$ ：控制当前时刻从状态中读出什么。

原论文的整体结构图把这三件事画得非常清楚：输入 x_t 一方面进入状态递推，另一方面经过线性投影（projection）产生选择性参数；这些参数只在快速片上存储 SRAM 中参与离散化和扫描，避免将完整扩展状态长期写回高带宽显存 HBM。

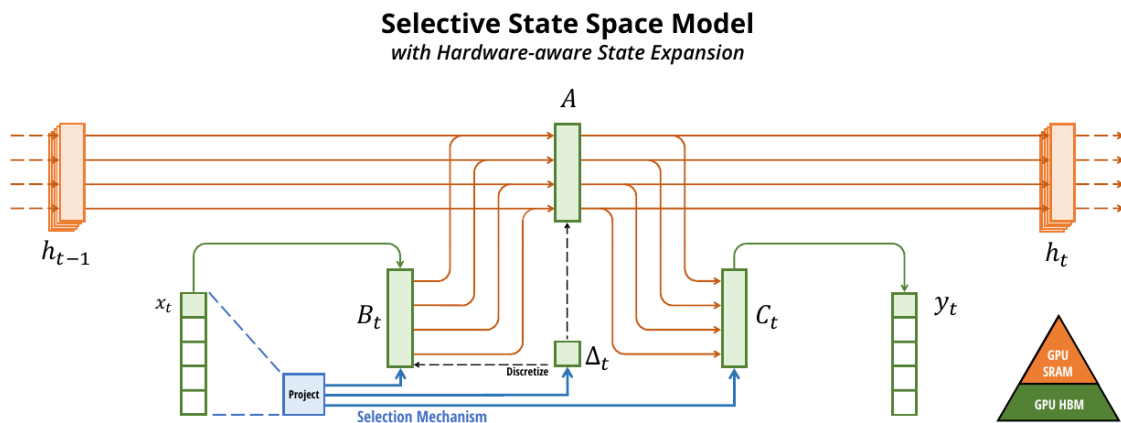


图 7: Mamba 原论文 Figure 1。图中 B_t, C_t, Δ_t 由当前输入产生；共享的 A 提供状态模态；GPU 的 SRAM/HBM 存储层级示意对应后面选择性扫描的硬件设计。

这一页只建立全局认识。后面依次回答四个问题：

参数怎样产生 → 离散化后怎样更新 → 为什么等价于门控 → 时变递推怎样并行计算。

来源：Mamba Fig. 1, Algorithm 2.

建议分页位置

3.4 第 4 页：先固定 Mamba 的张量与符号——避免后面 B 、 D 、状态维度混在一起

前两章讨论的是 SISO SSM。Mamba 实际作用在多通道深度网络中，因此这一页专门固定维度。设

B = batch size, L = 序列长度, D = SSM channel dimension, N = 状态维数。

外部 block 输入记为

$$\mathbf{u} \in \mathbb{R}^{B \times L \times d_{\text{model}}},$$

经过输入投影和局部卷积后，进入 selective SSM 的主分支记为

$$\mathbf{x} \in \mathbb{R}^{\mathcal{B} \times L \times D}.$$

对每一个通道 $d \in \{1, \dots, D\}$ ，都维护一个 N 维状态

$$\mathbf{h}_{t,d} \in \mathbb{R}^N.$$

Mamba 使用对角状态矩阵，因此 $\mathbf{A}$ 可用 $D \times N$ 个数表示；第 d 个通道对应的连续时间对角元素记为 $a_{d,n}$ 。

原论文 Algorithm 1/2 的形状变化可整理为：

表 12: 固定 SSM 与 selective SSM 的参数形状。注意 $\mathcal{B}$ 表示 batch size，粗体 $\mathbf{B}$ 仍表示写入参数。

参数	S4 / 固定 SSM	Mamba S6
$\mathbf{A}$	(D, N) ，训练参数	(D, N) ，仍为共享训练参数
$\mathbf{B}$	(D, N) 或每通道参数	$(\mathcal{B}, L, N)$ ，由输入投影得到
$\mathbf{C}$	(D, N) 或每通道参数	$(\mathcal{B}, L, N)$ ，由输入投影得到
Δ	(D) ，固定	$(\mathcal{B}, L, D)$ ，随 token 与通道变化
$\bar{\mathbf{A}}, \bar{\mathbf{B}}$	(D, N)	$(\mathcal{B}, L, D, N)$ ，离散化后随时间变化

按照论文的默认选择，令当前内部输入为 $\mathbf{x}_t \in \mathbb{R}^D$ ，则

$$\mathbf{B}_t = s_B(\mathbf{x}_t) = W_B \mathbf{x}_t \in \mathbb{R}^N, \quad \mathbf{C}_t = s_C(\mathbf{x}_t) = W_C \mathbf{x}_t \in \mathbb{R}^N, \quad (57)$$

而步长由低维投影再广播到 D 个通道：

$$\Delta_t = \tau_\Delta(\theta_\Delta + s_\Delta(\mathbf{x}_t)), \quad \tau_\Delta(z) = \text{softplus}(z). \quad (58)$$

后面任何出现 $\mathbf{B}_t, \mathbf{C}_t, \Delta_t$ 的地方，都沿用这一页定义，不再重新换记号。

来源：Mamba Algorithm 2, Sec. 3.2.

建议分页位置

3.5 第 5 页：选择性不是一句“门控”——从连续参数严格离散到 $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t$

Mamba 延续前两章的连续时间 SSM 参数化。对第 d 个通道，连续模型写成

$$\dot{\mathbf{h}}_d(t) = \mathbf{A}_d \mathbf{h}_d(t) + \mathbf{B}_t x_d(t), \quad y_d(t) = \mathbf{C}_t^\top \mathbf{h}_d(t), \quad (59)$$

其中 $\mathbf{A}_d = \text{diag}(a_{d,1}, \dots, a_{d,N})$ 。在第 t 个 token 处，步长变成输入相关的 $\Delta_{t,d} > 0$ 。使用 ZOH，得到

$$\bar{\mathbf{A}}_{t,d} = \exp(\Delta_{t,d} \mathbf{A}_d), \quad (60)$$

以及

$$\bar{\mathbf{B}}_{t,d} = (\Delta_{t,d} \mathbf{A}_d)^{-1} (\exp(\Delta_{t,d} \mathbf{A}_d) - \mathbf{I}) \Delta_{t,d} \mathbf{B}_t. \quad (61)$$

因为 $\mathbf{A}_d$ 对角，离散化逐状态模态独立：

$$\bar{a}_{t,d,n} = \exp(\Delta_{t,d} a_{d,n}), \quad (62)$$

$$\bar{b}_{t,d,n} = \frac{\exp(\Delta_{t,d} a_{d,n}) - 1}{a_{d,n}} B_{t,n}, \quad a_{d,n} \neq 0. \quad (63)$$

若 $a_{d,n} \rightarrow 0$ ，由极限

$$\lim_{a \rightarrow 0} \frac{e^{\Delta a} - 1}{a} = \Delta$$

得到 $\bar{b}_{t,d,n} \rightarrow \Delta_{t,d} B_{t,n}$ 。

于是每个通道、每个状态模态的实际递推是

$$h_{t,d,n} = \bar{a}_{t,d,n} h_{t-1,d,n} + \bar{b}_{t,d,n} x_{t,d}, \quad (64)$$

最终读出

$$y_{t,d} = \sum_{n=1}^N C_{t,n} h_{t,d,n}. \quad (65)$$

这四个式子是 Mamba selective SSM 最核心的数学对象。后面所谓 “skip” “remember” “reset”，都可以在 $\bar{a}_{t,d,n}, \bar{b}_{t,d,n}, C_{t,n}$ 的数值变化上直接解释。

来源：Mamba Eq. (4), Algorithm 2.

[建议分页位置](#)

3.6 第 6 页：为什么 Δ_t 是最关键的选择参数——从 ZOH 严格推出 RNN gate

Mamba 论文专门给出一个一维定理，把输入相关离散化与经典门控联系起来。取

$$N = 1, \quad A = -1, \quad B = 1,$$

连续系统就是 leaky integrator：

$$\dot{h}(t) = -h(t) + x(t). \quad (66)$$

令

$$\ell_t = b_\Delta + w_\Delta^\top \mathbf{x}_t, \quad \Delta_t = \text{softplus}(\ell_t) = \log(1 + e^{\ell_t}). \quad (67)$$

根据 ZOH,

$$\bar{A}_t = e^{-\Delta_t} = e^{-\log(1+e^{\ell_t})} = \frac{1}{1+e^{\ell_t}} = 1 - \sigma(\ell_t), \quad (68)$$

$$\bar{B}_t = 1 - e^{-\Delta_t} = \sigma(\ell_t). \quad (69)$$

定义

$$g_t = \sigma(\ell_t),$$

则 selective SSM 递推精确化为

$$h_t = (1 - g_t) h_{t-1} + g_t x_t. \quad (70)$$

因此 Δ_t 不是随意加进来的一个 “时间参数”，而是经连续时间离散化自然地产生了门控行为：

- $g_t \approx 0$: $h_t \approx h_{t-1}$ ，当前输入几乎被忽略；

- $g_t \approx 1$: $h_t \approx x_t$, 旧状态被快速覆盖;
- 中间值: 在旧状态与当前输入之间进行连续可学习的插值。

而在多维状态下, $\Delta_{t,d}$ 同时调节所有 N 个模态的时间尺度:

$$\bar{a}_{t,d,n} = e^{\Delta_{t,d} a_{d,n}}.$$

这解释了 Mamba 消融中 Δ 往往是最重要的 selective parameter: 它直接控制状态的保留与重置。

来源: Mamba Theorem 1, Appendix C.

[建议分页位置](#)

3.7 第 7 页: $\mathbf{B}_t$ 和 $\mathbf{C}_t$ 分别解决什么——把“写入”和“读出”从固定矩阵中解耦出来

Δ_t 主要改变时间尺度, 但它并不能完全决定“新信息写到状态的哪个方向”。这由 $\mathbf{B}_t$ 决定。由式(64), 当前输入 $x_{t,d}$ 对第 n 个状态模态的增量为

$$\Delta h_{t,d,n}^{\text{write}} = \bar{b}_{t,d,n} x_{t,d}.$$

若某个 token 与模态 n 无关, 模型可以让 $B_{t,n}$ 很小; 若需要把该 token 写入特定模态, 则可以增大对应 $B_{t,n}$ 。因此

$\mathbf{B}_t$: 决定当前 token 写入状态的方向与幅度。

读出端则由 $\mathbf{C}_t$ 决定。状态 $\mathbf{h}_{t,d}$ 中可能同时编码多种历史信息, 但当前输出只需要其中一部分:

$$y_{t,d} = \mathbf{C}_t^\top \mathbf{h}_{t,d}.$$

于是

$\mathbf{C}_t$: 根据当前上下文选择从状态中读出哪些分量。

三种 selective parameter 的功能可统一整理为:

表 13: Mamba 三类选择参数与状态操作的对应关系。

参数	直接影响	机械解释
Δ_t	$\bar{\mathbf{A}}_t = e^{\Delta_t \mathbf{A}}$	控制旧状态保持/衰减的时间常数, 可实现忽略与 reset
$\mathbf{B}_t$	$\bar{\mathbf{B}}_t x_t$	决定当前 token 写入哪些状态模态
$\mathbf{C}_t$	$y_t = \mathbf{C}_t^\top h_t$	决定当前时刻从状态里读取哪些历史信息

这三者共同解释论文讨论的三种现象: variable spacing、filtering context、boundary resetting。特别是当 $\Delta_t \rightarrow \infty$ 且 $\Re(a_{d,n}) < 0$ 时,

$$\bar{a}_{t,d,n} = e^{\Delta_t a_{d,n}} \rightarrow 0,$$

模型可以在文档/episode 边界上机械地清除旧状态。

来源: Mamba Sec. 3.5.2.

[建议分页位置](#)

3.8 第 8 页：为什么 Mamba 失去了固定卷积——时变系统的“核”依赖整条路径

上一章的 LTI SSM 可以写成固定卷积，是因为每一步都使用同一个 $\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}$ 。Mamba 变成时变系统以后，递推为

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t x_t, \quad y_t = \mathbf{C}_t^\top \mathbf{h}_t. \quad (71)$$

从零初始状态展开：

$$\mathbf{h}_t = \bar{\mathbf{B}}_t x_t + \bar{\mathbf{A}}_t \bar{\mathbf{B}}_{t-1} x_{t-1} + \bar{\mathbf{A}}_t \bar{\mathbf{A}}_{t-1} \bar{\mathbf{B}}_{t-2} x_{t-2} + \cdots \quad (72)$$

$$= \sum_{i=0}^t \left(\prod_{k=i+1}^t \bar{\mathbf{A}}_k \right) \bar{\mathbf{B}}_i x_i. \quad (73)$$

于是输出为

$$y_t = \sum_{i=0}^t \underbrace{\mathbf{C}_t^\top \left(\prod_{k=i+1}^t \bar{\mathbf{A}}_k \right) \bar{\mathbf{B}}_i}_{M_{t,i}} x_i. \quad (74)$$

这里 $M_{t,i}$ 不再只由距离 $t-i$ 决定，而是依赖从 i 到 t 的全部中间转移 $\bar{\mathbf{A}}_{i+1}, \dots, \bar{\mathbf{A}}_t$ ，还依赖起点 $\bar{\mathbf{B}}_i$ 和终点 $\mathbf{C}_t$ 。因此一般不能写成

$$M_{t,i} = K_{t-i}.$$

也就没有固定 convolution kernel 可以提前生成。

这一步非常关键：

Mamba 的表达力提升与固定卷积并行路径的消失，是同一个改动的两面。

所以接下来真正的工程问题不是“怎样再求一个卷积核”，而是“怎样把时变 recurrence 做成 GPU 上足够快的并行操作”。

来源：Mamba Sec. 3.2–3.3.

[建议分页位置](#)

3.9 第 9 页：选择性扫描的数学核心——仿射状态更新具有结合律，可以做并行前缀扫描

先忽略通道和状态下标，把一步递推写成仿射变换：

$$h_t = a_t h_{t-1} + b_t. \quad (75)$$

其中在 Mamba 中可理解为

$$a_t = \bar{\mathbf{A}}_t, \quad b_t = \bar{\mathbf{B}}_t x_t.$$

把每一步表示成二元组

$$q_t = (a_t, b_t),$$

并定义二元运算

$$(a_2, b_2) \otimes (a_1, b_1) = (a_2 a_1, b_2 + a_2 b_1). \quad (76)$$

这正对应两个仿射映射的复合：

$$h \xrightarrow{(a_1, b_1)} a_1 h + b_1 \xrightarrow{(a_2, b_2)} a_2 a_1 h + a_2 b_1 + b_2.$$

更重要的是， $\otimes$ 满足结合律。直接计算：

$$(a_3, b_3) \otimes [(a_2, b_2) \otimes (a_1, b_1)] = (a_3 a_2 a_1, b_3 + a_3 b_2 + a_3 a_2 b_1), \quad (77)$$

$$[(a_3, b_3) \otimes (a_2, b_2)] \otimes (a_1, b_1) = (a_3 a_2 a_1, b_3 + a_3 b_2 + a_3 a_2 b_1). \quad (78)$$

因此所有前缀

$$Q_t = q_t \otimes q_{t-1} \otimes \cdots \otimes q_0$$

可以使用并行前缀扫描（parallel prefix scan）计算，而不必严格逐 token 串行。

在实际 Mamba 中， a_t 是按通道、状态模态的逐元素（elementwise）转移，结合律逐元素成立。于是选择性扫描的核心并不是“把 recurrence 变成 convolution”，而是

利用仿射状态更新的结合律并行计算所有 prefix state。

来源：Mamba Sec. 3.3；并行扫描思想与 S5 的 scan 路线相承接。

[建议分页位置](#)

3.10 第 10 页：数学上能扫描还不够——Mamba 为什么必须做硬件感知的融合与重计算

基础知识补充：GPU 中的 HBM、SRAM 与“算术强度”

GPU 的高带宽显存（HBM）容量大但访问相对慢；芯片上的 SRAM/共享存储容量小但访问快。若一个算法反复把巨大的中间状态写回 HBM，再马上读回来，速度可能主要受内存带宽而不是浮点运算次数限制。Mamba 的硬件感知设计因此强调：尽量在 SRAM 中完成离散化、扫描和读出，只把必要输入/输出经过 HBM。

若直接实现式(64)，完整中间状态张量的规模为

$$\mathcal{B} \times L \times D \times N.$$

当 N 取几十到上百时，这比输入/输出 $\mathcal{B} \times L \times D$ 大一个 N 倍。若每一步都把 $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t$ 和 h_t 写回 GPU HBM，瓶颈会从 FLOPs 转移到内存带宽。

Mamba 的硬件感知的选择性扫描（hardware-aware 选择性扫描）使用三件事：

1. **算子融合（kernel fusion）**：从 HBM 读入 $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ 后，在 SRAM 中完成离散化、扫描和读出，避免物化巨大的 $\bar{\mathbf{A}}, \bar{\mathbf{B}}$ ；
2. **并行扫描（并行扫描）**：利用上一页的结合律减少严格序列依赖；
3. **Recomputation**：前向不保存所有中间状态，反向时重新计算，以减少 activation memory。

可把数据流概括成：

$$\text{HBM: } (\mathbf{x}, \Delta, \mathbf{A}, \mathbf{B}, \mathbf{C}) \longrightarrow \text{SRAM: 离散化} + \text{扫描} + \text{读出} \longrightarrow \text{HBM: } \mathbf{y}.$$

论文给出的复杂度比较是：朴素递推（naive recurrence）的 FLOPs 为

$$\mathcal{O}(\mathcal{B}LDN),$$

固定 convolution 路径约为

$$\mathcal{O}(BLD \log L).$$

前者的渐近式虽然含 N ，但当 N 不是特别大时具有较低常数；真正困难在于内存访问与并行组织，所以 Mamba 的贡献包含了系统实现，而不仅是模型公式。

教学上应把“选择性扫描”拆成两层理解：第一层是结合律带来的并行扫描；第二层是 SRAM/HBM 层级下的融合计算 + 重计算（fusion + recomputation）。只讲其中一层都会把 Mamba 的效率来源讲残缺。

来源：Mamba Sec. 3.3, Appendix D.

[建议分页位置](#)

3.11 第 11 页：Mamba 模块（Mamba Block）——selective SSM 如何成为完整神经网络层

Selective SSM 本身只是 sequence transformation。完整 Mamba block 还需要局部特征处理、非线性门控和输入输出投影。设 block 输入为

$$\mathbf{u} \in \mathbb{R}^{L \times d_{\text{model}}}.$$

输入投影产生两条分支：

$$\mathbf{x} = \mathbf{u}W_x, \quad \mathbf{z} = \mathbf{u}W_z, \quad (79)$$

主分支先经过 depthwise causal convolution 与 SiLU：

$$\tilde{\mathbf{x}} = \text{SiLU}(\text{Conv1D}(\mathbf{x})). \quad (80)$$

再由 $\tilde{\mathbf{x}}_t$ 产生选择性参数

$$(\mathbf{B}_t, \mathbf{C}_t, \Delta_t) = \text{Select}(\tilde{\mathbf{x}}_t),$$

执行 selective SSM：

$$\mathbf{s} = \text{S6}(\tilde{\mathbf{x}}; \mathbf{A}, \mathbf{B}_{1:L}, \mathbf{C}_{1:L}, \Delta_{1:L}). \quad (81)$$

旁路作为 gate：

$$\mathbf{g} = \text{SiLU}(\mathbf{z}), \quad \mathbf{y} = (\mathbf{s} \odot \mathbf{g})W_o. \quad (82)$$

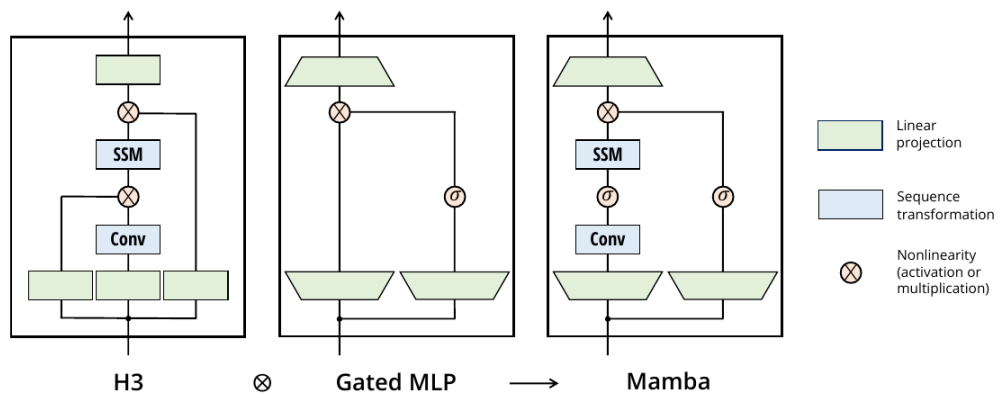


图 8: Mamba 原论文 Figure 3。Mamba 可以看成把 H3 式 SSM 分支与 gated MLP 融合成一个同构 block：局部 Conv 处理短程模式，selective SSM 维护长历史，乘法分支提供额外门控。

从参数量看，block 的大头仍在 W_x, W_z, W_o 等线性投影，而内层 SSM 参数相对较少。这一点解释了为什么 Mamba 的主要创新集中在序列混合器（token mixer），而不是通过大量额外参数换取性能。

来源：Mamba Fig. 3, Sec. 3.4.

建议分页位置

3.12 第 12 页：把 S4 与 S6 放到同一张算法表——真正改变的是“谁开始依赖 token”

为了帮助学生把第三章与第二章接起来，下面不用原论文截图，而是把 Algorithm 1/2 重新翻译并按同一符号写成一张表。

表 14: Fixed SSM (S4) 与 Selective SSM (S6) 的计算流程对照。

步骤	S4 / 固定 SSM	S6 / selective SSM
1	$\mathbf{A} \in \mathbb{R}^{D \times N}$: 共享训练参数	$\mathbf{A} \in \mathbb{R}^{D \times N}$: 仍为共享训练参数
2	$\mathbf{B}$: 固定参数	$\mathbf{B}_t = s_B(\mathbf{x}_t)$: 逐 token 生成
3	$\mathbf{C}$: 固定参数	$\mathbf{C}_t = s_C(\mathbf{x}_t)$: 逐 token 生成
4	Δ : 固定正步长	$\Delta_t = \text{softplus}(\theta_\Delta + s_\Delta(\mathbf{x}_t))$
5	$(\mathbf{A}, \mathbf{B}, \Delta) \mapsto (\bar{\mathbf{A}}, \bar{\mathbf{B}})$	$(\mathbf{A}, \mathbf{B}_t, \Delta_t) \mapsto (\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t)$
6	recurrence 或 convolution	时变递推 / scan

数学上最核心的差别可以压缩成

$$\begin{aligned} \text{S4: } M_{t,i} &= \mathbf{C} \bar{\mathbf{A}}^{t-i} \bar{\mathbf{B}}, \\ \text{S6: } M_{t,i} &= \mathbf{C}_t^\top \bar{\mathbf{A}}_t \bar{\mathbf{A}}_{t-1} \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i. \end{aligned}$$

前者是 Toeplitz/卷积式的距离核；后者是路径依赖的时变混合器。后面 Mamba-2 的整个 SSD 推导，就是从第二行 $M_{t,i}$ 出发，把它作为一个结构化下三角矩阵重新理解。

来源：Mamba Algorithm 1-2.

建议分页位置

3.13 第 13 页：实验怎样验证 “selectivity” 而不是只验证 “架构更大”

Selective Copying 的消融是理解 Mamba 最重要的一张表。关键比较不是 “最终 Mamba vs S4” 这一行，而是尽量固定外层 architecture，只替换内部 mixer。

表 15: Mamba 原论文 Table 1 (Selective Copying) 重排。数值为准确率 (%)。

模型名	外层架构	内层 mixer	是否 selective	Acc.
S4	no gate	S4	否	18.3
-	no gate	S6	是	97.0
H3	H3	S4	否	57.0
Hyena	H3	Hyena	否	30.1
-	H3	S6	是	99.7
-	Mamba	S4	否	56.4
-	Mamba	Hyena	否	28.4
Mamba	Mamba	S6	是	99.8

第一组比较最干净：没有外层 gate 时，仅把 S4 换成 S6，准确率从 18.3% 跳到 97.0%。这直接支持 “输入相关状态参数” 是解决 Selective Copying 的主因，而不是 Mamba block 其它门控结构。

Induction Heads 则验证了更接近 associative recall 的能力。模型在长度 256 上训练，再向长序列外推；论文中 Mamba 的曲线在极长测试长度下仍保持高准确率，而多种位置编码 attention 与其它 mixer 很快退化。

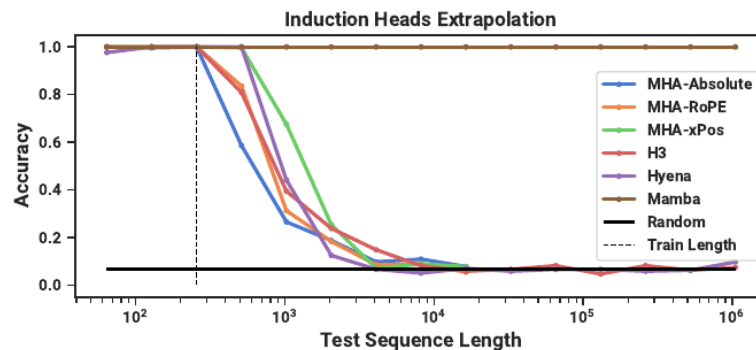


图 9: Mamba 原论文 Table 2 的 Induction Heads 外推曲线。虚线表示训练长度；横轴测试长度按对数增长。

需要强调：这些合成任务证明的是 “selection mechanism 能解决固定 LTI 的特定失败模式”，并不等价于证明有限状态可以无损替代 attention 的所有精确检索能力。这个边界会在后续 DeltaNet/Hybrid

章节再次出现。

来源：Mamba Table 1–2, Sec. 4.1.

[建议分页位置](#)

3.14 第 14 页：第三章收束——Mamba 已经解决 content-awareness，但计算和状态容量留下两个新问题

到这里可以把 Mamba 的改动压缩成一条连续链：

$$\underbrace{K_j = \mathbf{C} \bar{\mathbf{A}}^j \bar{\mathbf{B}}}_{\text{S4 固定距离核}} \implies \underbrace{M_{t,i} = \mathbf{C}_t^\top \left(\prod_{k=i+1}^t \bar{\mathbf{A}}_k \right) \bar{\mathbf{B}}_i}_{\text{Mamba 内容相关时变核}}.$$

其中

$$\bar{\mathbf{A}}_t = e^{\Delta_t \mathbf{A}}, \quad \mathbf{B}_t = s_B(\mathbf{x}_t), \quad \mathbf{C}_t = s_C(\mathbf{x}_t), \quad \Delta_t = \text{softplus}(\theta_\Delta + s_\Delta(\mathbf{x}_t)).$$

Mamba 由此获得了三类能力：过滤无关 token、在需要时重置历史、按当前上下文改变写入与读出。

但两个问题会自然把课程带到下一章：

1. **计算问题：**Mamba-1 需要融合式选择性扫描，主要是 elementwise recurrence，未充分利用 GPU/TPU 最擅长的大矩阵乘；状态维度 N 增大时扫描成本几乎线性上升。
2. **表达/容量问题：**更大的 N 往往能保存更多信息，但在 Mamba-1 中增大 N 会直接增加 scan 负担，因而“提高状态容量”和“保持硬件效率”发生冲突。

因此 Mamba-2 不只是“把 Mamba 再优化一个 kernel”，而是提出一个新的问题：

能否把时变 SSM 重写成一种既保留 recurrence，又能大量使用矩阵乘法结构？

答案就是 Structured State Space 对偶性（Duality）（SSD）。

来源：Mamba Sec. 5；Mamba-2 Introduction.

[建议分页位置](#)

4 第四章：选择性状态如何高效计算——Mamba-2 与结构化状态空间对偶性

本章只解决一个问题：选择性递推已经有表达力后，怎样把它改写成 GPU 友好的矩阵计算而不丢掉线性推理？

本章给出的答案：先把整段时变递推写成一个下三角混合矩阵，识别其中由有限状态产生的低秩结构，再把计算拆成“块内矩阵乘 + 块间状态传递”。

4.1 第 1 页：先看 Mamba-2 要优化什么——不是重新发明 selectivity，而是重构计算视图

第三章已经得到 selective recurrence

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t x_t, \quad y_t = \mathbf{C}_t^\top \mathbf{h}_t. \quad (83)$$

Mamba-1 的选择性扫描能在线性时间内计算它，但训练时存在两个系统层面的限制：

- 扫描以逐元素状态更新为主，不像 attention 那样天然转成大规模 GEMM，难以充分利用张量核心（Tensor Core）；
- 状态维度 N 越大，scan 的算术和内存成本越高，而更大的 N 又往往意味着更强的信息容量。

Mamba-2 的核心视角是：**不要只把 SSM 看成 recurrence，把整段序列到整段序列的映射直接写成一个矩阵。**一旦这样做，SSM 会暴露出一种经典的低秩结构——semiseparable matrix。

这一章的学习路线因此是：

recurrence $\rightarrow$ sequence-mixing matrix $\mathbf{M} \rightarrow$ semiseparable structure
 $\rightarrow$ attention dual form $\rightarrow$ 分块通用矩阵乘（chunked GEMM）算法
 $\rightarrow$ Mamba-2 block.

特别注意：本章继续沿用课程前面的离散记号 $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t, \mathbf{C}_t$ 。Mamba-2 原论文为了简化排版把离散参数直接记作 A_t, B_t, C_t ；这里不跟随那次记号变化，以保证学生前后阅读连贯。

来源：Mamba-2 Sec. 2, Introduction.

[建议分页位置](#)

4.2 第 2 页：第一步——把时变 SSM 完整展开成一个下三角矩阵乘法

从式(83)出发，设 $\mathbf{h}_{-1} = 0$ 。逐步展开：

$$\mathbf{h}_0 = \bar{\mathbf{B}}_0 x_0, \quad (84)$$

$$\mathbf{h}_1 = \bar{\mathbf{A}}_1 \bar{\mathbf{B}}_0 x_0 + \bar{\mathbf{B}}_1 x_1, \quad (85)$$

$$\mathbf{h}_2 = \bar{\mathbf{A}}_2 \bar{\mathbf{A}}_1 \bar{\mathbf{B}}_0 x_0 + \bar{\mathbf{A}}_2 \bar{\mathbf{B}}_1 x_1 + \bar{\mathbf{B}}_2 x_2. \quad (86)$$

一般地，定义有序乘积

$$\bar{\mathbf{A}}_{t:i+1} := \bar{\mathbf{A}}_t \bar{\mathbf{A}}_{t-1} \cdots \bar{\mathbf{A}}_{i+1}, \quad \bar{\mathbf{A}}_{i:i+1} := \mathbf{I},$$

则

$$\mathbf{h}_t = \sum_{i=0}^t \bar{\mathbf{A}}_{t:i+1} \bar{\mathbf{B}}_i x_i. \quad (87)$$

再乘以 $\mathbf{C}_t^\top$ ：

$$y_t = \sum_{i=0}^t \mathbf{C}_t^\top \bar{\mathbf{A}}_{t:i+1} \bar{\mathbf{B}}_i x_i. \quad (88)$$

定义序列混合矩阵

$$M_{t,i} = \begin{cases} \mathbf{C}_t^\top \bar{\mathbf{A}}_{t:i+1} \bar{\mathbf{B}}_i, & t \geq i, \\ 0, & t < i, \end{cases} \quad (89)$$

则整段序列可写成

$$\mathbf{y} = \mathbf{M}\mathbf{x}. \quad (90)$$

这一步看起来只是把 recurrence “展开了一遍”，但它改变了后面的计算语言：现在 SSM 与 attention 一样，都是一个作用在序列轴上的矩阵混合器（matrix mixer）。

来源：Mamba-2 Eq. (3), Sec. 3.1.

建议分页位置

4.3 第 3 页：把 $\mathbf{M}$ 写出来——学生先从一个 $T = 4$ 的显式矩阵看结构

令序列长度 $T = 4$ 。由式(89),

$$\mathbf{M} = \begin{bmatrix} \mathbf{C}_0^\top \bar{\mathbf{B}}_0 & 0 & 0 & 0 \\ \mathbf{C}_1^\top \bar{\mathbf{A}}_1 \bar{\mathbf{B}}_0 & \mathbf{C}_1^\top \bar{\mathbf{B}}_1 & 0 & 0 \\ \mathbf{C}_2^\top \bar{\mathbf{A}}_2 \bar{\mathbf{A}}_1 \bar{\mathbf{B}}_0 & \mathbf{C}_2^\top \bar{\mathbf{A}}_2 \bar{\mathbf{B}}_1 & \mathbf{C}_2^\top \bar{\mathbf{B}}_2 & 0 \\ \mathbf{C}_3^\top \bar{\mathbf{A}}_3 \bar{\mathbf{A}}_2 \bar{\mathbf{A}}_1 \bar{\mathbf{B}}_0 & \mathbf{C}_3^\top \bar{\mathbf{A}}_3 \bar{\mathbf{A}}_2 \bar{\mathbf{B}}_1 & \mathbf{C}_3^\top \bar{\mathbf{A}}_3 \bar{\mathbf{B}}_2 & \mathbf{C}_3^\top \bar{\mathbf{B}}_3 \end{bmatrix}.$$

由于因果性， $\mathbf{M}$ 必然是下三角矩阵。更重要的是，看左下角跨越某个时间边界的块。例如把边界放在 $k = 1$ ，取行 $t \geq 2$ 、列 $i \leq 1$ ：

$$M_{t,i} = \underbrace{\mathbf{C}_t^\top \bar{\mathbf{A}}_t \cdots \bar{\mathbf{A}}_2}_{\ell_t^\top \in \mathbb{R}^{1 \times N}} \underbrace{\bar{\mathbf{A}}_1 \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i}_{r_i \in \mathbb{R}^N}.$$

因此整个跨边界块都可以写成

$$\mathbf{M}_{2:4, 0:2} = \mathbf{L}\mathbf{R}, \quad \text{rank}(\mathbf{M}_{2:4, 0:2}) \leq N. \quad (91)$$

这就是半可分结构的核心直觉：远距离历史对未来的所有影响，必须通过 N 维状态这个“瓶颈”跨越时间边界，所以跨边界块的秩最多是 N 。

不要先背 semiseparable 的定义。先看到式(91)：状态只有 N 维，因此任何跨时间边界的信息传递都只能通过一个秩不超过 N 的因子化。这就是结构产生的原因。

来源：Mamba-2 Sec. 3.1–3.2.

建议分页位置

4.4 第 4 页：再给严格定义——什么是 N -半可分矩阵，为什么 SSM 天然属于它

基础知识补充：矩阵秩与低秩分解

矩阵的秩 (**rank**) 可以理解为其真正包含的独立方向数量。若一个大矩阵能写成

$$\mathbf{M}_{21} = \mathbf{O}\mathbf{H}^\top, \quad \mathbf{O} \in \mathbb{R}^{m \times N}, \mathbf{H} \in \mathbb{R}^{n \times N},$$

那么 $\text{rank}(\mathbf{M}_{21}) \leq N$ 。这意味着即使 $\mathbf{M}_{21}$ 很大，它跨越两个时间区段传递的信息也只有 N 个独立通道。SSM 的状态恰好只有 N 维，因此跨时间边界的块天然具有低秩结构。

有了上一页直觉以后，再给定义就不突兀。对下三角矩阵 $\mathbf{M} \in \mathbb{R}^{T \times T}$ ，若其下三角区域中任意完全位于对角线下方的子矩阵秩都不超过 N ，则称 $\mathbf{M}$ 为 N -半可分矩阵 (N -semiseparable matrix)。

Mamba-2 使用的序列半可分 (sequentially semiseparable, SSS) 表示直接写成

$$M_{j,i} = \mathbf{C}_j^\top \bar{\mathbf{A}}_j \bar{\mathbf{A}}_{j-1} \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i, \quad j \geq i. \quad (92)$$

这与式(89)完全相同。因此：

$$\text{SSM}(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}) \equiv \text{SSS}(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}).$$

为什么任意跨边界块秩 $\leq N$ ？选边界 k ，对所有 $j > k \geq i$ ：

$$M_{j,i} = \mathbf{C}_j^\top \bar{\mathbf{A}}_j \cdots \bar{\mathbf{A}}_{k+1} \bar{\mathbf{A}}_k \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i \quad (93)$$

$$= \underbrace{(\mathbf{C}_j^\top \bar{\mathbf{A}}_j \cdots \bar{\mathbf{A}}_{k+1})}_{\ell_j^\top} \underbrace{(\bar{\mathbf{A}}_k \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i)}_{r_i}. \quad (94)$$

堆叠所有 j 和 i 后就是

$$\mathbf{M}_{\text{block}} = \mathbf{L}\mathbf{R}, \quad \mathbf{L} \in \mathbb{R}^{m \times N}, \quad \mathbf{R} \in \mathbb{R}^{N \times n},$$

所以

$$\text{rank}(\mathbf{M}_{\text{block}}) \leq N.$$

这里的“低秩”不是额外假设，而是由递推状态的维度自动诱导出来的。

来源：Mamba-2 Definition 3.1–3.2, Lemma 3.3, Theorem 3.5.

[建议分页位置](#)

4.5 第 5 页：最小例子——1-半可分矩阵就是一维标量 recurrence

先把 $N = 1$ 的情况彻底算清楚。考虑标量 recurrence

$$h_t = a_t h_{t-1} + b_t x_t, \quad y_t = c_t h_t. \quad (95)$$

展开得到

$$M_{j,i} = c_j \left(\prod_{k=i+1}^j a_k \right) b_i, \quad j \geq i. \quad (96)$$

若暂时忽略两端的 b_i, c_j ，核心 mask 为

$$L_{j,i} = \prod_{k=i+1}^j a_k.$$

于是

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 & \cdots \\ a_1 & 1 & 0 & \cdots \\ a_2 a_1 & a_2 & 1 & \cdots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix}.$$

乘法 $\mathbf{y} = \mathbf{L}\mathbf{x}$ 完全等价于

$$y_t = a_t y_{t-1} + x_t.$$

因此 1-SS matrix 不是一个抽象矩阵名词，它就是**标量递推展开后得到的下三角权重矩阵**。

更一般地，把 b_i, c_j 放回去：

$$\mathbf{M} = \text{diag}(\mathbf{c}) \mathbf{L} \text{diag}(\mathbf{b}). \quad (97)$$

这一形式马上会与 attention 的 $QK^\top$ 发生联系。

来源：Mamba-2 Sec. 3.2.2, Eq. (6)–(7).

[建议分页位置](#)

4.6 第 6 页：从另一侧看——Linear Attention 其实也是“矩阵状态 + recurrence”

为了理解 SSD，先快速回顾因果核注意力（causal kernel attention）。设

$$Q \in \mathbb{R}^{T \times N}, \quad K \in \mathbb{R}^{T \times N}, \quad V \in \mathbb{R}^{T \times P}.$$

去掉 softmax 后，因果掩码核注意力写成

$$Y = (L_{\text{causal}} \circ QK^\top)V, \quad (98)$$

其中

$$(L_{\text{causal}})_{t,i} = \mathbf{1}[i \leq t].$$

逐时刻展开：

$$y_t = \sum_{i=0}^t (q_t^\top k_i) v_i \quad (99)$$

$$= q_t^\top \left(\sum_{i=0}^t k_i v_i^\top \right). \quad (100)$$

定义矩阵状态

$$S_t = S_{t-1} + k_t v_t^\top, \quad y_t = q_t^\top S_t. \quad (101)$$

这就是 linear attention 的 recurrent form。状态 $S_t \in \mathbb{R}^{N \times P}$ 将全部历史 key-value 外积压缩成固定大小矩阵。

如果把 causal mask 的全 1 累加换成上一页的 1-SS mask

$$L_{t,i} = \prod_{k=i+1}^t a_k,$$

则 recurrence 变为

$$S_t = a_t S_{t-1} + k_t v_t^\top, \quad y_t = q_t^\top S_t. \quad (102)$$

这已经非常接近 scalar-identity SSM。SSD 的核心统一就在这里。

来源：Mamba-2 Sec. 4.1–4.2.

建议分页位置

4.7 第 7 页：结构化掩码注意力 (Structured Masked Attention) ——把“因果全 1 mask”推广成可快速乘的结构矩阵

标准 linear attention 快，是因为 causal mask 的乘法等价于 cumulative sum。Mamba-2 的观察是：真正需要的不是“mask 必须全 1”，而是

mask $\mathbf{L}$ 必须具有快速矩阵乘法。

于是定义 Structured Masked Attention (SMA)：

$$Y = (\mathbf{L} \circ QK^\top)V, \quad (103)$$

其中 $\mathbf{L}$ 可以是任意具有 sub-quadratic matvec 的结构矩阵。

这一函数可以用两种 contr 动作 (action) 顺序计算。二次形式先算 pairwise score：

$$G = QK^\top, \quad (104)$$

$$M = G \circ L, \quad (105)$$

$$Y = MV. \quad (106)$$

线性形式则先扩展 value-key 特征，再做结构 mask 的乘法，最后由 query 收缩：

$$Z_i = v_i k_i^\top, \quad (107)$$

$$H = LZ, \quad (108)$$

$$y_t = q_t^\top H_t. \quad (109)$$

二者是同一个四元张量 contraction 的不同收缩顺序。计算复杂度能否降下来，关键集中到 L 的结构上。

当 $L = L_{\text{causal}}$ 时得到普通线性注意力；当 L 是指数衰减掩码时接近 RetNet；当 L 是 1-半可分掩码时，就进入 SSD。

来源：Mamba-2 Definition 4.2, Eq. (12)–(15).

建议分页位置

4.8 第 8 页：SSD 核心恒等式——Scalar-Identity SSM 与 1-SS Structured Attention 是同一个函数

现在把 SSM 与 attention 两条线真正合起来。考虑特殊状态转移

$$\bar{\mathbf{A}}_t = a_t \mathbf{I}. \quad (110)$$

代入 SSM 的矩阵元素：

$$M_{j,i} = \mathbf{C}_j^\top \bar{\mathbf{A}}_j \cdots \bar{\mathbf{A}}_{i+1} \bar{\mathbf{B}}_i \quad (111)$$

$$= \left(\prod_{k=i+1}^j a_k \right) \mathbf{C}_j^\top \bar{\mathbf{B}}_i. \quad (112)$$

定义 1-SS mask

$$L_{j,i} = \prod_{k=i+1}^j a_k,$$

以及 Gram-like 矩阵

$$G_{j,i} = \mathbf{C}_j^\top \bar{\mathbf{B}}_i,$$

则

$$\mathbf{M} = \mathbf{L} \circ (\mathbf{C} \bar{\mathbf{B}}^\top). \quad (113)$$

令 attention 对应关系

$$\mathbf{Q} \leftrightarrow \mathbf{C}, \quad \mathbf{K} \leftrightarrow \bar{\mathbf{B}}, \quad \mathbf{V} \leftrightarrow \mathbf{x},$$

便得到

$$Y = (L \circ QK^\top)V. \quad (114)$$

这就是 Structured State Space Duality 的最核心等式：特定的 selective SSM 与特定的 structured masked attention 不是“相似”，而是**同一个 sequence transformation 的两种表示**。

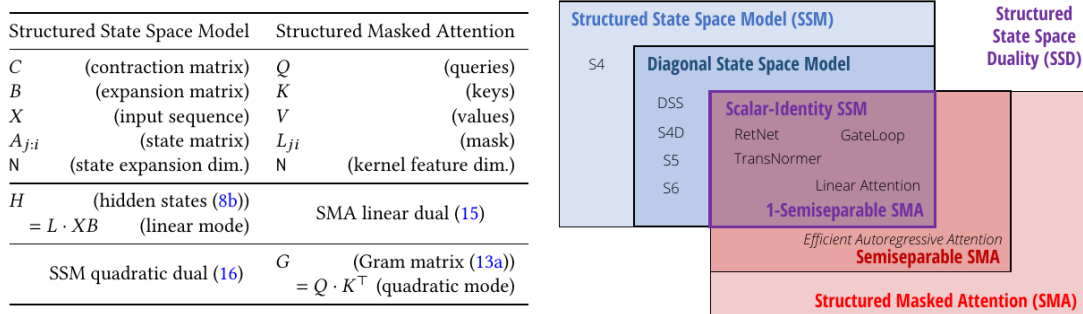


图 10: Mamba-2 原论文 Figure 4。左侧给出 SSM 与结构化注意力（structured attention）的变量对应；右侧展示 SSM、SMA 与 SSD 的集合关系。

来源：Mamba-2 Sec. 5.1–5.3, Fig. 4.

[建议分页位置](#)

4.9 第 9 页：为什么叫“对偶性”——同一个函数既有线性 recurrence，也有二次 attention 计算

对式(114)，若直接物化 $T \times T$ 的 M ：

$$G = CB^\top \quad \mathcal{O}(T^2N), \quad (115)$$

$$M = L \circ G \quad \mathcal{O}(T^2), \quad (116)$$

$$Y = MX \quad \mathcal{O}(T^2P). \quad (117)$$

这是 quadratic / 类似注意力的矩阵模式。它的优点是大量使用矩阵乘法，对较短 chunk 很友好。

另一方面，用递推状态计算同一个函数：

$$H_t = a_t H_{t-1} + B_t X_t^\top, \quad Y_t = C_t^\top H_t, \quad (118)$$

其中 $H_t \in \mathbb{R}^{N \times P}$ 。这是 linear / recurrent mode，复杂度随 T 线性增长。

因此“duality”不是说两个模型性能差不多，而是说：

同一个 $Y = (L \circ CB^\top)X$	{	可以按 pairwise matrix mixer 计算， 也可以按递推状态计算。
------------------------------	---	--

Mamba-2 的算法设计就是在不同尺度上混合这两个计算模式：块内用 quadratic matmul，块间用递推状态。

来源：Mamba-2 Sec. 3.4, 4.3, 5.3.

建议分页位置

4.10 第 10 页：为什么还要设计新算法——纯递推和纯二次矩阵计算都只利用了一半结构

基础知识补充：什么是 GEMM，为什么深度学习硬件偏爱它

GEMM 是通用矩阵乘（General Matrix Multiplication）的常用简称，即 $C \leftarrow AB + C$ 一类稠密矩阵运算。现代 GPU/TPU 的张量核心对这类规则的大矩阵乘有极高吞吐。逐元素递推虽然理论 FLOPs 不一定多，却难以把硬件算力“喂满”；因此 Mamba-2 的目标之一就是尽可能多的序列计算改写成块内 GEMM。

若全程用 recurrence，时间复杂度线性，但每一步都包含状态更新，硬件利用率不高；若全程物化 M ，可以用 GEMM，却会付出 $\mathcal{O}(T^2)$ 的序列代价。SSD 的分块算法选择中间路线。

把长度 T 按 chunk size Q 分成 $J = T/Q$ 个块。矩阵 M 被分成 $J \times J$ 个 block：

$$M = \begin{bmatrix} M^{(0,0)} & 0 & \dots \\ M^{(1,0)} & M^{(1,1)} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

由半可分性质：

- 对角块 $M^{(j,j)}$ ：只是长度 Q 的小型同类问题，可以用二次形式的对偶计算（quadratic dual）直接 GEMM；

- 非对角块 $M^{(j,i)}, j > i$: 秩至多 N , 可以通过 chunk state 低秩分解, 不需要物化完整 $Q \times Q$ 交互。

这等价于把输出拆成

$$Y^{(j)} = Y_{\text{intra}}^{(j)} + Y_{\text{inter}}^{(j)}, \quad (119)$$

其中第一项只由当前块内输入产生, 第二项由前面所有 chunk 压缩成的 state 产生。

来源: Mamba-2 Sec. 6.

建议分页位置

4.11 第 11 页: SSD 分块算法 (SSD Block Algorithm) ——对角块做局部 attention, 非对角块通过状态传递

论文 Figure 5 把算法画成了“输入—状态—输出”的块级传播。可以先按物理意义读图: 橙色对角块是当前块内的 input-to-output; 绿色把一个 chunk 压成 state; 黄色负责 state-to-state; 蓝色把进入当前 chunk 的 state 展开到各输出位置。

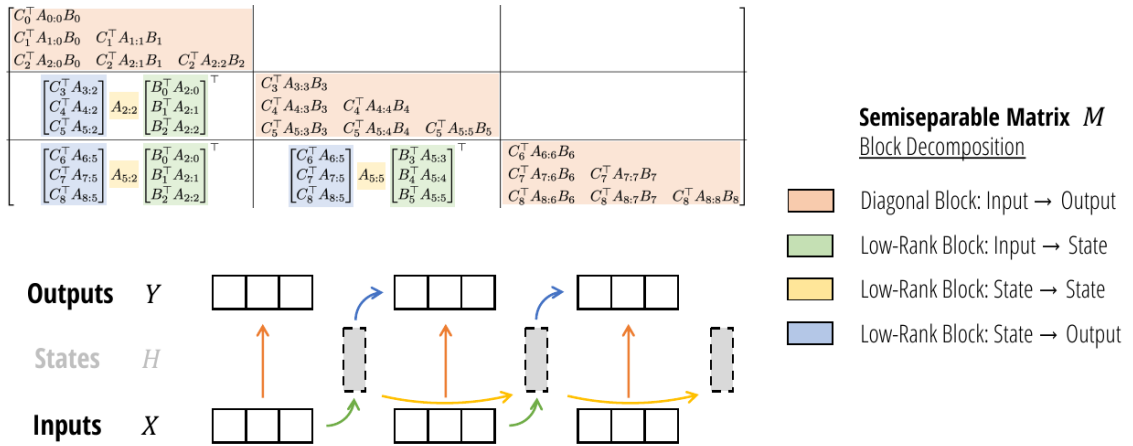


图 11: Mamba-2 原论文 Figure 5。SSD 通过半可分分块 (semiseparable block decomposition), 把全局矩阵乘拆成块内稠密矩阵乘 (dense matmul) 与块间状态传递 (state passing)。

对第 j 个 chunk, 区间记为

$$R_j = \{jQ, \dots, (j+1)Q - 1\}.$$

对角块直接计算

$$Y_{\text{intra}}^{(j)} = \text{SSD}_{R_j}(X_{R_j}; A_{R_j}, B_{R_j}, C_{R_j}), \quad (120)$$

但这里采用局部二次计算模式 (quadratic mode): 只在 $Q \times Q$ 范围内物化局部结构矩阵。因为 Q 通常是固定小值, 这一步可以转化为高吞吐 GEMM, 而且所有 chunk 可并行。

来源: Mamba-2 Fig. 5, Sec. 6.1.

建议分页位置

4.12 第 12 页：非对角低秩块的三步——右因子 (Right Factor)、中心因子 (Center Factor)、左因子 (Left Factor)

非对角块之所以便宜，是因为它们通过状态维度 N 因子化。算法可以严格对应三步。

第一步：Right / B-block factor——把每个 chunk 压成末状态。 对 chunk j ，暂设进入该 chunk 的初始状态为零。把块内所有输入聚合成末状态

$$H_j^{\text{local}} = \sum_{i \in R_j} \left(\prod_{k=i+1}^{(j+1)Q-1} a_k \right) B_i X_i^\top \in \mathbb{R}^{N \times P}. \quad (121)$$

这一步本质上是 $(N, Q) \times (Q, P)$ 的 batched matmul。

第二步：Center / A-block factor——在 chunk 级别递推状态。 令

$$\alpha_j = \prod_{t=jQ}^{(j+1)Q-1} a_t,$$

则真正的 chunk 末状态满足

$$H_j = \alpha_j H_{j-1} + H_j^{\text{local}}. \quad (122)$$

序列长度从 T 降为 $J = T/Q$ ，因此这是一个短得多的 scalar-SSM scan。

第三步：Left / C-block factor——把进入 chunk 的历史状态展开到每个输出位置。 若 H_{j-1} 是进入 chunk j 前的正确历史状态，则对 $t \in R_j$ ：

$$Y_{\text{inter},t} = C_t^\top \left(\prod_{k=jQ}^t a_k \right) H_{j-1}. \quad (123)$$

这又是 $(Q, N) \times (N, P)$ 的矩阵乘。

最后

$$Y^{(j)} = Y_{\text{intra}}^{(j)} + Y_{\text{inter}}^{(j)}.$$

学生若能理解这三步，就已经理解 SSD 分块算法的主体：**input** $\rightarrow$ **state**, **state** $\rightarrow$ **state**, **state** $\rightarrow$ **output**。

来源：Mamba-2 Sec. 6.2.

[建议分页位置](#)

4.13 第 13 页：复杂度为什么同时像 SSM 和 Attention——把训练、推理、内存放到一张表

在论文主要比较中取 head dimension $P = N$ 。此时 SSD 的矩阵状态大小为 $N \times N$ 。三种机制的理论成本可整理为：

表 16: Mamba-2 论文 Sec. 6.3 的复杂度比较。这里的 T 是序列长度， N 是 state/head dimension。

	Attention	传统 SSM scan	SSD
有效状态规模	T	N	N
训练 FLOPs	$\mathcal{O}(T^2N)$	$\mathcal{O}(TN^2)$	$\mathcal{O}(TN^2)$
单步/推理核心 FLOPs	$\mathcal{O}(TN)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$
Naive memory	$\mathcal{O}(T^2)$	$\mathcal{O}(TN^2)$	$\mathcal{O}(TN)$
主要算子可转 GEMM	是	弱	是

这张表说明 SSD 的设计目标：

保留 SSM 对 T 的线性缩放，同时把大部分工作改造成 attention 擅长的矩阵乘法。

论文的实际 A100 benchmark 也验证了这一点：当 state expansion 较大时，SSD 比 Mamba fused scan 快 2–8×；在较长序列上也可快于 FlashAttention-2。

Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	30.8	51.7
RWKV4-1.5B	NeoX	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	34.0	51.4
Mamba-1.4B	NeoX	6.80	5.04	65.0	59.1	74.2	65.5	32.8	61.5	36.4	56.4
Mamba-2-1.3B	NeoX	6.66	5.02	65.7	59.9	73.2	64.3	33.3	60.9	37.8	56.4
Hybrid H3-2.7B	GPT2	—	7.92	55.7	59.7	73.3	65.6	32.3	61.4	33.6	54.5
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	35.2	55.7
RWKV4-3B	NeoX	7.00	5.24	63.9	59.6	73.7	67.8	33.1	59.6	37.0	56.4
Mamba-2.8B	NeoX	6.22	4.23	69.2	66.1	75.2	69.7	36.3	63.5	39.6	59.9
Mamba-2-2.7B	NeoX	6.09	4.10	69.7	66.6	76.4	69.6	36.4	64.0	38.8	60.2

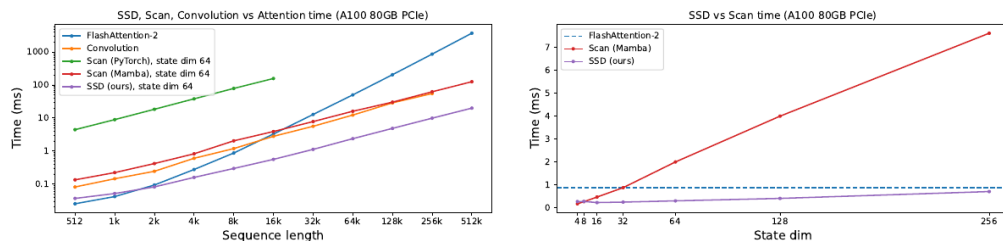


Figure 10: (Efficiency Benchmarks.) (Left) Our SSD is 2–8× faster than a Mamba fused scan for large state expansion ($N = 64$) and faster than FlashAttention-2 for sequence length 2k and above. (Right) Sequence length 4K: Increasing state expansion slows down the Mamba optimized scan implementation linearly. SSD can handle much larger state expansion faster without much slowdown.

图 12: Mamba-2 原论文 Figure 10 的效率 benchmark。左图比较序列长度扩展；右图固定序列长度、增加状态维数，Mamba scan 时间近似线性上升，而 SSD 更能利用矩阵乘法单元。

来源：Mamba-2 Theorem 6.1, Sec. 6.3, Fig. 10.

建议分页位置

4.14 第 14 页：Mamba-2 Block 不只是换内核——参数生成从 sequential 改成 parallel

SSD 解决的是内层 sequence mixer。Mamba-2 还修改了 block 外围结构，使大模型训练更友好。回顾 Mamba-1：先从 u 投影得到 SSM 主输入 X ，再由 X 产生 Δ, B, C 。因此参数投影存在串行依赖：

$$u \rightarrow X \rightarrow (\Delta, B, C) \rightarrow \text{SSM}(X).$$

Mamba-2 改成在 block 开头并行产生所有输入：

$$(X, Z, \Delta, B, C) = \text{Proj}(u), \quad (124)$$

然后

$$X_c = \text{Conv1D}(X), \quad (125)$$

$$Y = \text{SSD}(X_c; A, B, C, \Delta), \quad (126)$$

$$Y_g = Y \odot \phi(Z), \quad (127)$$

$$Y_n = \text{Norm}(Y_g), \quad (128)$$

$$\text{out} = Y_n W_o. \quad (129)$$

这样做有两个直接好处：

1. 消除生成 B, C, Δ 前对完整 X 的依赖，更适合 tensor parallel sharding；
2. 在输出投影前加入额外归一化，提高大模型训练稳定性。

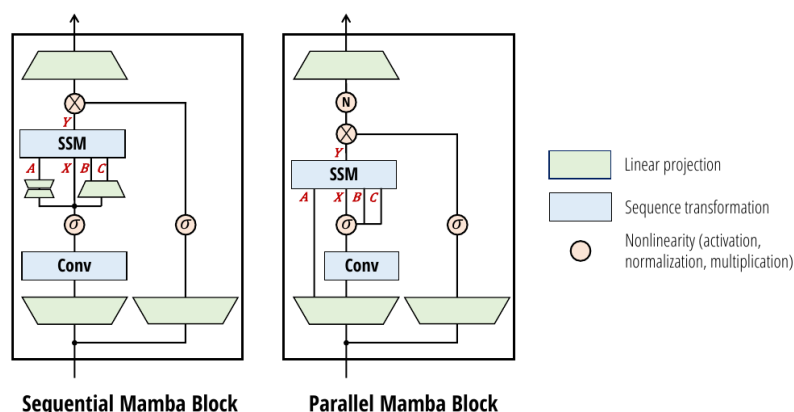


图 13: Mamba-2 原论文 Figure 6。左：Mamba-1 sequential projections；右：Mamba-2 parallel projections，并增加额外归一化。

来源：Mamba-2 Sec. 7.1, Fig. 6.

[建议分页位置](#)

4.15 第 16 页：Mamba-2 到底改了什么——把“数学层”“算法层”“网络层”分开总结

如果只说“Mamba-2 用矩阵乘更快”，学生很难形成结构化认识。应该把改动分为三层：

表 17: Mamba-1 到 Mamba-2 的三层变化。

层次	Mamba-1	Mamba-2
数学表示	选择性递推 / 扫描	递推 $\Leftrightarrow$ 半可分矩阵 $\Leftrightarrow$ 结构化注意力
核心算法	fused 并行扫描	分块 SSD: 块内二次复杂度的局部矩阵乘 (quadratic GEMM) + 块间递推
网络结构	sequential 生成 B, C, Δ	parallel projection 生成 A, X, B, C , 额外归一化
状态容量	增大 N 会明显拖慢 scan	SSD 更适合较大 N , 状态容量 (state capacity) 更容易扩展

从课程主线看, Mamba-2 并没有推翻第三章的 selectivity; 它保留输入相关的时变状态, 只把状态转移限制到更有利于 duality 的 scalar-identity 结构, 并通过矩阵结构把计算重新组织起来。

换句话说:

Mamba-1 主要回答“哪些信息该进入状态”;
Mamba-2 主要回答“同样的时变状态如何更高效地算”。

来源: Mamba-2 Sec. 5–7.

建议分页位置

4.16 第 17 页：实验与下一步——Mamba-2 提高了效率和 state capacity, 但“可编辑记忆”问题仍未结束

Mamba-2 的实验有两类结论值得带走。第一, SSD 算法把状态维数做大时的硬件成本显著降低; 第二, 在需要存储更多键值对的 MQAR 等任务中, 更大的状态大小会带来更强性能。这验证了“状态容量”确实是递推模型的核心资源。

但 Mamba-2 自己的混合实验也提示了纯有限状态的边界。350M 模型、48 层实验中, 加入少量 attention block 后困惑度 (perplexity) 还能继续下降:

表 18: Mamba-2 论文 Table 2 的部分结果: 在 48 层 350M 模型中加入少量 attention。

Attention blocks	0	1	2	3	4	5	6	7
Perplexity ↓	8.60	8.38	8.32	8.29	8.29	8.28	8.26	8.27

这不应在本章展开成 Hybrid 结论, 但它给出一个重要过渡: 即便 state mixer 已经更强、更大、更高效, 某些精确检索仍可能从显式 attention 中受益。

此外, Mamba / Mamba-2 的状态更新仍然主要是

旧状态衰减 + 新内容累加.

若同一个 key 已经存了旧 value, 现在需要**定点覆盖成新 value**, 简单累加会产生冲突。后续 DeltaNet

/ Gated DeltaNet 正是把问题从“选择是否写入”继续推进到“怎样按预测误差精确修改已存在的关联记忆”。

因此第四章的终点可以写成：

S4: long memory → Mamba: selective memory → Mamba-2: hardware-efficient large state → 下一步： editable memory.

来源：Mamba-2 Sec. 9.2–9.4; Table 2; MQAR 讨论.

建议分页位置

5 第五章：有限状态如何精确改写——DeltaNet、门控 DeltaNet 与 KDA

本章只解决一个问题：有限状态除了“写入”和“遗忘”，怎样对已经存在的 key-value 关联做定点修改？

本章给出的答案：把状态解释为关联记忆矩阵，使用预测误差校正实现 overwrite，再加入遗忘与细粒度衰减。

5.1 第 1 页：先看这一章解决什么——Mamba-2 已经能高效选择和衰减，为什么还需要 Delta Rule？

上一章把 Mamba-2 写成了 State Space Duality (SSD) 下的结构化矩阵混合器。为了与上一章符号衔接，本章首先保留“递归状态”的视角，但状态对象从向量状态 $\mathbf{h}_t$ 扩展为一个矩阵型关联记忆

$$\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}.$$

这里 $\mathbf{k}_t, \mathbf{q}_t \in \mathbb{R}^{d_k}$ 分别是 key 和 query, $\mathbf{v}_t \in \mathbb{R}^{d_v}$ 是 value, 输出为

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^{d_v}. \quad (130)$$

这个矩阵状态并不是突然换了一套语言。上一章已经看到：当 Mamba-2 的状态转移退化为标量乘单位阵时，SSM 可以与 gated linear attention 放在同一 SSD 框架中。于是一个 Mamba-2/线性注意力风格的 head 可以写成

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad 0 < \alpha_t < 1, \quad (131)$$

其中 α_t 负责整体衰减, $\mathbf{v}_t \mathbf{k}_t^\top$ 负责增加新的键-值 (key-value) 关联。

式(131)已经能“忘”，但它主要做的是**整体缩放 + 新关联叠加**。如果同一个 key 先后对应不同 value, 例如

$$\mathbf{k} \mapsto \mathbf{v}_{\text{old}} \longrightarrow \mathbf{k} \mapsto \mathbf{v}_{\text{new}},$$

我们真正希望的是**修改原关联**，而不是把新旧 value 继续叠在一起。于是这一章的核心问题变成

有限状态除了“记住/遗忘”，能否对指定 key 对应的内容进行精确修正？

DeltaNet、Gated DeltaNet (GDN) 和 Kimi Delta Attention (KDA) 正是在回答这个问题。

表 19: 从 Mamba-2 到 Delta/KDA, 状态操作从“衰减”逐渐走向“可编辑”。

方法	状态更新的主要动作	主要不足/下一步
Linear Attention	不断累加 $\mathbf{v}_t \mathbf{k}_t^\top$	旧关联无法主动删除, 易碰撞
Mamba-2	/	能忘, 但不能针对某个 key 精确改写
scalar gate	整体衰减 $\alpha_t \mathbf{S}_{t-1}$ 再写入	
DeltaNet	先读取当前 key 的旧值, 再按误差纠正	缺少显式长期遗忘, 长度外推有限
Gated DeltaNet	全局衰减 + delta correction	标量 gate 对各子空间仍较粗
KDA	细粒度对角 gate + delta correction	更复杂, 需要新的分块算法

来源：Gated DeltaNet Sec. 2–3；DeltaNet Sec. 2；Kimi Linear Sec. 2–3.

建议分页位置

5.2 第 2 页：先把 Linear Attention 看成“矩阵关联记忆”——后面的 Delta Rule 才有对象可以修改

最基本的 causal linear attention 可写成

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t. \quad (132)$$

若 $\mathbf{S}_0 = 0$ ，递推展开为

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top, \quad \mathbf{o}_t = \sum_{i=1}^t \mathbf{v}_i (\mathbf{k}_i^\top \mathbf{q}_t). \quad (133)$$

这说明 $\mathbf{S}_t$ 可以被理解为一个快速权重联想记忆（fast-weight associative memory）：每个外积 $\mathbf{v}_i \mathbf{k}_i^\top$ 都把“key $\mathbf{k}_i$ 到 value $\mathbf{v}_i$ 的线性映射”写进状态矩阵；query $\mathbf{q}_t$ 作为输入，状态矩阵把它映射成输出。

如果把所有 query/key/value 按时间堆叠为 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ ，则式(133)也可写成并行形式

$$\mathbf{O} = (\mathbf{Q} \mathbf{K}^\top \odot \mathbf{M}) \mathbf{V}, \quad (134)$$

其中 $\mathbf{M}$ 是 causal mask。于是同一个模型同时有两种视图：

推理：矩阵状态递推	$\iff$	训练：masked 矩阵乘法
-----------	--------	----------------

这与前面 SSM 的“recurrence–matrix mixer 双视图”完全一致。

从在线学习角度，Linear Attention 的加法更新可以理解为不断强化相关性，其目标近似写成

$$\mathcal{L}_t(\mathbf{S}) = -\langle \mathbf{S} \mathbf{k}_t, \mathbf{v}_t \rangle. \quad (135)$$

这个目标只鼓励“相关性更大”，却没有任何一项说明旧记忆什么时候应该被删除。问题因此自然出现：当状态容量固定、写入次数不断增加时，旧 key–value 会互相干扰。

来源：DeltaNet Sec. 2.1；Gated DeltaNet Sec. 2.1；Kimi Linear Sec. 2.2.

建议分页位置

5.3 第 3 页：为什么纯加法会产生记忆冲突（memory collision）——先用同一个 key 覆盖的最小例子算清楚

假设 $\|\mathbf{k}\|_2 = 1$ ，同一个 key 第一次对应 value $\mathbf{v}_1$ ：

$$\mathbf{S}_1 = \mathbf{v}_1 \mathbf{k}^\top, \quad \mathbf{S}_1 \mathbf{k} = \mathbf{v}_1.$$

后来同一个 key 又出现，但目标已经更新为 $\mathbf{v}_2$ 。若仍使用加法规则，得到

$$\mathbf{S}_2 = \mathbf{S}_1 + \mathbf{v}_2 \mathbf{k}^\top = (\mathbf{v}_1 + \mathbf{v}_2) \mathbf{k}^\top, \quad (136)$$

于是再次查询 $\mathbf{k}$ 时

$$\mathbf{S}_2 \mathbf{k} = \mathbf{v}_1 + \mathbf{v}_2, \quad (137)$$

旧值没有被覆盖，而是和新值混在了一起。

更一般地，当 $t > d_k$ 时，大量 key 必然在有限 d_k 维空间中发生相似或相关，累积状态

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top$$

会把越来越多关联压进固定大小的矩阵。对 query $\mathbf{q}$ ，不同历史 key 的交叉项

$$\mathbf{v}_i(\mathbf{k}_i^\top \mathbf{q})$$

会共同进入输出。如果某些 $\mathbf{k}_i$ 与当前 query 非正交，就形成记忆干扰。

因此“可编辑记忆”的理想操作应当是：

1. 用当前 key 先从旧状态中读出旧值；
2. 计算旧值和目标值之间的误差；
3. 只沿当前 key 对应的方向修正状态，而尽量不影响其他 key 子空间。

这正是经典 Widrow–Hoff / delta rule 的思路。

教学上应把 Delta Rule 理解成“先读后改”：它不是简单增加一个新 value，而是根据当前记忆对该 key 的预测误差，决定需要改多少。

来源：DeltaNet Sec. 2.2；Gated DeltaNet Sec. 2.2.

[建议分页位置](#)

5.4 第 4 页：Delta Rule 从一个在线回归目标直接推出来——不是经验公式

希望状态矩阵对当前 key 满足

$$\mathbf{S} \mathbf{k}_t \approx \mathbf{v}_t.$$

最自然的瞬时回归损失是

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} \mathbf{k}_t - \mathbf{v}_t\|_2^2. \quad (138)$$

对 $\mathbf{S}$ 求梯度：

$$\nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top. \quad (139)$$

以 $\beta_t > 0$ 为在线学习率，从上一状态 $\mathbf{S}_{t-1}$ 做一步 SGD：

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \beta_t \nabla_{\mathbf{S}} \mathcal{L}_t(\mathbf{S}_{t-1}) \quad (140)$$

$$= \mathbf{S}_{t-1} - \beta_t (\mathbf{S}_{t-1} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top. \quad (141)$$

整理得

$$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (142)$$

式(142)就是 DeltaNet 的核心状态更新。与 Linear Attention 的区别清楚地集中在第一项：它不再原封不动保留 $\mathbf{S}_{t-1}$ ，而是先沿 $\mathbf{k}_t$ 方向删去当前错误关联，再写入目标 value。

如果把误差定义成

$$\mathbf{e}_t = \mathbf{v}_t - \mathbf{S}_{t-1}\mathbf{k}_t,$$

那么更新可以更直观地写成

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \beta_t \mathbf{e}_t \mathbf{k}_t^\top. \quad (143)$$

这就是“delta”的由来：真正被写入状态的是 prediction error，而不是 value 本身。

来源：DeltaNet Sec. 2.2；Gated DeltaNet Sec. 3.1；Widrow-Hoff/LMS 解释见 DeltaNet 脚注。

建议分页位置

5.5 第 5 页：同一个更新再看一遍——读取-删除-写入（Read-Remove-Write）解释比梯度公式更直观

定义当前 key 在旧状态中的读出

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1}\mathbf{k}_t. \quad (144)$$

DeltaNet 先构造新的目标值

$$\mathbf{v}_t^{\text{new}} = \beta_t \mathbf{v}_t + (1 - \beta_t) \mathbf{v}_t^{\text{old}}. \quad (145)$$

然后把旧 key-value 关联去掉，再写回新关联：

$$\boxed{\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{old}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{new}} \mathbf{k}_t^\top.} \quad (146)$$

将式(145)代入式(146)，立即得到式(142)。

$\beta_t \in (0, 1)$ 可以理解为 soft writing strength：

- $\beta_t = 0$ ： $\mathbf{v}_t^{\text{new}} = \mathbf{v}_t^{\text{old}}$ ，状态完全不变；
- $\beta_t = 1$ ：旧值完全移除，当前 key 被完整改写成 $\mathbf{v}_t$ ；
- $0 < \beta_t < 1$ ：在旧关联和新目标之间做平滑修正。

如果同一个归一化 key 从 $\mathbf{v}_1$ 更新为 $\mathbf{v}_2$ ，取 $\beta = 1$ ，则

$$\mathbf{S}_2 = \mathbf{v}_1 \mathbf{k}^\top - (\mathbf{v}_1 \mathbf{k}^\top \mathbf{k}) \mathbf{k}^\top + \mathbf{v}_2 \mathbf{k}^\top = \mathbf{v}_2 \mathbf{k}^\top,$$

这与式(137)形成直接对照：加法记忆（additive memory）得到 $\mathbf{v}_1 + \mathbf{v}_2$ ，Delta 记忆得到真正的覆盖 $\mathbf{v}_2$ 。

来源：DeltaNet Sec. 2.2.

建议分页位置

5.6 第 6 页：为什么 $\mathbf{I} - \beta \mathbf{k} \mathbf{k}^\top$ 很重要——DeltaNet 的“删除”本质是一个广义豪斯霍尔德变换

基础知识补充：投影矩阵、秩一更新与豪斯霍尔德反射

若 $\|\mathbf{k}\|_2 = 1$ ，则 $\mathbf{k} \mathbf{k}^\top$ 是投影到方向 $\mathbf{k}$ 上的秩一投影矩阵。矩阵

$$\mathbf{I} - \beta \mathbf{k} \mathbf{k}^\top$$

只改变沿 $\mathbf{k}$ 方向的分量，而保持其正交补空间不变；当 $\beta = 2$ 时， $\mathbf{I} - 2\mathbf{k} \mathbf{k}^\top$ 就是经典的豪斯霍尔德反射。因此 DeltaNet 的“删除旧记忆”实际上是一个方向非常明确的线性几何操作。

把式(142)中的状态转移记为

$$\mathbf{H}_t = \mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top. \quad (147)$$

若 $\|\mathbf{k}_t\|_2 = 1$ ，则对与 $\mathbf{k}_t$ 正交的任意向量 $\mathbf{z}$ ，

$$\mathbf{H}_t \mathbf{z} = \mathbf{z};$$

而沿 $\mathbf{k}_t$ 方向，

$$\mathbf{H}_t \mathbf{k}_t = (1 - \beta_t) \mathbf{k}_t.$$

所以 $\mathbf{H}_t$ 的特征值为

$$\lambda(\mathbf{H}_t) = \{1, \dots, 1, 1 - \beta_t\}. \quad (148)$$

这给出了非常清楚的几何解释：

- $\beta_t = 1$ 时，当前 key 方向被投影掉，相当于把旧关联从该子空间中擦除；
- $0 < \beta_t < 1$ 时，只部分擦除；
- 若允许 $\beta_t \in (0, 2)$ ，则 $1 - \beta_t$ 可为负，状态可出现“翻转”动力学；后面第六章会看到负特征值与状态跟踪能力直接相关。

特别地，当参数化为 $\mathbf{I} - 2\mathbf{k} \mathbf{k}^\top$ 时，就是经典豪斯霍尔德反射（Householder reflection）。

因此 DeltaNet 的创新不只是“损失函数换成平方误差”，它同时把每一步状态转移变成了 **identity + 秩一（rank-one）** 的结构化矩阵。这一点将直接决定后面的并行算法：一串豪斯霍尔德型矩阵虽然逐步依赖，但其乘积可以用紧凑 WY 表示（compact WY representation）压缩。

来源：DeltaNet Sec. 3.1；Gated DeltaNet Sec. 2.2；负特征值说明亦见 DeltaNet 脚注 2。

[建议分页位置](#)

5.7 第 7 页：DeltaNet 为什么一开始难以扩展——表达力提高以后，序列并行性变差了

DeltaNet 的纯递推式

$$\mathbf{S}_t = \mathbf{S}_{t-1} \mathbf{H}_t + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

需要先得到 $\mathbf{S}_{t-1}$ 才能计算 $\mathbf{S}_t$ 。若把所有 $\mathbf{S}_1, \dots, \mathbf{S}_L$ 显式物化（materialize），状态大小为 $d_v d_k$ ，长序列训练会产生巨大的 HBM 读写开销；若不 materialize，则时间维又近似串行。

DeltaNet 2024 的关键算法工作不是再次修改状态规则，而是寻找一个等价表示：

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{u}_i \mathbf{k}_i^\top, \quad (149)$$

其中 $\mathbf{u}_i$ 被称为 pseudo-value。对 Linear Attention，pseudo-value 就是原始 value；对 DeltaNet，它吸收了“删除旧关联”的影响。

由式(142)和式(149)可递推得到

$$\mathbf{u}_t = \beta_t \left(\mathbf{v}_t - \sum_{i=1}^{t-1} \mathbf{u}_i (\mathbf{k}_i^\top \mathbf{k}_t) \right). \quad (150)$$

一旦所有 $\mathbf{u}_t$ 被构造出来，输出就重新变成普通 linear attention 的形式

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{U}. \quad (151)$$

问题被转化为：怎样在不逐序列单元物化 $\mathbf{S}_t$ 的情况下，成批构造 $\mathbf{U}$ ？这就是 WY / 分块计算推导的入口。

来源：DeltaNet Sec. 3.1.

[建议分页位置](#)

5.8 第 8 页：紧凑 WY 表示 (Compact WY) 的作用——把一串 rank-one 状态转移压成两个“瘦矩阵”

考虑长度为 r 的一段序列，定义状态转移积

$$\mathbf{P}_r = \prod_{i=1}^r (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top). \quad (152)$$

直接存储每一个 $d_k \times d_k$ 中间矩阵并不划算。紧凑 WY 表示利用每个因子都是“单位阵 + 秩一修正”，写成

$$\mathbf{P}_r = \mathbf{I} - \sum_{i=1}^r \mathbf{w}_i \mathbf{k}_i^\top. \quad (153)$$

其中 $\mathbf{w}_i \in \mathbb{R}^{d_k}$ 满足递推

$$\mathbf{w}_r = \beta_r \left(\mathbf{k}_r - \sum_{i=1}^{r-1} \mathbf{w}_i (\mathbf{k}_i^\top \mathbf{k}_r) \right). \quad (154)$$

对应的输入累积项也可写成

$$\mathbf{H}_r = \sum_{i=1}^r \mathbf{u}_i \mathbf{k}_i^\top, \quad \mathbf{u}_r = \beta_r \left(\mathbf{v}_r - \sum_{i=1}^{r-1} \mathbf{u}_i (\mathbf{k}_i^\top \mathbf{k}_r) \right). \quad (155)$$

这里最重要的不是记住 $\mathbf{w}_i$ 和 $\mathbf{u}_i$ 两条式子，而是看出：

原来需要存 $d_k \times d_k$ 的变换积，现在只需存一组长度 d_k 的向量。

这为块内用矩阵乘一次性处理多个 token 提供了可能。

来源：DeltaNet Sec. 3.1；compact WY 参考 Bischof & Loan (1985)，论文中已给出对应推导。

[建议分页位置](#)

5.9 第 9 页：从序列单元递推到分块递推——DeltaNet 怎样重新获得张量核心 (Tensor Core) 友好的矩阵乘

把序列切成长度 C 的 chunk。第 c 个 chunk 的 key/value/pseudo-value 矩阵记为

$$\mathbf{K}_{[c]}, \mathbf{V}_{[c]}, \mathbf{U}_{[c]} \in \mathbb{R}^{C \times d},$$

并令块开始时的状态为 $\mathbf{S}_{[c]}$ 。DeltaNet 使用 UT 变换 (UT transform) 将式(154)–(155) 中的序列递推整理为

$$\mathbf{T}_{[c]} = \left[\mathbf{I} + \text{strictLower} \left(\text{Diag}(\beta_{[c]}) \mathbf{K}_{[c]} \mathbf{K}_{[c]}^\top \right) \right]^{-1} \text{Diag}(\beta_{[c]}), \quad (156)$$

然后

$$\mathbf{W}_{[c]} = \mathbf{T}_{[c]} \mathbf{K}_{[c]}, \quad \mathbf{U}_{[c]} = \mathbf{T}_{[c]} \mathbf{V}_{[c]}. \quad (157)$$

这样块结束后的状态可写成

$$\boxed{\mathbf{S}_{[c+1]} = \mathbf{S}_{[c]} + (\mathbf{U}_{[c]} - \mathbf{W}_{[c]} \mathbf{S}_{[c]}^\top)^\top \mathbf{K}_{[c]}.} \quad (158)$$

块内所有输出为

$$\boxed{\mathbf{O}_{[c]} = \mathbf{Q}_{[c]} \mathbf{S}_{[c]}^\top + (\mathbf{Q}_{[c]} \mathbf{K}_{[c]}^\top \odot \mathbf{M}_C) (\mathbf{U}_{[c]} - \mathbf{W}_{[c]} \mathbf{S}_{[c]}^\top).} \quad (159)$$

式(158)–(159)把计算拆成了熟悉的两部分：

- **块间递推 (inter-chunk recurrence)**：每个 chunk 只传一个矩阵状态；
- **块内矩阵乘 (intra-chunk matmul)**：块内大部分计算转成 GEMM，可利用 Tensor Cores。

所以 DeltaNet 的算法路线与 Mamba-2 很相似：都不是取消递推，而是把细粒度 token recurrence 重新组织成“块内并行 + 块间状态传播”。

来源：DeltaNet Eq. (8)–(11), Sec. 3.2.

[建议分页位置](#)

5.10 第 10 页：DeltaNet 不只是一个公式——完整 block 仍包含卷积、归一化、门控和投影

实际语言模型中的 DeltaNet layer 并不是只把式(142)孤立地塞进网络。原论文的 block 仍采用 Transformer-like 外壳：归一化后进入 Delta Rule mixer，配合 short convolution、feature 归一化、输出门控和 MLP。

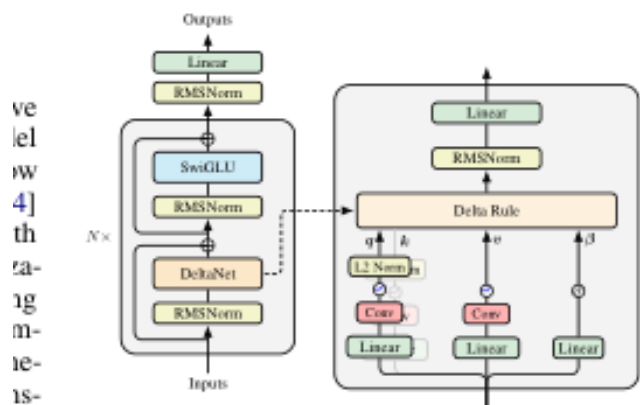


Figure 2: An illustration of DeltaNet neural architecture.

query vectors are given by $k_t = \frac{\text{SiLU}(\mathbf{W}_K \mathbf{x}_t)}{\|\text{SiLU}(\mathbf{W}_K \mathbf{x}_t)\|_2}$, $q_t =$ follow Katharopoulos et al. [48] and apply a “FI.11+1”

图 14: DeltaNet 原论文 Figure 2 的结构图裁剪。右侧展示 Delta Rule mixer 内部的 key/value/query 路径，左侧展示它如何嵌入 Transformer-like block。

论文特别强调 query/key 归一化。若 $\|\mathbf{k}_t\|_2 = 1$ 且 $0 \leq \beta_t \leq 1$ ，则式(148)中的特征值都落在 $[0, 1]$ ，有利于稳定递推；同时 L2 归一化往往比 sum 归一化更适合后续更强的遗忘/targeted 遗忘。

从参数量看，DeltaNet mixer 的主投影规模与标准 Transformer++ 保持同一量级，因此性能比较并不是靠显著扩大参数获得。真正需要验证的是：同样有限状态预算下，delta update 是否比简单衰减/累加更会利用记忆容量。

来源：DeltaNet Fig. 2, Sec. 3.3.

建议分页位置

5.11 第 11 页：先看 DeltaNet 得到了什么——关联回忆更强，但“长期遗忘”仍是空缺

DeltaNet 在 MQAR 等关联回忆任务上明显优于普通 linear attention，这和式(138)的训练目标一致：模型直接最小化 key 对应 value 的重构误差，而不是仅最大化相关性。

但是论文在局限性分析中也指出，DeltaNet 缺少显式 decay factor。若信息分布持续变化，旧关联虽然能在相似 key 到来时被修正，却不会像 Mamba-2 的 α_t 那样主动整体释放容量。这会影响长度外推和长上下文中的状态管理。

可以把两种机制放在同一张“记忆操作表”里：

表 20: Mamba-2 与 DeltaNet 的互补性。

	Mamba-2	DeltaNet
核心更新	$\alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$	$\mathbf{S}_{t-1} (I - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
擅长	主动衰减无关历史、控制记忆寿命	对当前 key 做误差校正和覆盖
不足	不知道“同一个 key 应改成什么”	无显式全局遗忘，旧状态寿命偏长
直觉	erase globally	overwrite locally

于是下一步几乎是唯一自然选择：把两者合在同一个 recurrence 里。

来源：DeltaNet Sec. 5.1; Gated DeltaNet Introduction/Sec. 3.

建议分页位置

5.12 第 12 页：门控 Delta 规则 (Gated Delta Rule) ——先衰减旧状态, 再对当前 key 做 delta correction

Gated DeltaNet 的核心式非常简洁：

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (160)$$

其中 $\alpha_t \in (0, 1)$ 是 data-dependent forget gate, β_t 是写入/校正强度。为了看清它的意义, 先定义衰减后的临时状态

$$\tilde{\mathbf{S}}_{t-1} = \alpha_t \mathbf{S}_{t-1}. \quad (161)$$

则式(160)等价于

$$\mathbf{S}_t = \tilde{\mathbf{S}}_{t-1} - \beta_t (\tilde{\mathbf{S}}_{t-1} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top. \quad (162)$$

这就把两个操作完全分开了：

$$\underbrace{\mathbf{S}_{t-1} \rightarrow \alpha_t \mathbf{S}_{t-1}}_{\text{管理整体历史寿命}} \longrightarrow \underbrace{\tilde{\mathbf{S}}_{t-1} \rightarrow \mathbf{S}_t}_{\text{对当前 key 做定点误差修正}}$$

因此 GDN 不是“DeltaNet 多一个 gate”这么简单；它实际上明确分离了 **memory lifetime** 与 **memory content correction** 两种不同控制自由度。

来源：Gated DeltaNet Eq. (10), Sec. 3.1.

建议分页位置

5.13 第 13 页：在线学习视角把 LA、Mamba-2、DeltaNet、GDN 放进同一张表

Gated DeltaNet 论文使用 online learning 框架说明不同 recurrent mixer 的差别。下面保留其核心四类模型, 并统一为本章的 $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ 记号。

表 21: 几种矩阵状态模型的在线目标与闭式更新。这里 $\langle \cdot, \cdot \rangle$ 表示向量内积。

方法	在线目标 (省略与 $\mathbf{S}_t$ 无关项)	状态更新
Linear Attention	$\ \mathbf{S}_t - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \mathbf{v}_t \rangle$	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$
Mamba-2	$\ \mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \mathbf{v}_t \rangle$	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$
DeltaNet	$\ \mathbf{S}_t - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$	$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
Gated DeltaNet	$\ \mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$	式(160)

从这张表可以看到两条清晰演化线：

1. $\mathbf{S}_{t-1} \rightarrow \alpha_t \mathbf{S}_{t-1}$ ：允许状态主动偏离旧值，产生 adaptive 遗忘；
2. $\mathbf{v}_t \rightarrow \mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t$ ：从相关性累积改为 reconstruction error correction。

GDN 恰好同时包含这两条改动。

来源：Gated DeltaNet Table 1, Sec. 3.1.

建议分页位置

5.14 第 14 页：S-NIAH 把“gate”和“delta”的互补性量化出来——不是只看语言建模 perplexity

Gated DeltaNet 设计了 Single Needle-In-A-Haystack (S-NIAH) 对比。三类任务的难点不同：口令检索 (pass-key retrieval) 更接近简单关联回忆；干扰上下文中的数字/UUID 检索对无关上下文干扰更敏感。表22摘取论文中关键长度结果。

表 22: Gated DeltaNet 原论文 S-NIAH 零样本结果 (1.3B 模型, %)。数值用于观察 update rule 的结构差异。

模型	S-NIAH-1			S-NIAH-2			S-NIAH-3	
	2K	4K	8K	2K	4K	8K	2K	4K
DeltaNet	96.8	99.0	98.8	45.6	18.6	14.4	47.0	22.4
Mamba-2	98.8	65.4	30.4	98.8	56.2	17.0	47.6	4.6
Gated DeltaNet	88.4	91.4	91.8	99.8	92.2	29.6	84.2	27.6

这张表比一句“GDN 更强”有价值得多：

- DeltaNet 在 S-NIAH-1 长度外推非常稳定，说明 delta correction 对关联保持有效；
- Mamba-2 在 S-NIAH-2 的中等长度较好，说明 gated 遗忘有助于过滤干扰上下文 (haystack)；
- GDN 在多数 setting 中更均衡，说明两种操作确实互补。

但表中 8K S-NIAH-2 仍明显下降，也提醒我们：更聪明的有限状态更新并没有消除容量限制。这会成为第六章的主线。

来源：Gated DeltaNet Table 2, Sec. 3.2.

建议分页位置

5.15 第 15 页：GDN 的分块算法如何从 DeltaNet 延续下来——只增加衰减感知重标度(decay-aware rescaling)

GDN 仍要解决和 DeltaNet 相同的训练问题：式(160)逐 token 递推不适合 GPU。定义块内第 i 到第 j 个 token 的累计衰减

$$\gamma_{i \rightarrow j} = \prod_{r=i}^j \alpha_r. \quad (163)$$

部分展开 recurrence，可以把当前 chunk 的状态写成

$$\mathbf{S}_{[c]}^{(r)} = \mathbf{S}_{[c]} \mathbf{F}_{[c]}^{(r)} + \mathbf{G}_{[c]}^{(r)}, \quad (164)$$

其中

$$\mathbf{F}_{[c]}^{(r)} = \prod_{i=1}^r \alpha_i (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \gamma_{1 \rightarrow r} \mathbf{P}_{[c]}^{(r)}. \quad (165)$$

因为 α_i 是标量，它可以从豪斯霍尔德乘积中抽出来，于是 $\mathbf{P}^{(r)}$ 仍然可以复用 DeltaNet 的 WY 表示。对输入贡献，则需要把伪值按相对衰减重新缩放。论文进一步用 UT transform 把它整理成矩阵乘。

最终，GDN 的 chunk 更新仍保持 DeltaNet 的骨架：

$$\mathbf{S}_{[c+1]} = \vec{\mathbf{S}}_{[c]} + \left(\mathbf{U}_{[c]}^g - \vec{\mathbf{W}}_{[c]} \mathbf{S}_{[c]}^\top \right)^\top \vec{\mathbf{K}}_{[c]}, \quad (166)$$

其中箭头符号表示由累计 γ 进行前向/反向 rescaling 后的矩阵。课堂上不必把每个箭头记号机械背诵，但要理解算法结构没有变：

DeltaNet 的 WY/UT 分块算法 + 标量衰减重标度。

因此 GDN 在增强记忆管理的同时，训练吞吐基本保持在 DeltaNet 同一量级，只比 Mamba-2 略慢。

来源：Gated DeltaNet Sec. 3.3, Eq. (11)–(12).

建议分页位置

5.16 第 16 页：Gated DeltaNet 的网络结构——recurrent primitive 只是 block 中的核心 mixer

GDN 原论文进一步构建了纯 GDN 模型和两类 hybrid：Gated DeltaNet-H1 使用 GDN + sliding-window attention；H2 再加入 Mamba-2。其 block 将 $q/k/v$ 路径、 α/β 参数生成和输出门控明确分开。

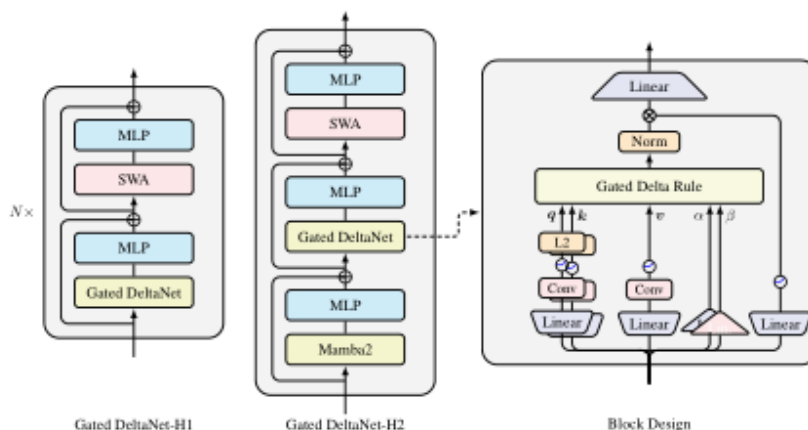


Figure 1: Visualization of the (hybrid) architecture and block design of Gated DeltaNet models. Gated DeltaNet-H1 and H2 use Gated DeltaNet + SWA and Mamba2 + Gated DeltaNet + SWA patterns respectively.

图 15: Gated DeltaNet 原论文 Figure 1 裁剪。左/中为两种 hybrid layer pattern，右为 Gated Delta Rule block。

这张图值得和前面的 Mamba/Mamba-2 block 对照：

- q, k 路径经过 linear projection、short convolution、SiLU、L2 归一化；

- v 路径经过 linear projection、short convolution、SiLU；
- α_t 控制状态寿命， β_t 控制 correction strength；
- 输出端仍有归一化和 data-dependent 输出门（output gate）。

因此从工程角度，GDN 不是“在 Mamba2 后面额外加一个误差项”，而是一个以新的 state update rule 为核心、重新参数化的完整 token mixer。

来源：Gated DeltaNet Fig. 1, Sec. 3.4.

建议分页位置

5.17 第 17 页：为什么 GDN 之后还要 KDA——标量 α_t 对整个 key 子空间同时遗忘，仍然太粗

GDN 的 α_t 是一个标量：

$$\tilde{\mathbf{S}}_{t-1} = \alpha_t \mathbf{S}_{t-1}.$$

这意味着同一时刻，整个 key 维度上的所有子空间被统一缩放。如果状态的不同通道承载不同时间尺度的信息，就会遇到冲突：某些通道需要快速清空，另一些通道却应继续保留。

GLA 已经证明细粒度门控（fine-grained gating）可以把 scalar decay 推广到 diagonal decay。Kimi Linear 进一步把这种细粒度 decay 和 delta correction 合在一起，得到 Kimi Delta Attention (KDA)。

为了严格跟随 Kimi 论文公式，这一小节暂时采用其转置状态记号

$$\tilde{\mathbf{S}}_t = \mathbf{S}_t^\top \in \mathbb{R}^{d_k \times d_v}.$$

定义通道级 gate

$$\mathbf{D}_t = \text{Diag}(\boldsymbol{\alpha}_t), \quad \boldsymbol{\alpha}_t \in [0, 1]^{d_k}. \quad (167)$$

KDA 的核心更新为

$$\tilde{\mathbf{S}}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \tilde{\mathbf{S}}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top, \quad (168)$$

输出为

$$\mathbf{o}_t = \tilde{\mathbf{S}}_t^\top \mathbf{q}_t. \quad (169)$$

相较 GDN， α_t 从一个数变成 d_k 个通道级 decay，因此状态各方向拥有不同记忆寿命。

来源：Kimi Linear Sec. 3, Eq. (1)；GLA 用于 fine-grained gating 背景。

建议分页位置

5.18 第 20 页：Kimi Linear 的完整架构——KDA 本身很强，但作者仍然保留了少量完整多头潜在注意力（Multi-Head Latent Attention, MLA）

Kimi Linear 的主干网络（backbone）以 KDA 为主要 token mixer，同时周期性插入完整全局多头潜在注意力（MLA）。论文默认采用

$$\boxed{3 \text{ KDA layers} : 1 \text{ Full MLA layer}}$$

的 layerwise hybrid。作者明确指出：**long-context 检索仍然是 pure linear attention 的主要瓶颈**，因此保留少量 global attention。

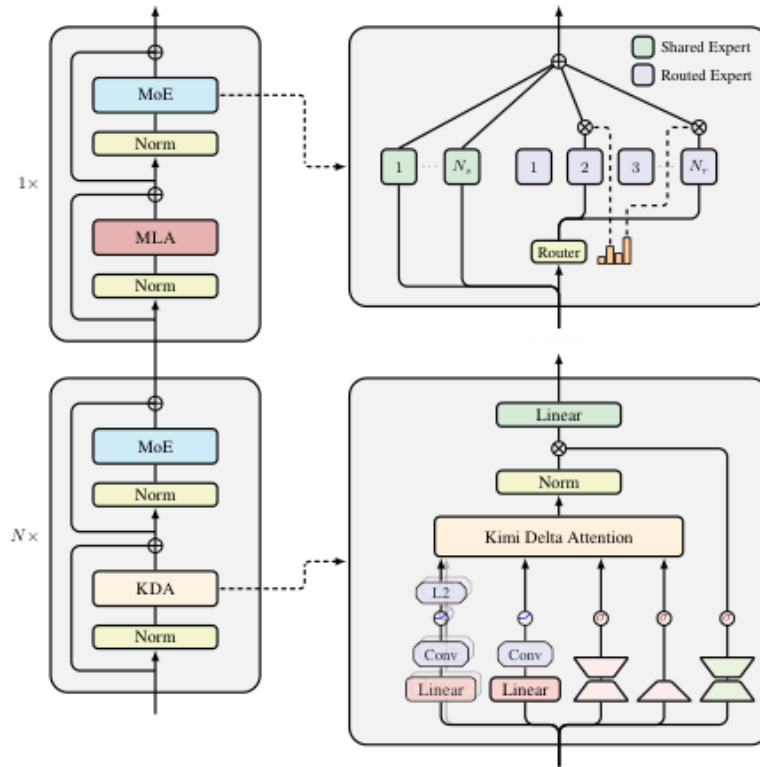


Illustration of our Kimi Linear model architecture, which consists of a stack of blocks containing a token mixing layer by a MoE channel-mixing layer. Specifically, we interleave N KDA layers with one MLA layer for token mixing, where

图 16: Kimi Linear Figure 3 裁剪。左侧是 3:1 KDA/MLA 的逐层（layerwise）结构，右侧是 KDA 与混合专家模块（Mixture-of-Experts, MoE block）。

其 head 内参数化包括

$$\mathbf{q}_t^h, \mathbf{k}_t^h = \text{L2Norm}(\text{Swish}(\text{ShortConv}(W_{q/k}^h \mathbf{x}_t))), \quad (170)$$

$$\mathbf{v}_t^h = \text{Swish}(\text{ShortConv}(W_v^h \mathbf{x}_t)), \quad (171)$$

$$\alpha_t^h = f(W_\alpha^\uparrow W_\alpha^\downarrow \mathbf{x}_t), \quad (172)$$

$$\beta_t^h = \sigma(W_\beta^h \mathbf{x}_t). \quad (173)$$

所以 KDA 同时包含局部卷积、归一化、通道级 decay 与 delta correction。它已经远超“一个简单 recurrent equation”的复杂度。

来源：Kimi Linear Fig. 3, Sec. 4.

建议分页位置

5.19 第 21 页：KDA 的合成任务说明了什么——有限状态可以被做得更聪明，但不是无限容量

Kimi Linear 用回文（Palindrome）、MQAR 和栈（Stack）三类任务测试不同线性递推混合器：

- Palindrome：要求从有限状态中精确重构/反转整段序列；
- MQAR：要求按 query 精确找回历史 key-value；
- Stack：要求维护 LIFO 离散状态，强调状态跟踪。

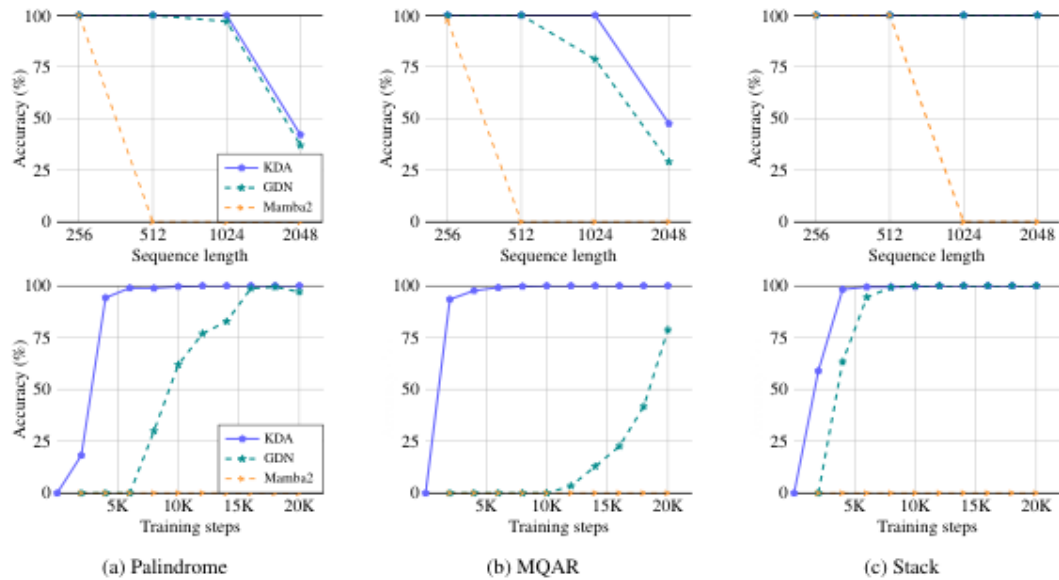


Figure 4: Results on synthetic tasks: palindrome, multi query associative recall, and the state tracking.

图 17: Kimi Linear Figure 4 裁剪。上排是长度外推准确率，下排是训练收敛速度；KDA、GDN、Mamba-2 的差异在 recall 与 state-tracking 任务上非常明显。

从课程主线看，这里得出两个结论：

1. update rule 的细节确实决定有限状态如何利用容量；从 Mamba-2 到 GDN/KDA，关联回忆与状态跟踪能力可以显著变化；
2. 但无论 update rule 多聪明，状态大小仍然固定为 $d_k d_v$ 。如果任务要求对任意长历史做 exact token-level recall，固定状态最终仍会遇到信息论边界。

第二点正是第六章为什么必须讨论 Attention/Hybrid，而不是继续无限制地把 recurrent update 做复杂。

来源：Kimi Linear Fig. 4, Sec. 5.1.

6 第六章：有限状态的边界——精确回忆、状态跟踪与混合注意力

本章只解决一个问题：无论更新规则多聪明，固定大小状态最终做不到什么？什么时候必须保留显式检索？

本章给出的答案：固定状态适合压缩记忆，但精确回忆需要显式索引；状态跟踪又要求更强的顺序状态变换，因此 Hybrid 成为自然分工。

6.1 第 1 页：先建立两个彼此不同的能力轴——精确回忆与状态跟踪

基础知识补充：“记住细节”和“维护状态”不是同一件事

精确回忆 (exact recall) 要求从长历史中精确找回某个具体键值或序列片段，像“查字典”；它主要受记忆容量与索引能力限制。**状态跟踪 (state tracking)** 要求随着输入一步步更新一个离散或连续的内部变量，像“执行状态机”；它主要受递推更新规则的表达能力的限制。一个模型可能很会压缩趋势，却不擅长精确检索；也可能能检索历史 token，却不擅长严格执行栈、计数器等状态更新。

经过前五章，我们已经看到两类完全不同的“记忆”问题：

精确回忆 (exact recall)：历史中某个 token/value 必须被**精确找回**。例如 MQAR、pass-key 检索、复制、从数组中读取某个索引。它要求模型保留足够多的原始信息。

State tracking：历史不是被逐项保存，而是不断修改一个内部世界状态。任务要求按正确顺序组合更新，例如堆栈 push/pop、变量交换、棋盘状态更新、程序执行。它更关注**状态转移是否足够表达顺序敏感的组合**。

这两类任务可以用不同数学对象描述：

$$\text{Recall: } (\mathbf{k}_i, \mathbf{v}_i)_{i \leq t}, \mathbf{q}_t \mapsto \mathbf{v}_{i^*}, \quad (174)$$

$$\text{State tracking: } x_t = (\delta_t \circ \delta_{t-1} \circ \cdots \circ \delta_1)(x_0). \quad (175)$$

它们对应两种不同的瓶颈：

表 23: Recall 与状态跟踪的瓶颈不是同一个问题。

能力	主要瓶颈	典型优势机制
Exact recall	状态容量是否足以保存任意位置的细节	全注意力 (full attention) / KV cache
State tracking	状态转移 (transition) 是否能表达顺序敏感、非交换的状态组合	真正的递推 (true recurrence) / 更强 transition

于是 Hybrid 的逻辑不是“SSM 不够强，所以随便加几层 attention”，而是：

让递推状态做擅长的状态计算，让 attention 做擅长的精确检索。

后面将分别用 BASED、The Illusion of State 和 OLMo Hybrid 把这两个边界讲清。

建议分页位置

6.2 第 2 页：精确回忆首先是一个“状态容量”问题——MQAR 直接测有限状态能装多少键值对

多查询联想回忆（Multi-Query Associative Recall, MQAR）的输入包含若干 key-value 对和多个 query。模型必须在 query 出现时输出历史中对应的 value。例如

$$(A, 4), (B, 3), (C, 6), \dots, \quad C? \mapsto 6.$$

如果模型使用 full attention，历史 key/value 被显式保存在 KV cache 中，状态大小随长度增长；如果使用固定状态递推，所有 key/value 必须被压缩进一个不随 L 增长的状态。

BASED 论文系统地改变不同 recurrent mixer 的状态大小，并画出“memory-recall 帕累托前沿 (Pareto frontier)”。其结果表明：在同一架构内部，状态越大 recall 越好；不同架构在相同状态大小下，记忆利用效率也不同，但不存在“无限长精确 recall 且状态固定”的免费午餐。

更重要的是，论文给出通信复杂度下界：对长度为 N 的 MQAR，任何因果递推模型（causal 递推模型）若要求 exact solve，都需要

$$\boxed{\Omega(N) \text{ bits of 递推状态.}} \quad (176)$$

这意味着固定维度、固定精度的状态最终一定会有在足够长的精确回忆任务上失败。这个结论与具体采用 Mamba、DeltaNet 还是 KDA 无关；update rule 可以把 Pareto frontier 推得更好，却不能消除基本容量下界。

来源：BASED Sec. 3.1–3.2, Theorem 3.1.

建议分页位置

6.3 第 3 页：BASED 的 Pareto frontier——Attention 记得最准，但代价是状态随上下文增长

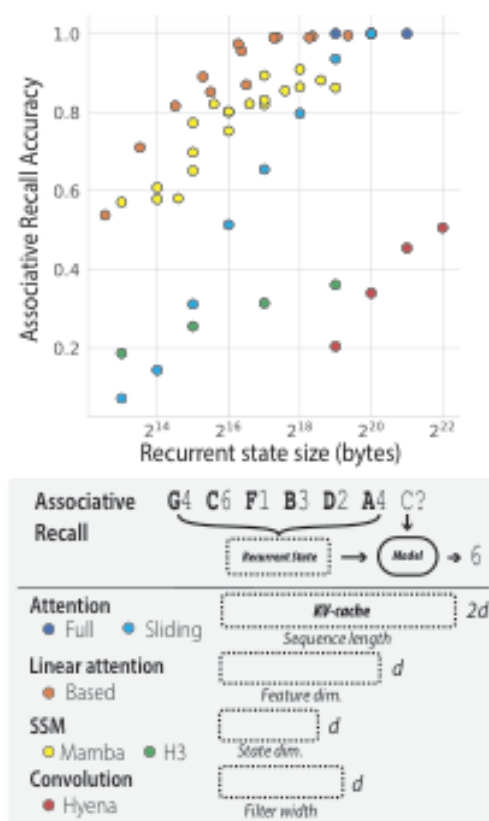


Figure 2. Throughput (memory) - recall tradeoff. x-axis

图 18: BASED Figure 2 裁剪：横轴是生成时递推状态大小，纵轴是 MQAR associative recall 准确率（accuracy）。Attention 位于高 recall/高 memory 端；Mamba 等有限状态模型位于低 memory 端。

这张图需要和前面的模型演化一起读：

- S4/Mamba 的价值不是“比 attention 存得更多”，而是用有限状态换取恒定/低增长推理内存；
- Delta/GDN/KDA 的价值是让相同有限状态更会管理 key-value，而不是打破容量下界；
- full attention 的 KV cache 随上下文增长，但它保留了 token-level exact addressing 所需的显式历史。

因此“有限状态 vs attention”不是简单的优劣关系，而是一个资源分配问题：

$$\text{更小的递推状态} \iff \text{更激进的信息压缩} \iff \text{精确回忆风险更高.} \quad (177)$$

这也是后续 hybrid 架构中“attention layer 不能完全删光”的第一个理由。

来源：BASED Fig. 2, Sec. 3.

建议分页位置

6.4 第 4 页：BASED 已经给出最小 Hybrid 原型——全局压缩记忆 + 很小的局部精确窗口

BASED 并没有简单回到 full attention，而是组合两种互补机制：

1. linear attention 提供全局、固定大小的压缩状态；
2. tiny sliding-window softmax attention（典型窗口 64–128）负责局部精确比较、shift 和 token 检索。

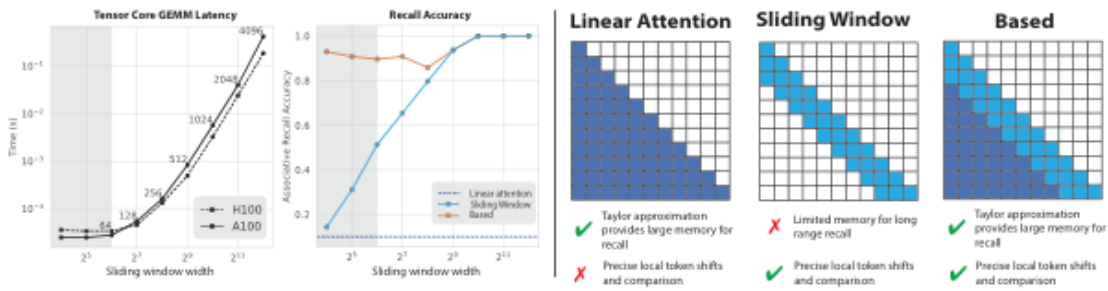


Figure 1. **BASED overview.** Combining linear attention with *tiny* sliding window softmax attention (e.g., 64 or 128 tokens in width) enables improved recall accuracy with limited efficiency overhead vs. smaller tile sizes. (Left) Time to execute Cutlass GEMMs (y) vs. sliding window attention size (x), with batch size 512 on tensor cores. (Center) Model recall accuracy (y) vs. sliding window attention size (x). We compare linear attention alone (dark blue), sliding window attention alone (light blue), and their combination (BASED).

图 19: BASED Figure 1 裁剪。右侧三幅示意图非常适合作为 Hybrid 的最小直觉：linear state 覆盖全局但模糊，sliding window 精确但范围小，两者组合得到 BASED。

其 linear attention 使用二阶泰勒特征映射（Taylor feature map）近似 softmax 核：

$$\phi(\mathbf{q})^\top \phi(\mathbf{k}) = 1 + \mathbf{q}^\top \mathbf{k} + \frac{(\mathbf{q}^\top \mathbf{k})^2}{2}. \quad (178)$$

对应递推状态可写成

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \phi(\mathbf{k}_t)^\top, \quad (179)$$

$$\mathbf{z}_t = \mathbf{z}_{t-1} + \phi(\mathbf{k}_t), \quad (180)$$

$$\mathbf{y}_t = \frac{\mathbf{S}_t \phi(\mathbf{q}_t)}{\mathbf{z}_t^\top \phi(\mathbf{q}_t)}. \quad (181)$$

它仍然是 fixed-state global memory；tiny softmax window 则保留一小段显式 KV cache。这个结构已经体现了后面所有 Hybrid LLM 的共同原则：把大多数 token mixing 交给低成本状态，把少数精确检索交给 attention。

来源：BASED Fig. 1, Sec. 4.

建议分页位置

6.5 第 5 页：状态跟踪是另一类边界——“有递推状态”不自动等于“能做真正的顺序状态计算”

The Illusion of State（《状态的幻觉》）提出一个重要反例：S4/Mamba 的内部确实有 state，但在固定深度、有限/对数精度等标准理论设定下，它们未必拥有一般 RNN 那样的状态跟踪表达能力。

一般化线性 SSM 可写成

$$\mathbf{h}_i = \bar{\mathbf{A}}_i \mathbf{h}_{i-1} + \bar{\mathbf{B}}_i \mathbf{x}_i, \quad (182)$$

其中 $\bar{\mathbf{A}}_i, \bar{\mathbf{B}}_i$ 可以依赖当前输入。展开后

$$\mathbf{h}_i = \sum_{j=1}^i \left(\prod_{k=j+1}^i \bar{\mathbf{A}}_k \right) \bar{\mathbf{B}}_j \mathbf{x}_j. \quad (183)$$

若这些 transition 是固定、对角或可同时对角化的，区间乘积可以被高度并行地计算。论文把这类计算归入 L-uniform TC^0 ；而某些困难状态跟踪问题（例如非交换置换的连续组合）是 NC^1 -complete。若采用通常的复杂度猜想

$$TC^0 \neq NC^1, \quad (184)$$

那么固定深度的这些 SSM 不能在任意长度上精确表达困难状态跟踪。

这不是说“SSM 实际上没有状态”，而是说：

一个可并行展开的线性状态表达，不一定等价于一般顺序 RNN 的计算能力。

来源：The Illusion of State Def. 2.1, Theorem 4.2/4.4, Corollary 4.7.

建议分页位置

6.6 第 6 页：先把 TC^0 与 NC^1 的含义讲清——这里关心的是“顺序组合复杂度”

基础知识补充：这里只需要知道的电路复杂度直觉

电路复杂度把一个计算写成许多可并行的逻辑/算术门，并用“深度”衡量必须串行经过多少层。 TC^0 可以粗略理解为常数深度、高度并行的一类计算； NC^1 允许对数深度，能表达更复杂的层次化组合。这里不需要掌握复杂度理论证明，只需要理解：如果一个任务本质上要求按顺序组合很多不可交换的状态更新，那么它可能比“把所有 token 并行聚合”更难。

对教学而言，不需要把电路复杂性课程全部搬进来，只需知道两点：

- TC^0 可理解为常数深度、高度并行的阈值电路类；大量求和、乘积、固定维矩阵运算都可并行完成；
- NC^1 允许对数深度，包含某些必须进行层次化/顺序组合才能完成的问题。

The Illusion of State 用置换组合说明区别。设有限状态 $x \in X$ ，每个 token 表示一个状态更新 $\delta_i : X \rightarrow X$ ，最终状态为

$$x_n = (\delta_n \circ \delta_{n-1} \circ \cdots \circ \delta_1)(x_0). \quad (185)$$

当这些更新是非交换的置换时，顺序非常重要：

$$\delta_2 \circ \delta_1 \neq \delta_1 \circ \delta_2.$$

这类计算无法只靠“把每个 token 独立映射后一次性聚合”完成。

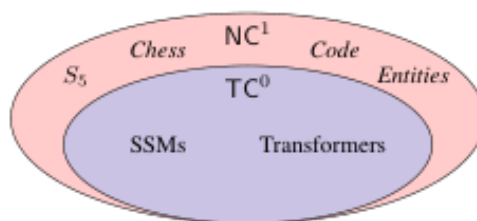


Figure 2: Complexity hierarchy within NC^1 . Transformers can only recognize languages within TC^0 (Merrill & Sabharwal, 2023a), and we show the same for SSMs (Theorems 4.2 and 4.4). Thus, both architectures cannot express the “hard state tracking” captured by NC^1 -complete problems like S_5 , which can be straightforwardly expressed by RNNs. The figure assumes the widely held conjecture $TC^0 \neq NC^1$.

图 20: The Illusion of State Figure 2 裁剪。论文在标准复杂度猜想下把常见 SSM 与 Transformer 放在 TC^0 一侧，而困难状态跟踪位于更强的 NC^1 。

这一理论结果解释了为什么“模型里有 h_t ”不能作为状态跟踪的证据；真正要看的是状态转移族 (transition family) 能否表达任务需要的非交换状态变换。

来源：The Illusion of State Sec. 2–4, Fig. 2.

建议分页位置

6.7 第 7 页：实验验证——S4/Mamba/Transformer 需要随序列增长不断加深，RNN 与更强 input-dependent SSM 不需要

论文使用有限群上的群词问题 / 置换组合作为状态跟踪基准 (state-tracking benchmark)。对每个输入 token，模型要执行一个置换；最终预测组合后的状态。实验测量达到 $> 90\%$ 验证准确率所需的最小层数。

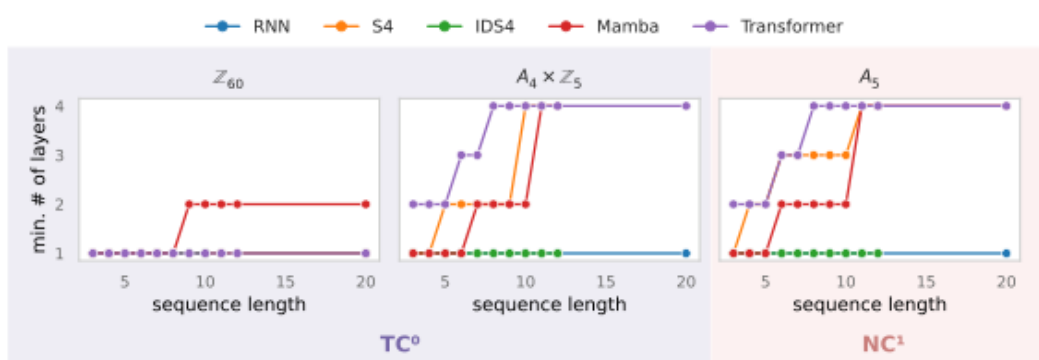


Figure 3: Minimum number of layers (lower is better) required to attain $> 90\%$ validation accuracy on group multiplication problems by sequence length and group. RNN and IDS4 models of constant depth can solve arbitrarily long sequences, while Transformer, S4, and Mamba models require depth proportional to the sequence length.

图 21: The Illusion of State Figure 3 裁剪。随着 sequence length 增大，Transformer、S4、Mamba 需要更深网络；RNN 和更强的输入相关 SSM (IDS4) 保持常数层数。

这张图的教学意义不是“RNN 一定比 Mamba 好”，而是明确区分长记忆和顺序状态计算：

- S4/Mamba 可以把远期信息稳定传播很长；

- 但“传播很远”并不意味着“能精确组合任意非交换状态更新”；
- transition matrix 的结构约束决定了可表达的状态计算类型。

后续 Delta/GDN 的广义豪斯霍尔德状态转移正好提供了比标量/对角衰减更丰富的状态变换。

来源：The Illusion of State Fig. 3, Sec. 6.

建议分页位置

6.8 第 8 页：把第五章和理论边界连起来——负特征值为什么突然变得重要

第五章式(148)已经得到 Delta transition

$$\mathbf{H}_t = \mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top, \quad \lambda_{\parallel} = 1 - \beta_t.$$

如果限制 $0 < \beta_t < 1$ ，所有特征值都非负，状态在当前 key 方向只会缩放，不会翻转。后续研究发现，为了表达 parity、swap、置换组合等 state-tracking 动力学，需要允许 transition 出现负特征值。

OLMo Hybrid 使用的 GDN 版本把低秩项写成 $2\beta_t$ ：

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - 2\beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \mathbf{v}_t \mathbf{k}_t^\top, \quad \alpha_t, \beta_t \in (0, 1). \quad (186)$$

若 $\|\mathbf{k}_t\| = 1$ ，沿 key 方向的特征值为

$$\lambda_{\parallel} = \alpha_t (1 - 2\beta_t), \quad (187)$$

当 $\beta_t > 1/2$ 时即可为负； $\beta_t = 1$ 时低秩部分就是 Householder reflection $\mathbf{I} - 2\mathbf{k}\mathbf{k}^\top$ 。

因此第五章“DeltaNet 可以把 β 扩展到 $(0, 2)$ ”并不是纯数学花活。它直接影响 state transition 是否能实现符号翻转、swap-like dynamics，从而决定某些 state-tracking 能力。

来源：OLMo Hybrid Definition 1；Grazzi et al. 相关结论由 OLMo Hybrid 和 Kimi Linear 引用；DeltaNet 脚注 2.

建议分页位置

6.9 第 9 页：现在可以形成一张真正有用的能力矩阵——Transformer 与 GDN 各强一边

到这里可以把两类边界统一起来：

表 24: 两类架构的优势不是同一维度。

	Full Attention / Transformer	GDN/Delta-style 递推状态
历史存储	显式 KV cache，随长度增长	固定大小矩阵状态
Exact recall	强：可直接按 query 访问历史 token	有容量下界，长度足够大时会压缩丢失
State tracking	固定深度下对 hard sequential composition 有理论限制	更强 transition（尤其允许负特征值）可表达更复杂状态变换
推理内存	长上下文成本高	近似常数状态
训练并行	高度并行	需 chunkwise/scan，但可保持高并行度

因此 Hybrid 的理论动机可以非常具体地写成

$$\text{Hybrid} = \underbrace{\text{递推状态}}_{\text{压缩历史 + 顺序状态计算}} + \underbrace{\text{attention}}_{\text{token-level 精确检索}}. \quad (188)$$

后面的 OLMo Hybrid 不再只是工程上“混几层”，而是构造了一个同时需要状态跟踪和 recall 的任务，证明二者组合可以表达单一机制无法表达的问题。

[建议分页位置](#)

6.10 第 10 页：State-Based Recall——为什么单纯“追踪状态”或单纯“检索”都不够

OLMo Hybrid 定义 state-based recall：输入由三部分组成

$$(\mathbf{x}, \mathbf{p}, \pi),$$

其中 $\mathbf{x} \in \{0, 1\}^n$ 是 bitstring, $p_1, \dots, p_5 \in [1, n]$ 是指针, $\pi_1, \dots, \pi_m$ 是作用在五个指针上的 swap/transposition。先做状态更新

$$\mathbf{q} = (\pi_m \circ \dots \circ \pi_1)(\mathbf{p}), \quad (189)$$

再根据更新后的第一个指针去 bitstring 中做精确检索

$$y = x_{q_1}. \quad (190)$$

这个任务故意把两个能力串在一起：

$$\text{swap sequence} \xrightarrow{\text{状态跟踪}} q_1 \xrightarrow{\text{精确回忆}} x_{q_1}.$$

如果只有 attention，前半段困难状态跟踪受限制；如果只有固定状态 GDN，后半段随着 bitstring 长度增加会碰到 recall 容量限制。

论文给出的 separation 结论是：在其理论设定与标准复杂度猜想下，带负特征值的 GDN + averaging-hard attention 只需一次 layer-type alternation 就能完成 state-based recall，而 pure architecture 各自缺少其中一个组成能力。

来源：OLMo Hybrid Definition 2, Theorem 1, Sec. 3.3.

[建议分页位置](#)

6.11 第 11 页：OLMo 的合成实验把这两个轴分别测出来——Hybrid 是唯一在组合任务上保持稳健的

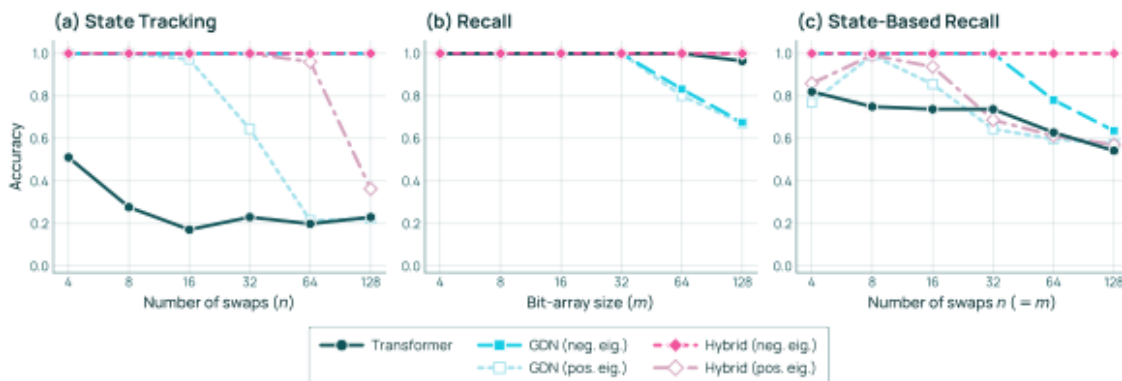


Figure 7 Synthetic task results. **(a)** State tracking accuracy vs. number of updates n : linear RNN and hybrid remain near-perfect, while the transformer falls quickly. **(b)** Recall accuracy vs. bit-array size m : transformer and hybrid maintain perfect recall, while the linear RNN degrades for large m . **(c)** State-based recall accuracy vs. number of swaps n ($m = n$): the hybrid remains robust, while both the transformer and linear RNN degrade for large m, n .

图 22: OLMo Hybrid Figure 7 裁剪。左：状态跟踪；中：recall；右：state-based recall。负特征值 GDN 与 Hybrid 在 tracking 上稳定，Transformer/Hybrid 在 recall 上稳定，只有 Hybrid 在组合任务上同时保持两种能力。

三幅图应当按列解释：

1. **State Tracking**：更新次数增加时，Transformer 快速退化；GDN/Hybrid（允许负特征值）保持近乎完美。
2. **Recall**：bit-array 变长时，纯 GDN 逐渐下降；Transformer/Hybrid 基本保持。
3. **State-Based Recall**：两个难度同时增长时，纯 Transformer 和纯 GDN 都下降，Hybrid 最稳健。

这比只看语言模型平均分更有教学价值，因为它直接把 architecture 中的不同 memory primitive 与对应能力一一对上。

来源：OLMo Hybrid Fig. 7, Sec. 3.5.

建议分页位置

6.12 第 12 页：Hybrid 还有第二个动机——状态大小和 KV cache 的数量级差异巨大

OLMo Hybrid 采用与 OLMo 3 7B 类似的整体结构，但约 75% token-mixing 层使用 GDN，25% 使用 multi-head attention，即

3 GDN layers : 1 Attention layer.

在 32K 上下文、FP16 下，论文给出的单层推理状态对比如下：

表 25: OLMo Hybrid Table 1: 单层 inference state 大小。

层类型	FP16 状态大小	相对 GDN
Multi-Head Attention	512 MiB	485×
Grouped-Query Attention	128 MiB	121×
Grouped-Query SWA	16.0 MiB	15.2×
OLMo Hybrid GDN	1.05 MiB	1×

这说明 Hybrid 并不需要保留很多 attention 层就能获得 exact-检索路径。只要把大多数层替换为固定状态 recurrence, KV-cache 压力就能显著下降; 少量 attention 层则负责处理递推状态不擅长的精确检索。



图 23: OLMo Hybrid 的一个 4 层 token-mixing 周期。前三层使用固定大小的 GDN 递推状态, 第 4 层保留显式 token-level attention; 随后重复这一周期。

这张结构图应与上表一起读: 如果令一个周期的 attention 比例为 $r = 1/4$, 则长期上下文中的显式 KV cache 只由这部分 attention 层产生, 而其余 3/4 的 token mixing 只维护固定维度状态。工程收益因此不是“GDN 本身比 attention 少一点内存”, 而是把随上下文长度 L 增长的状态

$$\mathcal{O}(Ld_{\text{kv}})$$

替换为与 L 无关的递推状态

$$\mathcal{O}(d_k d_v),$$

同时仍定期保留一个不经过压缩的全局检索通道。

来源: OLMo Hybrid Sec. 2.1, Table 1.

建议分页位置

6.13 第 15 页: Kimi Linear 与 Qwen3-Next 说明 3:1 已成为一种常见工程折中, 但不是理论常数

第五章已经看到 Kimi Linear 默认采用

$$3 \text{ KDA} : 1 \text{ Full MLA.}$$

Qwen3-Next 则采用

$$3 \text{ Gated DeltaNet} : 1 \text{ standard attention,}$$

即约 75% 层用 GDN, 25% 保留标准 attention。其技术说明明确指出, 这种 3:1 hybrid 比纯 GDN 或纯 attention 的单一结构在性能与效率上更均衡。

但“3:1”不能被当成理论定律。它的真正含义是:

$$\text{总成本} \approx n_{\text{state}} C_{\text{state}} + n_{\text{attn}} C_{\text{attn}}, \quad (191)$$

同时模型能力受到两条通路约束：

$$\text{state path : 压缩/状态计算,} \quad \text{attention path : 精确检索.} \quad (192)$$

最优比例取决于训练长度、部署长度、状态大小、attention 实现、模型宽度和数据任务组成。

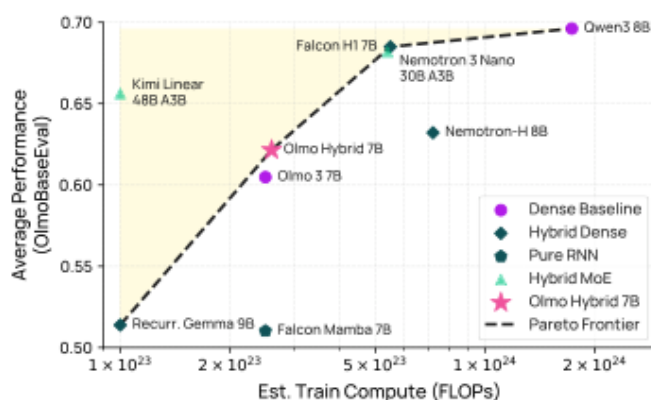
Kimi Linear 论文自己做 ratio ablation 后选择 3:1；OLMo Hybrid 也在 1:1、3:1、7:1 等配置中选择 3:1 interleaved GDN；Jamba 则选择更激进的 1:7 attention:Mamba。这些差异恰好说明：比例是一个架构超参数，而不是“混合模型的固定公式”。

来源：Kimi Linear Sec. 4/5.2；Qwen3-Next 技术报告 hybrid attention 说明；OLMo Hybrid Sec. 5.1；Jamba Sec. 2。

建议分页位置

6.14 第 16 页：OLMo Hybrid 的重点不只是“省 cache”——它把 Hybrid 放到表达力与缩放规律（scaling law）两个层面验证

OLMo Hybrid 7B 使用 3:1 GDN-to-attention 的 interleaved pattern，并尽量保持和 OLMo 3 相同的训练配置，从而做较受控比较。论文不仅比较最终 benchmark，还从 compute–performance Pareto frontier 和 scaling law 分析 Hybrid。



performance tradeoff of open-weight hybrid, RNN, and transformer base models on task suites. OLMo Hybrid 7B is on the Pareto frontier of open-weight dense models using the $C = 6ND$ heuristic from Kaplan et al. (2020), with reported token a

图 24: OLMo Hybrid Figure 2 裁剪：在公开 dense/hybrid/RNN 模型的训练计算量–性能平面上，OLMo Hybrid 7B 位于 Pareto frontier。

其 Chinchilla-style scaling law 写成

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (193)$$

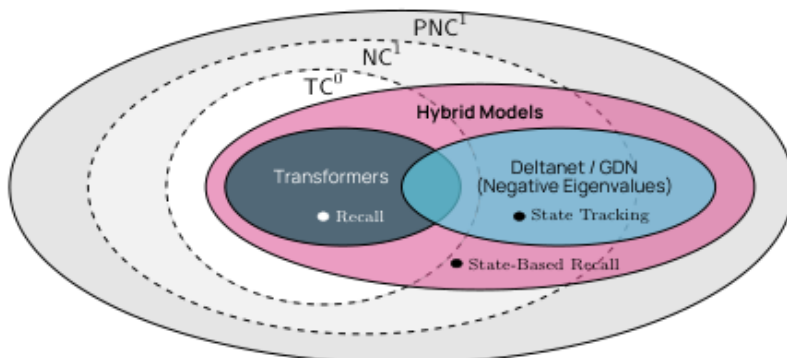
其中 N 是参数规模， D 是训练 token 数。论文发现 Hybrid 的 data coefficient B 明显更低；按拟合曲线，在相同目标 loss 下，Hybrid 可在大规模模型上使用更少训练 token。文中给出的投影包括约 $1.68 \times$ (7B)、 $1.82 \times$ (30B)、 $1.89 \times$ (70B) 的 token savings factor。

这些结果不能被解读成“所有 hybrid 必然省这么多数据”，但它们说明 Hybrid 的收益可能不只来自部署期 KV cache，还可能改变模型在有限 compute/data 预算下的表达效率。

来源：OLMo Hybrid Fig. 2, Sec. 4, Eq. scaling-law, 文中 Figure 8–10 的 fitted results。

建议分页位置

6.15 第 17 页：Hybrid 的理论图景——为什么 OLMo 说它可能比两种纯架构都更有表达力



expressive power of transformers, linear RNNs, and hybrid models relative to circuit complexity classes. They represent unproven but *conjectured* separations between classes (e.g., $TC^0 \neq NC^1$). Notably, transformers express recall (Section 3.1), and DeltaNet (or GDN) with negative eigenvalues can express state tracking (Section 3.2, 2025). Hybridizing gives both capabilities, and we prove that it also unlocks state-based recall, a primitive model can express on its own (Section 3.3). With the addition of padding tokens, transformers can express class TC^0 , whereas hybrid models can capture all of NC^1 , which enables solving boolean formula evaluation.

图 25: OLMo Hybrid Figure 3 裁剪。Transformer 擅长 recall；带负特征值的 Delta/GDN 擅长状态跟踪；Hybrid 同时覆盖两者，并能表达 state-based recall。虚线边界依赖复杂度类未证明分离假设。

这张图把整个第六章压缩成一句话：

Hybrid 不是把两个“差不多”的 mixer 混起来，而是组合两种互补计算原语。

Attention 具有动态、显式、token-level addressability；递推状态具有固定内存、顺序状态更新和低推理成本。两种原语交替后，前一层可以把问题变换成后一层擅长的形式。例如 state-based recall 有两种求解顺序：

方案 A: GDN tracking $\rightarrow$ attention recall, (194)

方案 B: attention prefetch $\rightarrow$ GDN composition. (195)

论文理论上证明只需要一次 alternation 就足以完成其构造任务；实际大模型则采用多次周期性交替，以兼顾层级表征和工程均衡。

来源：OLMo Hybrid Fig. 3, Theorem 1/3, Sec. 3.

建议分页位置

6.16 第 19 页：如何选择 Hybrid——先判断任务需要什么信息，再决定哪些信息压缩、哪些信息保留显式索引

从前面所有模型可以抽象出四类历史信息：

1. **平滑趋势 / 时间尺度**：适合 SSM/decay state；
2. **当前 key 的关联值**：适合 delta/KDA 这类可编辑 associative memory；
3. **顺序状态变量**：需要 transition 足够表达状态跟踪；
4. **任意 token-level 细节**：需要显式 attention/检索路径。

因此 architecture design 可以写成一个信息分配问题：

$$\mathcal{H}_t = \underbrace{\mathcal{C}(x_{1:t})}_{\text{compressed 递推状态}} \cup \underbrace{\mathcal{E}(x_{1:t})}_{\text{explicit retrievable history}}, \quad (196)$$

其中 $\mathcal{C}$ 的大小近似固定， $\mathcal{E}$ 的大小随显式 attention window/layer 数增长。模型真正要优化的是

$$\min \text{compute/memory} \quad \text{s.t.} \quad \text{需要的 recall 与 state computation 不丢失}. \quad (197)$$

对应到实际选择：

- 传感/时序趋势占主导：可提高 state-layer 比例；
- 代码、检索、长文档问答：需要更强 attention path；
- 具有环境状态更新的 agent/机器人：应特别关注 recurrent transition 的 state-tracking 能力；
- 长上下文 LLM：通常需要在 KV cache、throughput 和精确回忆之间寻找任务相关 Pareto frontier。

Hybrid 的核心不在“混了几种层”，而在于把不同类型的记忆职责分给最合适的计算原语。

建议分页位置

6.17 第 20 页：第五、六章收束——状态更新越来越聪明，但最终必须承认“有限压缩”和“显式检索”是两种不同资源

从第三章开始，状态模型的演化可以重新整理为一条连续逻辑：

- Mamba: 按内容选择写入/保留/读出,
- Mamba-2: 把 selective recurrence 重新组织成 SSD/block computation,
- DeltaNet: 把加法写入改成 reconstruction-error correction,
- GDN: 把 global 遗忘与 targeted overwrite 组合,
- KDA: 把 scalar 遗忘细化为 channel-wise decay,
- Hybrid: 把固定状态压缩与显式精确检索分工.

其中有两个贯穿始终的数学主题：

1. **表达力**：状态 transition / write rule 决定“有限状态会算什么”；
2. **计算结构**：每次提高表达力后，都要重新寻找 scan、WY、chunkwise、SSD、sparse attention 等并行实现。

也有两个不能混淆的极限：

有限状态容量限制精确回忆 $\neq$ 受限 transition 限制状态跟踪.

这一区分会直接服务后面的应用章节：时间序列、视觉、视频、机器人为什么对“记忆”的要求不同，也就能用同一套语言判断。

表 26: 第五、六章核心模型的最终定位。

模型	最关键数学改动	解决的主要问题
DeltaNet	$I - \beta k k^\top$ + prediction-error write	同一 key 的旧关联被精确修正
GDN	$\alpha_t(I - \beta k k^\top)$	targeted overwrite + adaptive 遗忘
KDA	$(I - \beta k k^\top) \text{Diag}(\alpha_t)$	通道级 memory lifetime 管理
Full Attention	显式 $QK^\top$ 全历史查询	exact token-level 检索
Hybrid	recurrent + attention 两条 memory path	recall、状态跟踪、效率三者折中

主线中场总结：到这里，“状态记忆”已经经历了三次本质升级

前六章可以用三类状态更新直接对齐。第一类是**线性传播型状态**：

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \quad (198)$$

S4 的重点是让这个 fixed recurrence 既记得长又算得快。第二类是**选择性传播型状态**：

$$\mathbf{h}_t = \bar{\mathbf{A}}_t\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t, \quad (199)$$

Mamba 让更新依赖 token，Mamba-2 再把同一函数写成 structured matrix 以获得高效块计算。第三类是**关联记忆型状态**：

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \quad (200)$$

它不仅能让旧信息衰减，还能沿当前 key 方向读取旧值、计算误差并覆写。

但这三类方法有一个共同点：**状态大小都不随上下文长度 T 增长**。因而它们的优势是固定或近似固定的推理内存，边界则是无法对任意长历史做无损精确回忆。第六章引入 Hybrid 的原因由此变得非常简单：

可压缩的历史 $\rightarrow$ state, 必须精确索引的历史 $\rightarrow$ attention / 检索.

第七章的应用选择、第八章的实践验证都以这条分工原则为准。

建议分页位置

7 第七章：状态模型何时适用——任务记忆结构、扫描路径与应用边界

本章只解决一个问题：把状态模型迁移到时间序列、图像、视频和机器人时，真正需要重新设计的是什么？

本章给出的答案：先判断 token、扫描路径、状态含义和精确检索需求，再决定 SSM、Mamba 或 Hybrid 是否合适。

7.1 第 1 页：应用章节先解决一个总问题——把 Mamba 放进新模态，真正需要重新设计的是什么？

前六章已经把 SSM 的核心对象固定下来：对于一维序列 $\{\mathbf{x}_t\}_{t=1}^T$ ，状态模型通过

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t, \quad (201)$$

把历史压缩进有限状态。S4 中 $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t, \mathbf{C}_t$ 固定；Mamba 中 $\mathbf{B}_t, \mathbf{C}_t, \Delta_t$ 由当前 token 产生，并通过

$$\bar{\mathbf{A}}_t = e^{\Delta_t \mathbf{A}}, \quad \bar{\mathbf{B}}_t = \left(\int_0^{\Delta_t} e^{\tau \mathbf{A}} d\tau \right) \mathbf{B}_t \quad (202)$$

得到输入相关的离散递推。

到了时间序列、图像、视频和机器人，真正的新问题不是“再抄一遍式(201)”，而是必须回答四件事：

1. **token 是什么：**时间点、patch、时空 tubelet、动作 token，还是多模态 token？
2. **扫描顺序是什么：**哪一个几何/时间顺序被当作递推方向？
3. **状态应该压缩什么：**趋势、空间上下文、视频阶段、动作历史，还是任务进度？
4. **哪些关系不适合被压缩：**是否还需要 attention、cross-attention 或显式局部模块？

因此应用章节统一采用如下判断顺序：

$$\text{数据几何} \rightarrow \text{扫描路径} \rightarrow \text{状态含义} \rightarrow \text{局部/全局分工} \rightarrow \text{是否需要 Hybrid.} \quad (203)$$

表 27: 不同任务的“记忆结构”并不相同，因此不能把同一种 Mamba block 原样搬过去。

任务	自然递推轴	主要希望状态保存	额外困难
音频/传感	时间	多尺度波形、周期、远期上下文	极长采样序列、AR 稳定性
多变量时序	时间 + 变量	时间趋势与跨变量依赖	变量没有天然线性顺序
图像	无天然时间轴	全局空间上下文	必须人为定义 2D scan
视频	空间 + 时间	长时动作/事件与空间上下文	$T \times H \times W$ token 爆炸
机器人策略	交互时间/动作块	任务阶段、动作协调、观测历史	精确 grounding、安全与闭环反馈

应用 Mamba 的第一步不是选模型名，而是先明确“状态沿哪条路径传播，以及这条路径是否符合任务结构”。

建议分页位置

7.2 第 2 页：先看最自然的场景——音频为什么几乎不需要重新定义扫描顺序？

原始音频波形本身就是严格的一维时间信号。若采样率为 f_s ，长度为 T 秒，则序列长度为

$$L = T f_s. \quad (204)$$

例如 16 kHz 音频仅 10 秒就有 1.6×10^5 个样本。此时 SSM 的递推方向和物理时间完全一致：

$$\mathbf{x}_1 \rightarrow \mathbf{x}_2 \rightarrow \cdots \rightarrow \mathbf{x}_L. \quad (205)$$

因此 S4/SaShiMi 的核心优势可以直接发挥：长卷积核负责大时间尺度依赖，递推形式又适合自回归生成。

SaShiMi 的设计不是单纯堆 S4，而是把**多尺度架构**与 S4 长核结合。设第 r 个尺度下采样倍率为 s_r ，则同一个物理时间跨度在更低分辨率上只需

$$L_r = \frac{L}{\prod_{j \leq r} s_j} \quad (206)$$

个 token 表示；这样深层状态可以用较短序列覆盖更长时间范围。其设计直觉是：

高分辨率层：局部波形细节	低分辨率层：长时间结构.
--------------	--------------

(207)

SaShiMi 还指出原始 S4 在自回归生成时可能出现状态不稳定，因此需要对状态参数化作稳定化处理。这说明一个重要边界：**离线分类/预测稳定，不自动等于自由运行的自回归递推稳定。**

来源：Goel et al., *It's Raw! Audio Generation with State-Space Models (SaShiMi)*, Sec.4–5.

建议分页位置

7.3 第 3 页：多变量时间序列比音频多一个难题——“变量”本身没有天然的线性顺序

考虑多变量预测输入

$$\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}]^\top \in \mathbb{R}^{K \times L}, \quad \mathbf{x}^{(k)} \in \mathbb{R}^L, \quad (208)$$

目标预测未来 T 步

$$\hat{\mathbf{Y}} \in \mathbb{R}^{K \times T}. \quad (209)$$

时间轴 $1, \dots, L$ 是自然的，但变量 $V_1, \dots, V_K$ 往往没有天然顺序。若简单展平（flatten）成

$$(V_1, V_2, \dots, V_K) \quad (210)$$

再扫描，相邻变量可能统计上毫无关联；而 Mamba 的递推会天然给“相邻 token”更直接的信息通路。

MambaTS 因而把问题拆成两个层次：

temporal dependency + inter-variable dependency.
--

(211)

第一部分仍由时间递推处理，第二部分则需要为变量寻找一个**依赖感知的扫描顺序**。

如果存在变量依赖 DAG $G = (\mathcal{V}, \mathcal{E})$ ，其拓扑序 σ 满足

$$(V_i \rightarrow V_j) \in \mathcal{E} \Rightarrow \sigma^{-1}(i) < \sigma^{-1}(j), \quad (212)$$

那么按该顺序线性扫描即可逐步条件化：

$$P(\mathcal{V}) = \prod_{m=1}^K P(V_{\sigma(m)} \mid \text{Pa}(V_{\sigma(m)})). \quad (213)$$

这给出了 MambaTS “一个线性 scan 也能承载全局变量依赖” 的理论直觉。

来源：MambaTS, Sec.4 Motivation, Proposition 1.

建议分页位置

7.4 第 4 页：MambaTS 先给整体方案——先解释 VST、VAST 与 TMB 各自解决什么

术语提示： MambaTS 把三个功能拆开命名：**VST** 是沿时间的变量扫描（Variable Scan along Time），负责把“变量 $\times$ 时间块”排成序列；**VAST** 是变量感知扫描树（Variable-Aware Scan Tree），负责根据变量关系改变扫描顺序；**TMB** 是时间 Mamba 模块（Temporal Mamba Block），负责沿已经确定的顺序执行状态递推。先记功能，再记缩写。

MambaTS 先对每个变量分 patch。若 patch stride/length 记为 P ，则每个变量大约形成

$$M = \left\lceil \frac{L}{P} \right\rceil \quad (214)$$

个 token。第 k 个变量第 m 个 patch 经线性映射后为

$$\mathbf{z}_m^{(k)} = W \mathbf{x}_m^{(k)} \in \mathbb{R}^D. \quad (215)$$

随后采用 VST 把不同变量与时间 patch 交错成一个序列，例如

$$\mathbf{Z} = (\mathbf{z}_1^{(1)}, \mathbf{z}_1^{(2)}, \dots, \mathbf{z}_1^{(K)}, \mathbf{z}_2^{(1)}, \dots, \mathbf{z}_M^{(K)}). \quad (216)$$

这样一个 recurrent mixer 同时沿时间和变量方向积累信息。

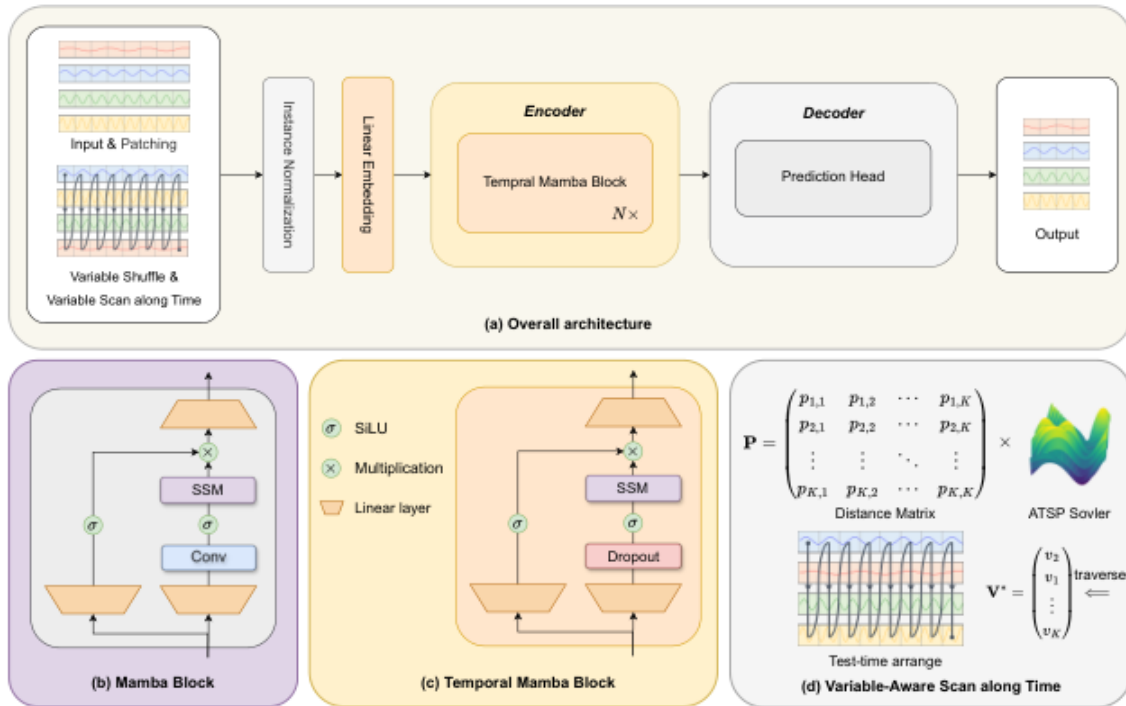


图 26: MambaTS 总体结构。左上：分块（patching）与变量扫描；中部：TMB；右下：VAST 在测试时根据学习到的变量关系重新排列扫描顺序。来源：MambaTS Fig.1。

原始 Mamba block 的局部 causal convolution 对语言 token 很有意义，但变量交错后，相邻 token 未必具有稳定的局部平移结构，因此 TMB 去掉 causal Conv，并在 selective parameter 路径加入随机失活 (dropout)。论文给出的简化写法为

$$\text{Mamba: } \mathbf{h} = \text{SSM}(\text{Conv}(W\mathbf{z})) + \sigma(W_g\mathbf{z}), \quad (217)$$

$$\text{TMB: } \mathbf{h} = \text{SSM}(\text{Dropout}(W\mathbf{z})) + \sigma(W_g\mathbf{z}). \quad (218)$$

MambaTS 的关键并不是“时间序列用了 Mamba”，而是意识到：扫描顺序本身就是多变量依赖建模的一部分。

来源：MambaTS, Fig.1 and Sec.5.1.

建议分页位置

7.5 第 6 页：MambaTS 的复杂度为什么有吸引力——跨变量建模不一定要 K^2

设 K 为变量数， L 为 look-back 长度，patch 大小/stride 为 P ，则 token 数约为

$$M \approx L/P, \quad (219)$$

MambaTS 的主扫描复杂度为

$$\mathcal{O}(KM) = \mathcal{O}(KL/P). \quad (220)$$

相比之下，变量维 self-attention 通常包含 $\mathcal{O}(K^2)$ 的配对成本。对 Traffic 数据，论文给出 $K = 862, L = 720, P = 48$ ，此时

$$M = 15, \quad KM = 12930, \quad K^2 = 743044, \quad (221)$$

数量级差异已经非常明显。

表 28: MambaTS 论文 Table 5 的复杂度信息重新整理。SA 表示 self-attention。

方法	时间混合	变量混合	主要复杂度
PatchTST	SA	无显式变量混合	约 $\mathcal{O}((L/P)^2)$
iTransformer	MLP	SA	$\mathcal{O}(K^2)$
FourierGNN	GNN	GNN	$\mathcal{O}(KL)$
MambaTS	TMB	TMB/VAST scan	$\mathcal{O}(KL/P)$

注意这不是说线性复杂度自动带来更好预测。MambaTS 的价值在于：它通过**结构化扫描**仍保留显式的跨变量交互，而不是简单采用 channel-independent 假设。

来源：MambaTS, Table 5 and Sec.6.3.

[建议分页位置](#)

7.6 第 7 页：从时间序列进入图像前，先看 S4ND——多维 SSM 可以从“多时间轴”而不是“flatten”出发

图像没有自然的一维时间轴。S4ND 的思路不是先 flatten 再扫描，而是把二维坐标 $(t^{(1)}, t^{(2)})$ 当成两个独立连续轴。对于输入

$$u = u(t^{(1)}, t^{(2)}), \quad (222)$$

二维 SSM 可以写成沿两个轴的线性偏微分方程（PDE）：

$$\frac{\partial x}{\partial t^{(1)}} = \mathcal{A}^{(1)}x + B^{(1)}u, \quad (223)$$

$$\frac{\partial x}{\partial t^{(2)}} = \mathcal{A}^{(2)}x + B^{(2)}u, \quad (224)$$

$$y = \langle C, x \rangle. \quad (225)$$

其二维卷积核为

$$K(t^{(1)}, t^{(2)}) = \left\langle C, \left(e^{t^{(1)} A^{(1)}} B^{(1)} \right) \otimes \left(e^{t^{(2)} A^{(2)}} B^{(2)} \right) \right\rangle. \quad (226)$$

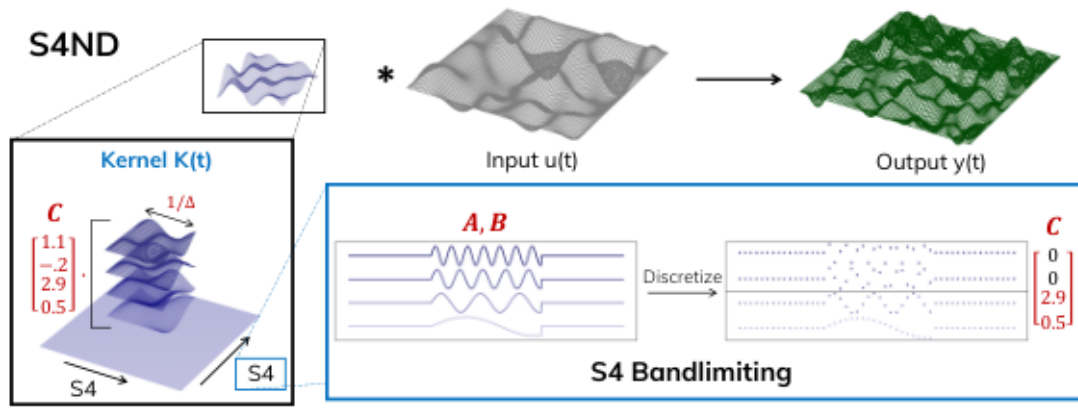


Figure 1: (S4ND.) (Parameters in red.) (Top) S4ND can be viewed as a depthwise convolution that maps a multidimensional input (black) to output (green) through a continuous convolution kernel (blue). (Bottom Left) The kernel can be interpreted as a linear combination (controlled by C) of basis functions (controlled by A, B) with flexible width (controlled by step size Δ). For structured C , the kernel can further be factored as a low-rank tensor product of 1D kernels, and can be interpreted as independent S4 transformations on each dimension. (Bottom Right)

图 27: S4ND 将二维/多维数据视作连续信号，并用各维的一维 S4 kernel 外积形成全局 ND kernel。来源：S4ND Fig.1。

如果 C 具有低秩张量分解

$$C = \sum_{r=1}^R C_r^{(1)} \otimes C_r^{(2)}, \quad (227)$$

则二维 kernel 也分解为

$$K(t^{(1)}, t^{(2)}) = \sum_{r=1}^R K_r^{(1)}(t^{(1)}) \otimes K_r^{(2)}(t^{(2)}), \quad (228)$$

因此高维全局卷积仍可由若干一维 SSM kernel 构造。

来源：S4ND, Definition 1, Theorem 1, Corollary 4.1.

建议分页位置

7.7 第 8 页：视觉 Mamba 的根本问题——若采用一维递推，必须说明二维网格怎样变成序列

对于 patch grid

$$X \in \mathbb{R}^{H \times W \times D}, \quad (229)$$

任何 1D Mamba 都需要一个双射

$$\pi: \{1, \dots, HW\} \rightarrow \{1, \dots, H\} \times \{1, \dots, W\}, \quad (230)$$

把网格映射为序列

$$\mathbf{z}_t = X_{\pi(t)}. \quad (231)$$

于是第 j 个位置对第 i 个位置的可达路径不再只由二维欧氏距离决定，而由扫描次序决定。若 $\pi^{-1}(p) \ll \pi^{-1}(q)$ ，则信息必须经过很多递推步骤才能从 p 到 q 。

因此视觉 SSM 的设计自由度可以写成

$$\text{视觉 SSM} = \text{patch embedding} + \text{scan permutation(s)} + \text{SSM mixer} + \text{merge/readout}. \quad (232)$$

同一个 Mamba mixer，换掉 π ，模型的空间归纳偏置就已经发生变化。

这正是 Vision Mamba、VMamba、MambaVision 等工作的差异来源之一。

[建议分页位置](#)

7.8 第 9 页：Vision Mamba (Vim)——最直接的修正是“前向 + 后向”双向状态传播

单向递推只能让位置 t 读取其扫描前缀。Vim 为图像构造前向与后向两套递推：

$$\mathbf{h}_t^{\rightarrow} = \bar{\mathbf{A}}_t^{\rightarrow} \mathbf{h}_{t-1}^{\rightarrow} + \bar{\mathbf{B}}_t^{\rightarrow} \mathbf{z}_t, \quad (233)$$

$$\mathbf{h}_t^{\leftarrow} = \bar{\mathbf{A}}_t^{\leftarrow} \mathbf{h}_{t+1}^{\leftarrow} + \bar{\mathbf{B}}_t^{\leftarrow} \mathbf{z}_t, \quad (234)$$

再融合两方向读出：

$$\mathbf{y}_t = \mathbf{C}_t^{\rightarrow \top} \mathbf{h}_t^{\rightarrow} + \mathbf{C}_t^{\leftarrow \top} \mathbf{h}_t^{\leftarrow}. \quad (235)$$

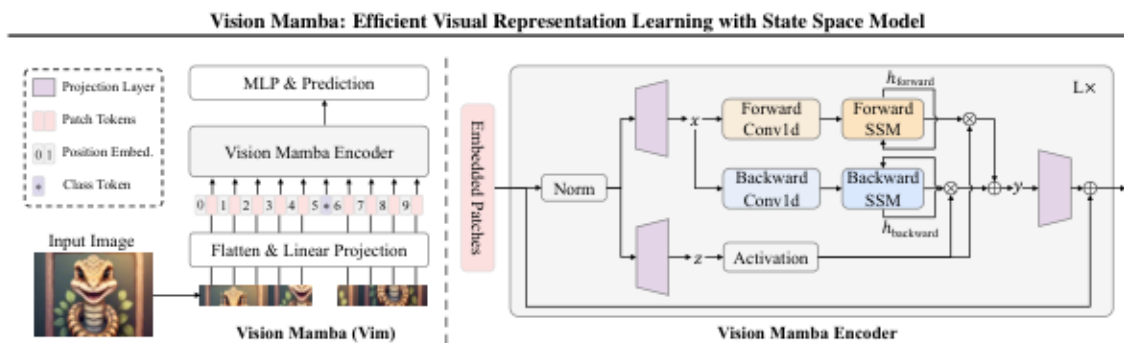


Figure 2: The overview of the proposed Vim model. We first split the input image into patches, and then project them into patch tokens. Last, we send the sequence of tokens to the proposed Vim encoder. To perform ImageNet classification, we concatenate an extra learnable classification token to the patch token sequence. Different from Mamba for text sequence

图 28: Vision Mamba 的 patch 序列与双向 Vim encoder。来源：Vision Mamba Fig.2。

Vim 的效率比较也非常直观。若 token 数为 M 、通道维度为 D 、状态维度 N 固定，则论文给出

$$\Omega(\text{self-attention}) = 4MD^2 + 2M^2D, \quad (236)$$

$$\Omega(\text{SSM}) = 3M(2D)N + M(2D)N, \quad (237)$$

后者对 M 为线性复杂度。这说明视觉 SSM 的工程优势会随分辨率和 token 数增长而放大。

来源：Vision Mamba, Sec.3.2–3.5, Fig.2, Eq.(5)–(6).

[建议分页位置](#)

7.9 第 10 页：VMamba——双向还不够，二维图像最好沿多条空间路径同时扫描

Vim 仍把二维 patch 串成一条主序列。VMamba 的 SS2D 更明确地保留二维几何：定义四条扫描路径 π_r ，例如

$$r \in \{\rightarrow, \leftarrow, \downarrow, \uparrow\}, \quad (238)$$

对每条路径独立执行 selective SSM:

$$\mathbf{z}_t^{(r)} = X_{\pi_r(t)}, \quad (239)$$

$$\mathbf{h}_t^{(r)} = \bar{\mathbf{A}}_t^{(r)} \mathbf{h}_{t-1}^{(r)} + \bar{\mathbf{B}}_t^{(r)} \mathbf{z}_t^{(r)}, \quad (240)$$

$$\tilde{\mathbf{Y}}_{\pi_r(t)}^{(r)} = \mathbf{C}_t^{(r)\top} \mathbf{h}_t^{(r)}, \quad (241)$$

最后进行 cross-merge:

$$\mathbf{Y} = \text{Merge} \left(\tilde{\mathbf{Y}}^{(1)}, \dots, \tilde{\mathbf{Y}}^{(4)} \right). \quad (242)$$

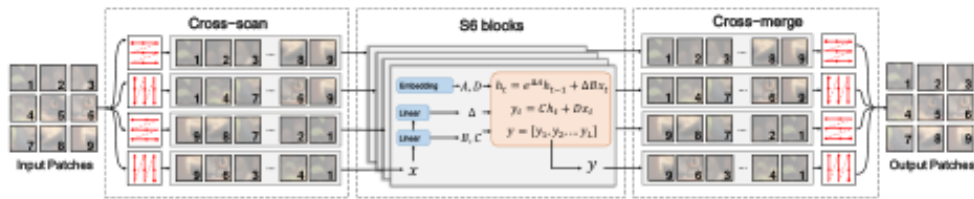


Figure 2: Illustration of 2D-Selective-Scan (SS2D). Input patches are traversed along four different scanning paths (*Cross-Scan*), with each sequence independently processed by separate S6 blocks. The results are then merged to construct a 2D feature map as the final output (*Cross-Merge*).

图 29: VMamba 的 2D Selective Scan (SS2D): Cross-Scan 将二维 patch 展开为四条序列, 经 S6 block 后再 Cross-Merge 回二维空间。来源: VMamba Fig.2。

这时“状态路径”已经成为模型结构本身: 同一像素可以从左右、上下多个方向获得全局上下文, 而不必等待一条蛇形序列经过整个图像。

来源: VMamba, Sec.4.2 and Fig.2.

[建议分页位置](#)

7.10 第 11 页: MambaVision——视觉上的经验结论逐渐变成 Hybrid: 低层局部结构与高层全局关系不必由同一种 mixer 负责

MambaVision 采用分层视觉 backbone。前两 stage 用卷积提取高分辨率局部特征; 后两 stage 使用 MambaVision mixer, 并在更深层穿插 Transformer block。

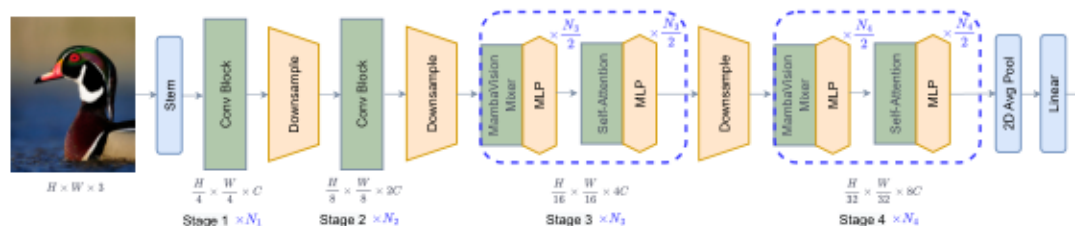


Figure 2 – The architecture of hierarchical MambaVision models. The first two stages use residual convolutional blocks for fast feature extraction. Stages 3 and 4 employ both MambaVision and Transformer blocks. Specifically, given N layers, we use $\frac{N}{2}$ MambaVision and MLP blocks which are followed by additional $\frac{N}{2}$ Transformer and MLP blocks. The Transformer blocks in the final layers allow for

图 30: MambaVision 的分层架构。低层 CNN 负责高分辨率局部特征，后层 MambaVision 与 Transformer 负责更长程的空间依赖。来源：MambaVision Fig.2。

从“记忆职责”看，可以把这种设计写成

early stages: local feature extraction → late stages: long-range mixing + exact interaction. (243)

这与第六章 Hybrid 的逻辑一致：当 token 数已经因下采样显著下降后，后层 attention 的二次复杂度更可接受，却能补充精确全局关系。

来源：MambaVision, Sec.3.1–3.3 and Fig.2.

建议分页位置

7.11 第 12 页：MambaOut 是必须保留的反例——“序列长”与“任务是否需要 causal state”是两个不同问题

MambaOut 从 memory perspective 区分 attention 与 RNN-like mixer: attention 显式保存历史 key/-value, RNN/SSM 把历史压缩为固定状态。

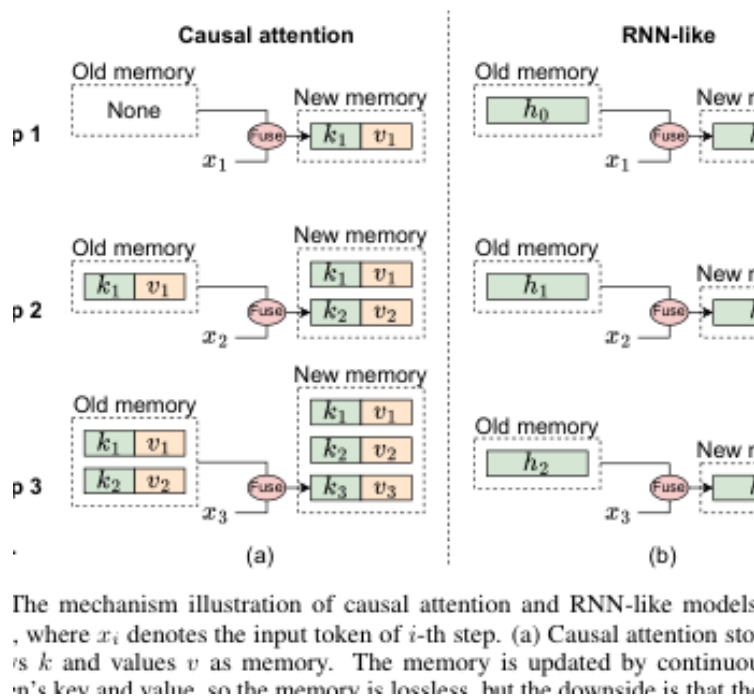


图 31: MambaOut 对 causal attention 与 RNN-like memory 的对照。前者显式累积 token memory，后者维持固定大小状态。来源：MambaOut Fig.2。

对图像分类，输入通常只有 $14 \times 14 = 196$ 个 patch，而且任务采用全可见序列混合；此时既不满足“超长序列”，也不天然需要因果递推。MambaOut 因而提出：ImageNet 分类未必需要 SSM；而检测/分割在高分辨率下 token 数更长，SSM 的线性复杂度和全局传播更可能有价值。

用课程语言概括：

是否适合 SSM $\neq$ 数据是不是图像； 关键是序列长度、传播方向与任务所需记忆。 (244)

来源：MambaOut, Sec.3, Fig.2–3 and Sec.4.

建议分页位置

7.12 第 13 页：把视觉路线放到一张表——S4ND、Vim、VMamba、MambaVision、MambaOut 实际回答的是不同问题

表 29: 视觉 SSM/Mamba 代表路线的设计问题。

方法	如何处理二维结构	是否保留 Attention	核心教学结论
S4ND	多时间轴/PDE, 1D kernel 外积	否	不必 flatten; 可直接构造连续 ND kernel
Vim	一维 patch 序列 + 双向 SSM	否	单向 causal state 不适合 fully-visible 图像
VMamba	四方向 Cross-Scan / Cross-Merge	否	scan path 本身是二维归纳偏置
MambaVision	分层 CNN + Mamba + Transformer	是	局部、长程与精确全局关系可以分工
MambaOut	去掉核心 SSM 的 gated CNN 对照	可选	分类不满足长序列/causal 特征时, SSM 未必必要

这张表也是后续选模型时更有价值的方式：不问“哪个 visual Mamba 名字更新”，而问二维路径、局部先验和显式检索分别由谁承担。

建议分页位置

7.13 第 14 页：视频把问题再放大一层——token 数从 HW 变成 THW ，同时需要空间与时间两类上下文

若视频被切成 T' 个时间 tubelet、每帧 $H' \times W'$ 个空间 patch，则 token 数为

$$M = T' H' W'. \quad (245)$$

全局 self-attention 的 token mixing 项随

$$\mathcal{O}(M^2 D) = \mathcal{O}(T'^2 H'^2 W'^2 D) \quad (246)$$

增长，而 SSM scan 仍近似

$$\mathcal{O}(MND). \quad (247)$$

因此长视频是 SSM 工程优势非常直接的场景。

但视频还要求扫描路径同时尊重两种邻接：

$$\text{spatial adjacency} \quad \text{and} \quad \text{temporal adjacency}. \quad (248)$$

如果简单把每帧 raster scan 后再拼接，两个空间相邻 patch 和两个时间相邻 patch 的序列距离可能完全不同。因此视频模型通常必须专门设计 3D scan。

建议分页位置

7.14 第 15 页：VideoMamba——三维块嵌入（3D patch embedding）后，用双向 Mamba 做长时空序列建模

VideoMamba 先通过 3D convolution 把视频变成时空 patch token。论文记号可写为

$$X = [X_{\text{cls}}, X_{\text{patch}}] + P_s + P_t, \quad (249)$$

其中 P_s 与 P_t 分别是空间和时间位置编码。随后采用 bidirectional Mamba block 对 3D token 序列进行双向状态传播。

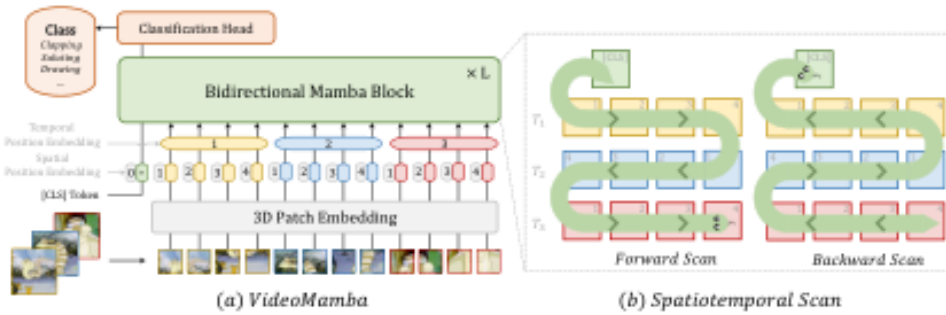


Fig. 3: Framework of VideoMamba. We strictly follow the architecture of vanilla ViT [15], and adapt the bidirectional mamba block [91] for 3D video sequences.

$\mathbb{R}^{B \times L \times N}$, and $\Delta \in \mathbb{R}^{B \times L \times D}$ are directly derived from the input data $x \in \mathbb{R}^{B \times L \times D}$, indicating an intrinsic capacity for contextual sensitivity and adaptive

图 32: VideoMamba 的整体结构与时空扫描示意。来源：VideoMamba Fig.3。

如果用两条 permutation π_f, π_b 表示正反扫描，则与式(235)类似：

$$Y = \text{Merge}(\text{Mamba}(X_{\pi_f}), \text{Reverse}\{\text{Mamba}(X_{\pi_b})\}). \quad (250)$$

论文进一步比较 temporal-first / spatial-first 等 3D 扫描策略；其核心不是公式变化，而是序列化方式改变了状态的时空传播路径。

来源：VideoMamba, Sec.3.1–3.2 and Fig.3–4.

建议分页位置

7.15 第 17 页：机器人场景里，“状态”终于重新对应到物理交互历史——但还必须区分推理（reasoning）与 action

机器人策略面对的是部分可观测闭环：

$$\mathbf{o}_t = \{I_t, q_t, f_t, \text{text}, \dots\}, \quad \mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_{\leq t}, \mathbf{a}_{< t}). \quad (251)$$

理想内部状态应压缩

$$\mathbf{h}_t \approx \phi(\text{object state, contact stage, task progress, action history, } \dots). \quad (252)$$

与文本不同，机器人最终动作还必须受到新观测闭环纠正。因此状态模型的优势主要体现在两类位置：

1. 长历史 reasoning / multimodal context 的高效压缩；
2. 一个动作块（action chunk）内部的有序时间协调。

而视觉 grounding、精确对象引用、语言-动作对齐常仍需要显式 cross-attention 或其他检索机制。

[建议分页位置](#)

7.16 第 19 页：MaIL——更贴近“序列策略”的问题：Mamba 直接参数化动作序列的去噪器

MaIL 把 Mamba 放进 imitation learning。对 behavioral cloning, 可用

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{s,a} \|\mu_{\theta}(s) - a\|_2^2. \quad (253)$$

更重要的是 diffusion policy：给定 noisy action sequence a_t 、diffusion step t 与 observation s ，Mamba 参数化噪声预测器

$$\epsilon_{\theta}(a_t, t, s), \quad (254)$$

训练目标为

$$\mathcal{L}_{\text{DPM}} = \mathbb{E}_{s,a,t} \|\epsilon_{\theta}(a_t, t, s) - \epsilon\|_2^2. \quad (255)$$

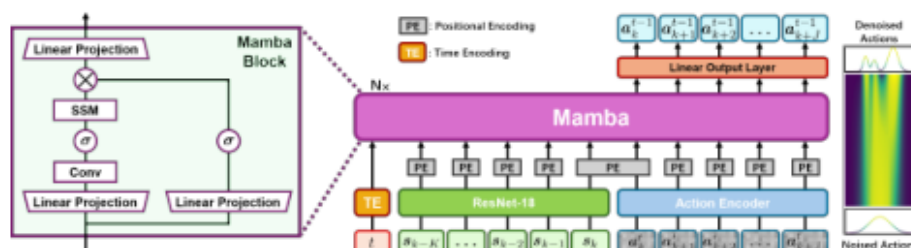


Figure 1: D-Ma: Mamba denoising architecture integrates ResNet-18 for state encoding and an action encoder for action encoding. The state sequence has a length of K , while the action sequence at diffusion step t has a length of J . Before feeding the inputs into the Mamba module, positional encoding (PE) and time encoding (TE) enhance the inputs, where the s_k and a_k share the same positional encoding. The mamba module has $N \times$ mamba blocks, with a detailed structure [13] shown on the left. The outputs from the mamba module are processed by a linear output layer, resulting the one-step denoising actions. The symbol $\times$ in the Mamba block denotes matrix multiplication, and σ the SiLU activation function.

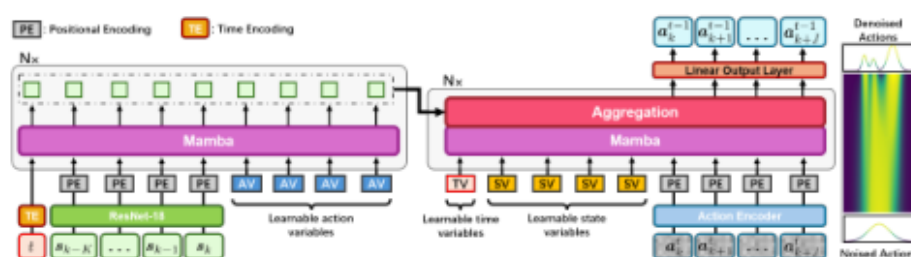


Figure 2: ED-Ma: Different from the D-Ma model, ED-Ma contains a Mamba encoder which is used to process the time embedding and state embedding, and a Mamba decoder which is used to

图 33: MaIL 的 Decoder-only Mamba (D-Ma) 与 Encoder-Decoder Mamba (ED-Ma)。ED-Ma 通过 learnable action/s-tate/time variables 实现长度不同序列间的信息聚合。来源：MaIL Fig.1-2。

这一工作对教学很有意义：它说明 Mamba 的输入 token 不一定是文本/patch，也可以是观测 embedding、noisy action embedding 与 diffusion-time embedding 的混合序列。

来源：MaIL, Sec.3.2-4.2 and Fig.1-2.

建议分页位置

7.17 第 20 页：SmolVLA Mamba

expert——一个非常好的“任务结构”案例：Mamba 只替换 action 块内的 causal temporal mixer

最新的 SmolVLA 分析没有把整个 VLA 都换成 Mamba，而是只替换 action expert 内部的 causal self-attention。跨模态 grounding 保持 cross-attention，不变；改变的只是 action 块内部时间混合器。

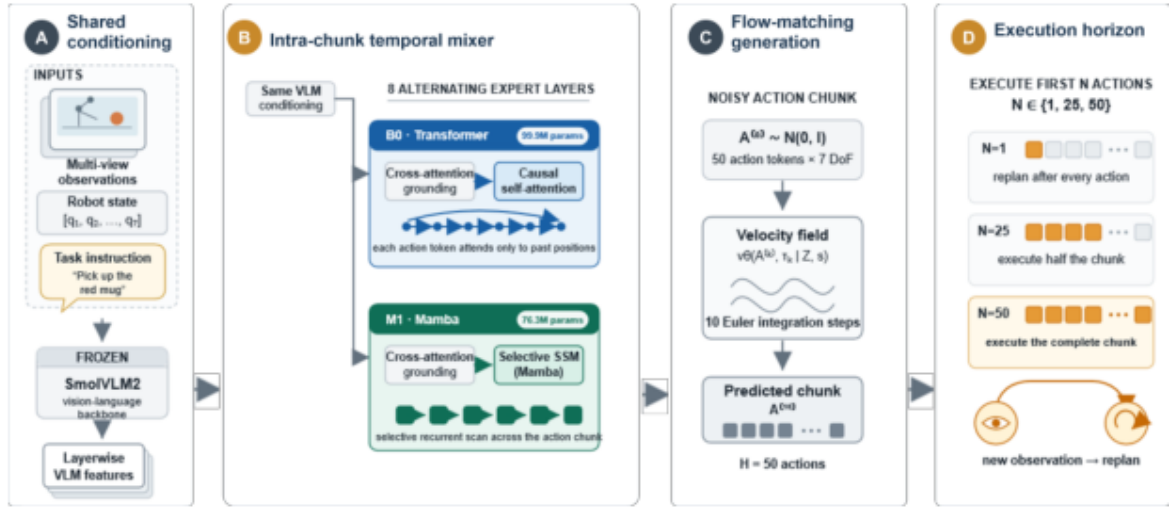


Fig. 2. Proposed Mamba vs. Transformer Action Expert (AE) pipelines. A) The multi-view RGB observations, the robot states, and the task instructions are jointly encoded by a SmoVLM2 backbone into layerwise features that condition the AE. The backbone, its inputs, and the resulting conditioning are identical for both variants. B) The AE alternates cross-attention grounding layers with intra-chunk temporal layers. The two variants only differ in the temporal operator: B0 keeps the original Transformer-based causal self-attention, in which each action token attends to earlier positions of the chunk, and M1 replaces it with a Mamba-1 selective state-space scan over the same positions, significantly reducing the number of trainable weights from 99.9M to 76.3M. C) Both

图 34: SmoVLA 实验隔离变量: B0 使用 causal self-attention 做 intra-chunk temporal mixing; M1 仅把这 8 个 temporal layer 换为 Mamba, cross-attention grounding 和 backbone 保持一致。来源: SmoVLA-Mamba Fig.2。

设一次策略调用生成长度 $H = 50$ 的动作块

$$A_t = (a_t, a_{t+1}, \dots, a_{t+H-1}). \quad (256)$$

执行 horizon N 表示每次重规划前实际执行前 N 个动作:

$$N = 1 \Rightarrow \text{每步重新观测}, \quad N = H \Rightarrow \text{整块 open-loop 执行}. \quad (257)$$

因此 N 越大, 模型越依赖 action 块内部的时间一致性, 而越少依赖中途新视觉反馈。

来源: Mamba-based Selective State Space Modeling Improves the Accuracy-Complexity Tradeoff of SmoVLA Experts, Sec.III.

建议分页位置

7.18 第 21 页: 执行 horizon 实验说明了什么——Mamba 的优势在“长开环块协调”时才明显出现

论文比较 Transformer expert B0 与 Mamba expert M1, 在 $N \in \{1, 25, 50\}$ 下的成功率。定义模型 m 的 horizon drop

$$\Delta_{\text{drop}}^m = \text{SR}_m(N = 1) - \text{SR}_m(N = 50), \quad (258)$$

以及 Mamba 相对 Transformer 的差

$$\Delta_{\text{M1-B0}}(N) = \text{SR}_{\text{M1}}(N) - \text{SR}_{\text{B0}}(N). \quad (259)$$

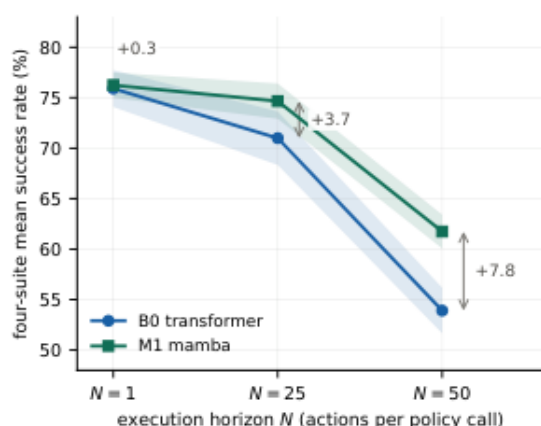


Fig. 3. Four-suite mean success rate as a function of the execution horizon N , mean across three seeds with ± 1 standard deviation bands. Annotations give the M1-B0 gap at each horizon.

evaluation protocol. Both variants are trained on the released

TABLE I
FOUR-SUITE MEAN SUCCESS RATE (%) AS A FUNCTION OF EXECUTION HORIZON, MEAN $\pm$ STANDARD DEVIATION ACROSS SEEDS. THE FINAL COLUMN GIVES THE MEAN PER-TASK DIFFERENCE WITH A 95% BOOTSTRAP CONFIDENCE INTERVAL OVER THE 40 TASKS.

Horizon	B0	M1	M1-B0 [95% CI]
$N = 1$	75.9 ± 1.8	76.2 ± 1.2	$+0.3 [-3.0, +3.6]$
$N = 25$	71.0 ± 2.7	74.7 ± 1.7	$+3.7 [0.0, +7.3]$
$N = 50$	53.9 ± 2.3	61.8 ± 1.7	$+7.8 [+3.0, +12.6]$
$N=1 \rightarrow 50$ drop	22.0	14.5	7.5 less

emerges only as the policy is asked to execute longer prefixes without new observations.

Moreover, Moving from $N = 1$ to $N = 50$, B0 loses 22.0 points and M1 loses 14.5, so M1 better retains 7.5 additional points when the policy must execute a complete predicted chunk before receiving new visual feedback (Table I). M1 obtains this retention with a smaller AE which 76.3M weights against 99.9M trainable parameters for B0.

B. Per-Suite Structure

The aggregate advantage at $N = 50$ is not uniform across suites, as shown in Table II and Fig. 4. M1 significantly

图 35: 随着执行时域 (execution horizon) 增大, Mamba expert 的相对优势扩大: $N = 1$ 几乎持平, $N = 25$ 约 +3.7 点, $N = 50$ 约 +7.8 点; 同时 M1 的 trainable expert 参数更少。来源: SmolVLA-Mamba Fig.3/Table I。

这个结果比简单比较两个模型平均分更有教学价值: 只有当任务真正要求长动作块内部保持因果协调时, selective recurrent mixer 的归纳偏置才更明显地体现出来。

建议分页位置

7.19 第 22 页：第七章收束——不要按模态记模型名，要按“状态沿什么结构传播”来记

前面的应用可以统一写成如下结构：

$$\boxed{\text{原始数据} \xrightarrow{\text{tokenize}} \{z_i\} \xrightarrow{\text{choose scan graph/path}} \text{state propagation} \xrightarrow{\text{merge/readout}} \text{task output.}} \quad (260)$$

表 30: 第七章的最终任务-设计对应表。

场景	主要扫描设计	状态最适合保存	需要警惕的边界
音频/SaShiMi	原始时间轴 + 多尺度	波形上下文与长时间结构	AR 自由运行稳定性
MambaTS	时间 patch + 变量顺序	趋势 + 跨变量依赖	变量 scan order 是否有意义
Vim/VMamba	双向/四向空间 scan	全局空间上下文	flatten 路径造成空间偏置
MambaVision	层级状态 + 注意力	长程空间模式	exact global interaction 仍可由 attention 补强
VideoMamba	3D 时空 scan	长时事件/动作演化	空间、时间邻接冲突
RoboMamba / MaIL	multimodal / action sequence	reasoning、任务进度、动作历史	grounding 与闭环反馈不能全压进 state
SmolVLA	action chunk	块内动作协调	horizon 越短，recurrent 优势越不明显
Mamba expert	causal scan		

应用层面的核心问题始终是：任务真正需要压缩的是哪一类历史，以及数据本身是否提供了合理的递推方向。

建议分页位置

8 第八章：从公式到代码——最小 Mamba 的可验证实践

本章只解决一个问题：怎样把前七章的关键等式变成一套可运行、可对照、可解释的最小实验？
本章给出的答案：先做固定 SSM 的递推/卷积一致性，再实现选择性扫描与最小 Mamba，用合成任务验证内部选择机制。

8.1 第 1 页：实践部分的目标不是“调用 mamba-ssm 包”——而是亲手验证前三章的三条数学结论

实践建议采用一个逐级增加复杂度的路线：

1. **Fixed SSM：**验证递推与卷积完全等价；
2. **Selective SSM：**让 $\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ 依赖输入，完成教学版选择性扫描；
3. **最小 Mamba：**加输入投影、局部 Conv1d、gate、残差和输出投影；
4. **Selective Copying：**用一个真正需要 content-aware write/read 的任务检验选择性；
5. **可选扩展：**把 additive write 改为 Delta Rule，做 key-value overwrite。

整个实验只追踪五个变量：

$$\mathbf{A}, \Delta_t, \mathbf{B}_t, \mathbf{C}_t, \mathbf{h}_t. \quad (261)$$

如果这五个量在代码里含义始终清楚，学生就能把理论和实现对上；如果一开始直接进入 fused CUDA kernel，反而很难看见 Mamba 的核心。

[建议分页位置](#)

8.2 第 2 页：第一步先手写 fixed diagonal SSM——把状态模态和稳定性做成显式参数

为便于实现，采用对角连续状态矩阵

$$\mathbf{A} = \text{Diag}(a_1, \dots, a_N), \quad \Re(a_n) < 0. \quad (262)$$

最稳妥的可学习参数化是

$$a_n = -\exp(\theta_n), \quad \theta_n \in \mathbb{R}, \quad (263)$$

这样训练过程中连续极点始终在左半平面。

固定步长 $\Delta > 0$ 时，离散转移逐模态为

$$\bar{a}_n = e^{\Delta a_n} \in (0, 1), \quad (264)$$

因此

$$\bar{\mathbf{A}} = \text{Diag}(e^{\Delta a_1}, \dots, e^{\Delta a_N}). \quad (265)$$

若 a_n 接近 0^- ，则 $\bar{a}_n$ 接近 1，对应慢记忆；若 a_n 很负，则对应快衰减模态。

初始化时可以人为覆盖多时间尺度，例如设目标离散时间常数 τ_n ，令

$$\bar{a}_n = e^{-1/\tau_n}, \quad \tau_n \in [2, 4, 8, \dots, 2^m], \quad (266)$$

帮助学生直观看到不同状态维度承担不同历史尺度。

建议分页位置

8.3 第 3 页：严格写出 ZOH——实践里不要偷偷用 $\bar{\mathbf{B}} \approx \Delta \mathbf{B}$ 而不说明

对角 $\mathbf{A}$ 下 ZOH 可以逐元素精确计算。连续系统

$$\dot{\mathbf{h}}(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t) \quad (267)$$

在一个步长内输入保持常数，则

$$\bar{\mathbf{A}} = e^{\Delta \mathbf{A}}, \quad (268)$$

$$\bar{\mathbf{B}} = \int_0^{\Delta} e^{\tau \mathbf{A}} \mathbf{B} d\tau = \mathbf{A}^{-1}(e^{\Delta \mathbf{A}} - \mathbf{I})\mathbf{B}. \quad (269)$$

对第 n 个对角模态：

$$\bar{b}_n = \frac{e^{\Delta a_n} - 1}{a_n} b_n. \quad (270)$$

当 $a_n \rightarrow 0$ 时用极限

$$\bar{b}_n \rightarrow \Delta b_n \quad (271)$$

处理数值稳定性。

所以固定 SSM 的参考递推实现（reference recurrence）为

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \quad (272)$$

$$y_t = \mathbf{C}^\top \mathbf{h}_t + Dx_t. \quad (273)$$

这就是后面所有代码的 golden reference。

建议分页位置

8.4 第 4 页：先验证递推与卷积——这一步能一次排掉大量实现错误（bug）

零初始状态下，由前两章推导

$$K_i = \mathbf{C}^\top \bar{\mathbf{A}}^i \bar{\mathbf{B}}, \quad (274)$$

输出应满足

$$y_t = \sum_{i=0}^t K_i x_{t-i} + Dx_t. \quad (275)$$

因此实践第一项自动测试应为

$$\epsilon_{equiv} = \max_t |y_t^{\text{recurrence}} - y_t^{\text{convolution}}|. \quad (276)$$

对于 float64 的短序列，应该接近机器精度；float32 也应保持非常小的误差。

Listing 1: Fixed SSM 的最小数值验证逻辑。

```
h = zeros(N)
```

```

y_rec = []
for t in range(T):
    h = Abar * h + Bbar * x[t]
    y_rec.append((C * h).sum() + D * x[t])

K = []
apow = ones(N)
for i in range(T):
    K.append((C * apow * Bbar).sum())
    apow = apow * Abar

y_conv = causal_convolution(x, K) + D * x
assert max_abs(y_rec - y_conv) < tol

```

这一关通过后，学生才真正拥有一个可以信任的 SSM reference；后面任何 selective/parallel 实现都应该先与它对齐。

建议分页位置

8.5 第 5 页：从固定 SSM 到 selective SSM——只改三件事，不要一开始就把整个 Mamba block 搬进来

沿用第三章记号，让

$$\mathbf{B}_t = W_B \mathbf{x}_t, \quad (277)$$

$$\mathbf{C}_t = W_C \mathbf{x}_t, \quad (278)$$

$$\Delta_t = \text{softplus}(W_\Delta \mathbf{x}_t + b_\Delta). \quad (279)$$

共享的连续 $\mathbf{A}$ 仍由式(263)保证稳定。逐 token 离散化：

$$\bar{\mathbf{A}}_t = e^{\Delta_t \mathbf{A}}, \quad (280)$$

$$\bar{\mathbf{B}}_t = \mathbf{A}^{-1}(e^{\Delta_t \mathbf{A}} - \mathbf{I})\mathbf{B}_t. \quad (281)$$

最终 reference recurrence：

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t, \quad (282)$$

$$\mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t + D \mathbf{x}_t. \quad (283)$$

这一版本已经拥有 Mamba 最核心的 selective state mechanism，但尚未加局部卷积、gate 与 residual。教学上应先让学生看懂式(279)–(283)，再封装成完整 block。

建议分页位置

8.6 第 6 页：张量形状必须先固定——很多“公式看懂但代码写错”都来自广播维度不清

设 batch B_s 、序列长度 T 、模型维度 D_m 、每通道状态维度 N 。教学版可以采用 Mamba 常见的 per-channel state：

$$\mathbf{h}_t \in \mathbb{R}^{B_s \times D_m \times N}. \quad (284)$$

各张量建议固定为：

表 31: Minimal selective SSM 的建议张量形状。

变量	Shape
输入 x	(B_s, T, D_m)
连续 A	(D_m, N)
Δ_t	(B_s, T, D_m)
B_t	(B_s, T, N) 或 (B_s, T, D_m, N)
C_t	(B_s, T, N) 或 (B_s, T, D_m, N)
状态 h_t	(B_s, D_m, N)
输出 y	(B_s, T, D_m)

如果采用共享 B_t, C_t ，则写入时广播为

$$\bar{\mathbf{B}}_t[:, \text{None}, :] \odot x_t[:, :, \text{None}], \quad (285)$$

必须在代码注释中明确这一点。

[建议分页位置](#)

8.7 第 7 页：教学版选择性扫描先用逐步循环（for-loop）——先求正确，再求并行

Listing 2: 教学版选择性扫描的 golden reference。

```
def selective_scan_ref(x, A, delta, B_t, C_t, D):
    # x: [B,T,D]
    # A: [D,N], negative
    # delta: [B,T,D], positive
    h = zeros(B, D, N, device=x.device)
    ys = []
    for t in range(T):
        At = exp(delta[:,t,:,None] * A[None,:,:])
        # exact diagonal ZOH factor phi = (exp(delta*A)-1)/A
        phi = expm1(delta[:,t,:,None] * A[None,:,:]) / A[None,:,:]
        write = phi * B_t[:,t,None,:] * x[:,t,:,None]
        h = At * h + write
        y = (C_t[:,t,None,:] * h).sum(-1) + D * x[:,t]
        ys.append(y)
    return stack(ys, dim=1)
```

这里使用 $\text{expm1}(z) = e^z - 1$ ，是因为当 $z \approx 0$ 时直接计算 $e^z - 1$ 会发生消减误差。该细节非常适合课堂展示：**数学等价并不保证数值实现等价**。

后续任何 prefix scan / chunkwise 版本都必须验证

$$\max |y^{\text{fast}} - y^{\text{ref}}| < \varepsilon. \quad (286)$$

建议分页位置

8.8 第 8 页：再封装最小 Mamba Block——局部卷积和 gate 的功能要与 selective state 分开解释

一个足够教学化的 block 可以写成：

$$[u, z] = \text{split}(W_{\text{in}}x), \quad (287)$$

$$\tilde{u} = \text{SiLU}(\text{DWConv1D}(u)), \quad (288)$$

$$(\Delta_t, B_t, C_t) = \text{Proj}(\tilde{u}_t), \quad (289)$$

$$s = \text{SelectiveSSM}(\tilde{u}; A, \Delta, B, C), \quad (290)$$

$$y = W_{\text{out}}(s \odot \text{SiLU}(z)). \quad (291)$$

各部件职责不要混在一起：

表 32: 最小 Mamba block 中各部件的学习职责。

部件	主要功能
DWConv1D	捕捉邻近 token 的局部模式，避免全部短程结构都交给状态递推
Δ_t, B_t, C_t	内容相关地控制保留时间、写入方向和读出方向
Selective SSM	维护压缩的长期状态
gate $\text{SiLU}(z)$	控制输出通道与增加非线性
residual / norm	保证深层堆叠训练稳定

这样学生能够清楚地区分：“Mamba block”是完整神经网络模块，而“selective SSM”只是其中的核心 mixer。

建议分页位置

8.9 第 9 页：主实践任务为什么选 Selective Copying——它直接隔离“按内容写入”的能力

设计长度 T 的输入序列，其中大多数位置为 noise token；随机选 m 个位置 $p_1 < \dots < p_m$ 写入 key/value token，末尾加入 query/GO token。目标是在 query 后按顺序输出被选中的值：

$$\mathbf{x}_{1:T} = [0, 0, v_1, 0, \dots, v_2, \dots, \text{GO}], \quad \mathbf{y}^* = [v_1, v_2, \dots, v_m]. \quad (292)$$

关键是 p_i 每个样本随机变化，因此固定距离核无法依靠位置模板完成任务。

数据生成器最少需要控制四个变量：

$$T, \quad m, \quad |\mathcal{V}|, \quad \text{spacing distribution.} \quad (293)$$

建议训练长度 $T_{\text{train}} \in [64, 128]$ ，测试时额外评估

$$T_{\text{test}} \in \{T_{\text{train}}, 2T_{\text{train}}, 4T_{\text{train}}\}, \quad (294)$$

把“训练内准确率”和“长度外推”分开。

Listing 3: Selective Copying 数据的核心生成逻辑。

```
seq = zeros(T, dtype=long)
pos = sort(choice(arange(1, T-q_len-1), m, replace=False))
vals = randint(1, vocab_size, size=m)
seq[pos] = vals
seq[-1] = GO
label = vals # or an autoregressive target segment
```

建议分页位置

8.10 第 10 页：实验不能只比较最终 accuracy——必须把“模型改动”对应到可观测内部量

至少比较三组 baseline：

1. Fixed SSM：固定 B, C, Δ ；
2. LSTM/GRU：有门控但没有 SSM 连续时间结构；
3. 最小 Mamba：输入相关 Δ_t, B_t, C_t 。

主指标为任务准确率；同时记录长度外推曲线

$$\text{Acc}(T), \quad (295)$$

但更重要的是把位置分为 noise/key/query 三类，统计选择性参数：

$$\mu_{\Delta}^{\text{noise}}, \mu_{\Delta}^{\text{key}}, \mu_{\Delta}^{\text{query}}, \quad (296)$$

$$\|B_t\|_2 \text{ by token type}, \quad \|C_t\|_2 \text{ by token type.} \quad (297)$$

如果模型真的学会了 selection，应观察到 key 与 noise 位置的写入/时间尺度统计显著不同，query 位置的读出强度也可能出现变化。

“Mamba 能做 Selective Copying”只是结果；“参数在 key/noise/query 处形成可解释差异”才真正验证了 selectivity。

建议分页位置

8.11 第 11 页：把 Δ_t 画出来——它最适合用来解释“状态时间常数”

对单模态 $a < 0$ ，离散保留系数为

$$\alpha_t = e^{\Delta_t a}. \quad (298)$$

定义等效离散时间常数

$$\tau_t = -\frac{1}{\log |\alpha_t|} = -\frac{1}{\Delta_t a}. \quad (299)$$

因此画 Δ_t 与画 τ_t 的含义正好相反： Δ_t 大，旧状态更快衰减； τ_t 大，状态保留更久。

实践建议同时画四条曲线：

$$\Delta_t, \quad \|B_t\|, \quad \|C_t\|, \quad \|h_t\|. \quad (300)$$

在输入 token 上标注 noise/key/query。这样一张图就能把第三章“跳过-写入-读出”从语言描述变成可观察现象。

[建议分页位置](#)

8.12 第 12 页：长度外推要区分两种失败——记忆衰减失败与状态容量失败不是一回事

如果固定 SSM 在长序列上准确率下降，可能是

$$\|\bar{\mathbf{A}}^k\| \rightarrow 0 \quad (301)$$

导致关键信息衰减；这属于**时间尺度**问题。Mamba 可通过 Δ_t 动态调整缓解。

但当要同时保存的独立 key-value 数量 m 持续增长，而状态维度 N 固定时，即使时间尺度足够长，也会出现**容量**问题。可设计二维测试网格：

$$(T, m) \in \{64, 128, 256, 512\} \times \{1, 2, 4, 8, 16\}. \quad (302)$$

画 accuracy heatmap，就能区分：

- 随 T 增大下降：更像长程保持问题；
- 随 m 增大下降：更像有限状态容量/干扰问题。

这正好把实践与第六章精确回忆的理论边界接起来。

[建议分页位置](#)

8.13 第 13 页：从 for-loop 到并行扫描——实践里只需要证明结合律，不必一上来写 CUDA

每一步 selective recurrence 都可写成仿射变换

$$f_t(h) = A_t h + b_t. \quad (303)$$

定义二元对象

$$g_t = (A_t, b_t), \quad (304)$$

组合运算

$$(A_2, b_2) \circ (A_1, b_1) = (A_2 A_1, A_2 b_1 + b_2). \quad (305)$$

直接验证

$$(g_3 \circ g_2) \circ g_1 = g_3 \circ (g_2 \circ g_1), \quad (306)$$

所以可以使用 parallel prefix scan。

学生实践可以只完成一个 CPU/PyTorch tree-scan 版本，再与 for-loop reference 比较输出与梯度：

$$\epsilon_y = \max |y^{\text{scan}} - y^{\text{loop}}|, \quad \epsilon_g = \max \|\nabla \mathcal{L}^{\text{scan}} - \nabla \mathcal{L}^{\text{loop}}\|. \quad (307)$$

不需要在课程里重新实现 Triton/CUDA 融合算子才能理解并行原理。

[建议分页位置](#)

8.14 第 15 页：可选挑战任务——把第五章 Delta Rule 接回实践，做“同一 key 被新 value 覆盖”

构造 key-value overwrite 数据：序列中同一个 key 先出现旧值，再出现新值，最终 query 要求输出最新值：

$$(k, v_{\text{old}}), \dots, (k, v_{\text{new}}), \dots, q = k \Rightarrow \hat{v} = v_{\text{new}}. \quad (308)$$

加法 memory

$$S_t = S_{t-1} + v_t k_t^\top \quad (309)$$

会产生旧/新值叠加，而 delta update

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1} k_t) k_t^\top \quad (310)$$

显式写入 prediction error。

实践比较：

$$\text{最小 Mamba state} \quad \text{vs.} \quad \text{additive linear memory} \quad \text{vs.} \quad \text{Delta memory}. \quad (311)$$

这样第八章可以把第三章 Mamba 与第五章 DeltaNet 真正串成一个可运行实验，而不是在课程末尾各自孤立。

[建议分页位置](#)

8.15 第 16 页：建议最终作业结构——学生交付的不是一个 accuracy，而是一套“公式-代码-证据”闭环

表 33: 建议的课程实践交付项。

任务	必须交付的证据	建议权重
Fixed SSM	递推与卷积公式、代码、数值误差	15%
Selective SSM	Δ_t, B_t, C_t 生成与 ZOH 递推；与 reference 对齐	20%
最小 Mamba	Conv-SSM-gate-residual 结构与参数量说明	15%
Selective Copy-ing	训练/测试准确率、长度外推曲线	20%
可解释性	noise/key/query 的 Δ, B, C, h 可视化	15%
扩展	并行扫描、Delta overwrite 或 NPU 迁移任选一项	15%

作业报告必须回答三类问题：

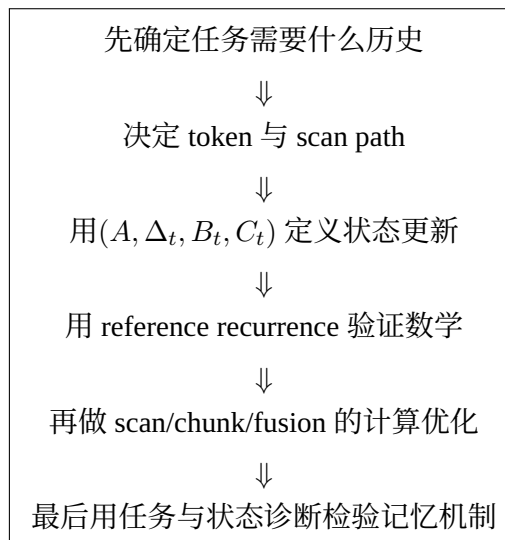
1. **数学：**为什么这个更新是从 SSM 离散化/Delta Rule 推出来的？
2. **算法：**为什么优化实现和 reference 数值一致？
3. **机制：**模型性能变化能否在 Δ_t, B_t, C_t 或状态轨迹中找到对应证据？

实践课的最终目标不是“训练一个 Mamba”，而是让学生能够从状态方程写出算法、验证算法，再用实验解释模型为什么有效或为什么失败。

建议分页位置

8.16 第 17 页：第七、八章最终收束——应用和实践都回到同一个判断框架

从应用到手搓实现，本课可以用下面一条完整链条结束：



(312)

前七章中出现的模型差异都可以放回这条链：S4 主要设计时间尺度基与快速 kernel；Mamba 增加输入相关 selection；Mamba-2 重构计算视图；Delta/GDN 改进状态写入语义；Hybrid 承认精确检索与压缩状态是不同资源；应用模型则进一步改变序列化方式（tokenization）与扫描几何（scan geometry）。

因此学生以后看到一个新的“XX-Mamba”时，真正应该问的不是名称，而是：

它改变了状态方程的哪一项？ 它改变了扫描路径吗？ 它解决了什么可验证的旧问题？

 (313)

A 扩展阅读：被弱化但仍值得保留的技术细节

这一部分收纳主线中不必第一次学习就展开的细节。它们没有被删除，只从正文移到扩展阅读，以避免打断“表示历史—选择更新—高效计算—可编辑记忆—Hybrid 边界”这条主线。

A. S4 简化分支中的额外细节

第 32 页：S4D 的关键结论——对角化在表达上很一般，但初始化与数值结构仍然重要

S4D 强调两个容易混淆的事实。第一，从纯表达能力看，几乎所有矩阵都可由可对角化矩阵逼近，因此“对角 SSM”并不必然表达力弱；第二，这并不保证任意对角参数化都容易训练。

S4D 用 HiPPO-N/正规近似（normal approximation）构造良好的对角初始化。对于实系统，为了得到实值卷积核，复特征值按共轭对出现。只参数化一半特征值后，实际 kernel 取

$$K_j = 2 \operatorname{Re} \left(\sum_{n=1}^{N/2} C_n B_n \bar{\lambda}_n^j \right). \quad (314)$$

这样既保证输出为实数，又减少一半复状态存储。

下图展示 S4-LegS 与 S4D-LegS 基函数的关系：

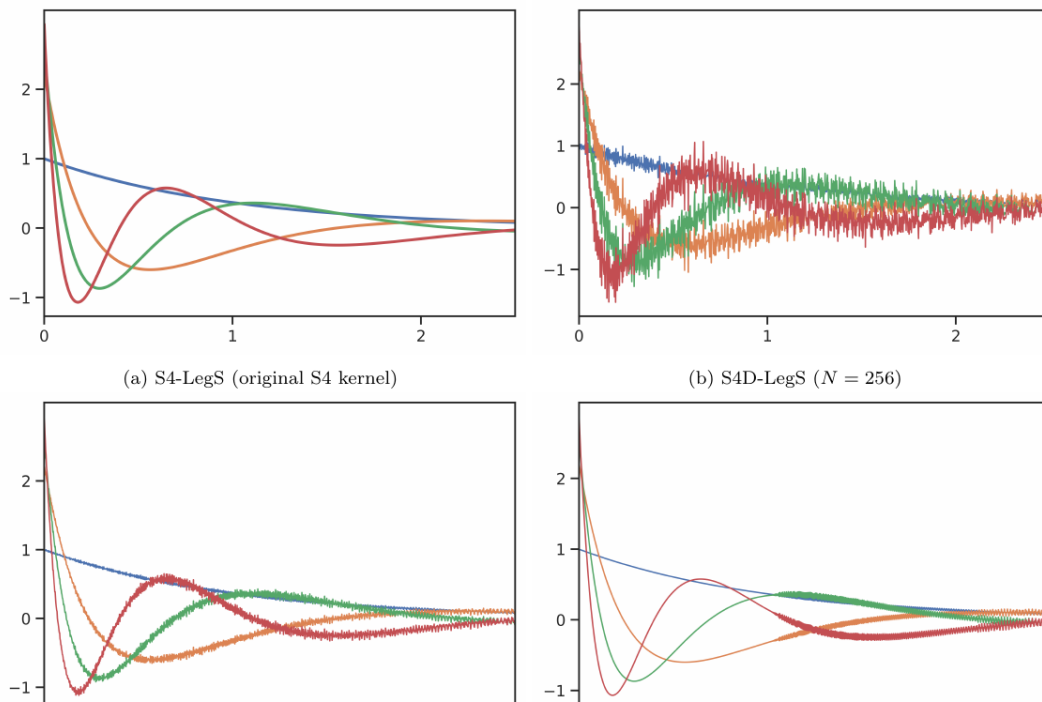


图 36: S4D 对 HiPPO/S4 基函数的近似可视化（原图来自 S4D Fig. 2）。左上是 S4-LegS 的平滑基，其余图展示对角近似随状态维数增加逐渐接近原结构。

因此 S4D 的教学意义不是“证明低秩项完全没用”，而是把 S4 中不同设计因素拆开：**状态谱的时间尺度结构与良好初始化往往比“必须保留某个具体低秩项”更本质。**

来源：S4D Theorem 3, Fig. 2, Sec. 5.

建议分页位置

第 34 页：S5 为什么要求对角动态——并行扫描需要逐状态独立的转移

S5 为了高效并行扫描，把连续状态矩阵对角化：

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}.$$

定义

$$\tilde{\mathbf{x}} = \mathbf{V}^{-1}\mathbf{x}, \quad \tilde{\mathbf{B}} = \mathbf{V}^{-1}\mathbf{B}, \quad \tilde{\mathbf{C}} = \mathbf{C}\mathbf{V},$$

得到

$$\dot{\tilde{\mathbf{x}}}(t) = \mathbf{\Lambda}\tilde{\mathbf{x}}(t) + \tilde{\mathbf{B}}\mathbf{u}(t), \quad \mathbf{y}(t) = \tilde{\mathbf{C}}\tilde{\mathbf{x}}(t) + \mathbf{D}\mathbf{u}(t). \quad (315)$$

ZOH 离散化后

$$\bar{\mathbf{A}} = e^{\mathbf{\Lambda}\Delta}, \quad \bar{\mathbf{B}} = \mathbf{\Lambda}^{-1}(e^{\mathbf{\Lambda}\Delta} - \mathbf{I})\tilde{\mathbf{B}}, \quad \bar{\mathbf{C}} = \tilde{\mathbf{C}}. \quad (316)$$

实际 S5 使用每个 latent mode 都可学习的时间尺度 Δ_p ，从而让不同状态维覆盖不同依赖范围。

对角递推的第 p 个状态满足

$$x_{k,p} = a_{k,p}x_{k-1,p} + b_{k,p}^{\top}\mathbf{u}_k. \quad (317)$$

两个仿射更新可以结合：

$$(a_2, b_2) \circ (a_1, b_1) = (a_2a_1, a_2b_1 + b_2), \quad (318)$$

而且该运算满足结合律，所以能用 parallel prefix scan 把整段递推并行化。这里已经出现了后面 Mamba 选择性扫描的数学雏形。

来源：S5 Eqs. (4)–(6), Sec. 3.3.

建议分页位置

B. Mamba-2 的 head-sharing 设计语言

第 15 页：从 Attention 借来的第二个语言——SSM 也可以定义 multi-head / shared-head 模式

SSD 统一了 SSM 与 attention 的表示以后，很多 attention 中的 head 设计可以翻译到 SSM。令总模型维度

$$D = HP,$$

其中 H 是 head 数， P 是每个 head 的输入/输出维度，状态维数为 N 。

Mamba-2 论文把几种模式整理为：

表 34: Mamba-2 中 SSM head pattern 与 attention head pattern 的对应。

SSM 模式	Attention 类比	参数共享直觉
Multi-head SSM (MHS)	MHA	X, A, B, C 均按 head 独立
Multi-contract (MCS)	MQA	共享 X, B , 不同 head 使用不同 C 做 contraction
Multi-expand (MES)	Multi-key	不同 head 使用不同 B , 共享部分 contraction
Multi-input (MIS)	Multi-value (MVA)	X 按 head 独立, 而 B, C 可共享; Mamba-2 默认偏向这一设计

这部分的教学意义不是让学生背四个缩写, 而是看到 SSD 统一视图的实际价值: 以前 SSM 与 attention 各有一套术语; 一旦二者都写成 sequence transformation, 头共享、核特征、归一化等设计可以跨体系迁移。

来源: Mamba-2 Sec. 7.2, Proposition 7.2.

[建议分页位置](#)

C. KDA 与动态 DPLR/chunkwise 的进一步细节

第 18 页: KDA 与 DPLR 的关系——为什么这条线又重新和 S4 的“对角 + 低秩”接上了

展开式(168)中的 transition:

$$(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t = \mathbf{D}_t - \beta_t \mathbf{k}_t (\mathbf{k}_t^\top \mathbf{D}_t) \quad (319)$$

$$= \mathbf{D}_t - \mathbf{a}_t \mathbf{b}_t^\top, \quad (320)$$

其中

$$\mathbf{a}_t = \beta_t \mathbf{k}_t, \quad \mathbf{b}_t = \mathbf{k}_t \odot \alpha_t. \quad (321)$$

因此 KDA 的状态转移正是一个 **data-dependent DPLR**:

$$\boxed{\mathbf{A}_t = \mathbf{D}_t - \mathbf{a}_t \mathbf{b}_t^\top} \quad (322)$$

这与第二章 S4 的静态 DPLR 在数学形式上高度相似, 但目的不同:

- S4: 静态 DPLR 主要用于长卷积核的快速计算;
- KDA: 动态 DPLR 用于 token-dependent memory decay + targeted overwrite.

一般动态 DPLR 虽更灵活, 但 chunkwise 计算中容易出现累计 decay 的除法和数值稳定问题。KDA 通过约束低秩向量都由同一个 $\mathbf{k}_t$ 生成, 使计算结构更规整。论文报告其 kernel execution time 相比一般 DPLR 约有接近 $2\times$ 的速度优势。

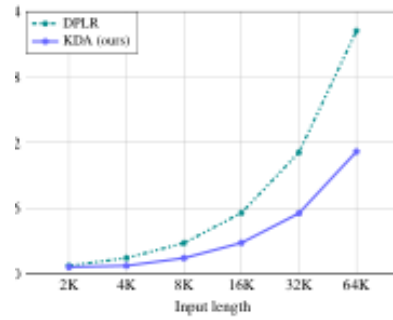


Figure 2: Execution time of kernels for vary-input lengths, with a uniform batch size and 16 heads.

图 37: Kimi Linear Figure 2 裁剪：KDA 与一般 DPLR kernel 的执行时间比较。

来源：Kimi Linear Sec. 3.2, Sec. 6.2.

建议分页位置

第 19 页：KDA 的 chunkwise 计算为什么更复杂——标量累计衰减变成“每个通道一条累计衰减轨迹”

对标量 gate，块内累计衰减只需一个数

$$\gamma_{i \rightarrow j} = \prod_{r=i}^j \alpha_r.$$

KDA 中则变成逐通道累计

$$\gamma_{i \rightarrow j} = \bigodot_{r=i}^j \alpha_r, \quad \text{Diag}(\gamma_{i \rightarrow j}) = \prod_{r=i}^j \text{Diag}(\alpha_r). \quad (323)$$

因此 chunk 部分展开中的“旧状态传播”不再是一个标量 γ 乘 $\mathbf{P}_r$ ，而是

$$\mathbf{P}_r = \prod_{i=1}^r (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) \mathbf{D}_i, \quad (324)$$

其 compact representation 要同时跟踪 rank-one correction 和通道级 cumulative decay。

Kimi 论文继续采用 WY + UT transform，把一串 rank-one 修改压成 dense chunk representation；但由于 diagonal gate 会引入更复杂的衰减比值，算法必须避免直接计算不稳定的 reciprocal cumulative decay。KDA 的约束 DPLR 正是在这里发挥作用：它减少二级 chunk 计算和额外矩阵乘，从而保持 half-precision GEMM 的硬件利用率。

这一页的学习重点不是复现所有 kernel 细节，而是建立一个连贯认识：

更细的 memory control $\Rightarrow$ 更复杂的 transition $\Rightarrow$ 必须重新设计 chunkwise algebra.

这与 S4、Mamba、Mamba-2、DeltaNet 的演化逻辑完全一致：每次提高表达力，都必须重新解决“怎样高效算”的问题。

来源：Kimi Linear Sec. 3.1–3.2, Eq. (2)–(9).

建议分页位置

D. Hybrid 大模型的补充案例与组织形式

第 13 页：Jamba 是很早的 production-scale 例子——Hybrid 一开始就同时在优化质量、吞吐和 KV cache

Jamba 把 Transformer attention、Mamba 和 MoE 组合在同一 decoder 中。其公开配置使用每个 block 8 层，attention-to-Mamba 比例

$$a : m = 1 : 7,$$

并每两层使用一次 MoE。它的设计自由度包括 attention/Mamba 层比例、MoE 频率、专家数与 active experts 数量。

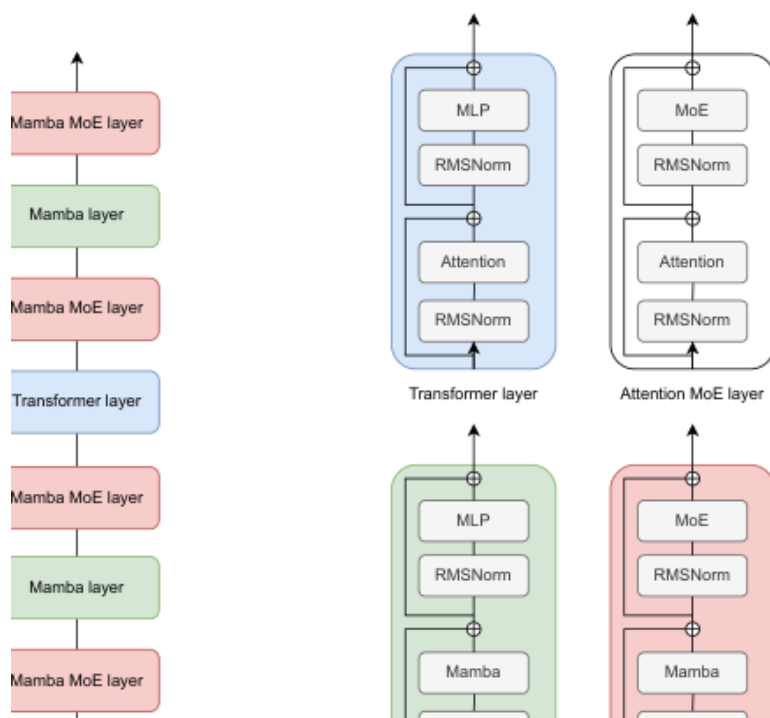


图 38: Jamba Figure 1 裁剪：一个 Jamba block 中 attention 与 Mamba layer 按 1:7 混合，并可在 MLP 路径上加入 MoE。

Jamba 原论文在 256K context、16-bit KV cache 假设下给出：

表 35: Jamba Table 1: 256K context 时 KV cache 量级。

模型	Available params	Active params	KV cache
LLaMA-2	6.7B	6.7B	128 GB
Mistral	7.2B	7.2B	32 GB
Mixtral	46.7B	12.9B	32 GB
Jamba	52B	12B	4 GB

因此早期 Hybrid 的工程逻辑已经很明确：把 attention 从“每层都用”变成“少量关键层使用”，KV cache 立刻按比例下降。

来源：Jamba Fig. 1, Table 1, Sec. 2–3.

建议分页位置

第 14 页：Jamba-1.5 给了一个很有意思的反例——有了 Attention 后，Mamba-2 的大状态优势未必还那么重要

Jamba-1.5 继续沿用 1 : 7 attention-to-Mamba layer ratio。论文专门比较 Mamba-1、Mamba-2、Mamba-1+Attention 和 Mamba-2+Attention：纯 recurrent 情况下 Mamba-2 明显优于 Mamba-1；但一旦插入 full attention，Mamba-1+Attention 的表现已经很强，Mamba-2 的额外收益明显缩小。

with Mamba-2 [6], a faster and improved version of Mamba, ba and Transformers separately. However, as Figure 1 shows, ie Mamba-1-Attention combination works better than Mamba-2. (We also found the hybrid architecture tothesize this is because some of the advantages of Mamba-2 to use a much larger state size, are less significant when we between the Mamba layers, as they can pool information from



图 39: Jamba-1.5 Figure 1 裁剪：Mamba-2 在纯 recurrent 模型中优势明显，但 full-attention layer interleaving 会部分替代大状态带来的信息汇聚优势。

作者给出的解释非常适合作为课程中的架构设计原则：Mamba-2 的更大状态大小在没有 attention 时有价值，因为所有历史必须压进递推状态；一旦中间定期出现 full attention，它可以重新从完整上下文汇聚信息，递推状态不再独自承担所有长期信息。

换句话说，Hybrid 中各模块不是独立“叠加效果”：

attention 的存在会改变递推状态最优容量与最优复杂度。

这也是为什么后续模型不能简单照搬纯 Mamba 的最优超参数，而要重新搜索 attention 比例、状态大小、head 数等配置。

来源：Jamba-1.5 Sec. 2, Fig. 1.

[建议分页位置](#)

第 18 页：2025–2026 的 Hybrid 不只有一种“层间交替”——还出现了 headwise parallel mixing

不同模型对“怎样混合”给出了不同答案。Jamba、Kimi、Qwen3-Next、OLMo Hybrid 主要采用 layerwise interleaving；Falcon-H1 则采用 parallel hybrid mixer：attention heads 与 Mamba heads 在同一 block 内并行处理同一输入，随后融合。

表 36: 代表性 Hybrid 架构的混合方式。比例仅代表论文/技术报告中的具体配置，不是普适最优值。

模型	recurrent tive	primiti- ve	attention 配置	主要混合方式/比例
Jamba / Jamba-1.5	Mamba-1		full attention	layerwise, attention:Mamba = 1 : 7
Kimi Linear	KDA		MLA	layerwise, KDA:MLA = 3 : 1
Qwen3-Next	Gated DeltaNet		standard/gated attention	layerwise, GDN:Attention = 3 : 1
OLMo Hybrid	GDN（允许负特征值）		MHA	interleaved, GDN:Attention = 3 : 1
Nemotron Nano 2	Mamba-2		self-attention	layerwise, 约 8% 层为 attention；12B 配置为 62 层中 6 attention、28 Mamba-2、28 FFN
Falcon-H1	Mamba		attention heads	headwise parallel, attention/Mamba head 比例可独立调节

这张表引出一个更一般的设计空间：Hybrid 不只是“每几层插一次 attention”，还可以在同层、同 head group 内重新分配计算；而工程部署会进一步影响最优 pattern，例如不同 mixer 的 Tensor Parallel / Mixer Parallelism 如何负载均衡。

来源：Jamba/Jamba-1.5；Kimi Linear；Qwen3-Next；OLMo Hybrid；NVIDIA Nemotron Nano 2；Falcon-H1 对应技术报告。

建议分页位置

E. 应用中的额外扫描与架构案例

第 5 页：VAST 怎样把“扫描顺序”变成可学习对象——训练时探索，推理时固定

由于真实变量图未知，MambaTS 在训练阶段使用 Variable Permutation Training (VPT)：每轮随机打乱变量顺序，得到一个排列

$$\pi = (v_1, v_2, \dots, v_K), \tag{325}$$

并由预测损失 $\ell^{(t)}$ 对相邻转移 (v_i, v_{i+1}) 的代价进行更新。论文维护有向代价矩阵

$$P^{(t)} \in \mathbb{R}^{K \times K}, \tag{326}$$

并采用指数滑动更新

$$p_{v_i, v_{i+1}}^{(t)} = \beta p_{v_i, v_{i+1}}^{(t-1)} + (1 - \beta) \ell^{(t)}. \tag{327}$$

批量版本对损失做尺度归一化，再更新完整矩阵：

$$P^{(t)} = \beta P^{(t-1)} + (1 - \beta) \bar{\ell}^{(t)}. \tag{328}$$

训练结束后，不再每次随机扫描，而是把 P 当成有向图边权，通过 ATSP 近似求一条总代价较低的 Hamilton 路径：

$$\pi^* \approx \arg \min_{\pi \in S_K} \sum_{i=1}^{K-1} P_{\pi_i, \pi_{i+1}}. \tag{329}$$

论文采用模拟退火等启发式求解器，而不是指数复杂度的精确 ATSP。

这给学生一个很有价值的启发：当数据没有天然一维顺序时，**scan order** 可以被视为模型结构变量，而不仅是数据预处理。

来源：MambaTS, Sec.5.2 Variable-Aware Scan along Time.

建议分页位置

第 16 页：Video Mamba Suite 给出了更一般的答案——Mamba 在视频里不只有“做 backbone”一种角色

Video Mamba Suite 系统总结了 SSM 在视频理解中的四类角色：

1. temporal modeling：直接承担长时间序列建模；
2. temporal module：作为现有视觉 backbone 中的时间模块；
3. multimodal interaction：在视频-文本等模态间负责序列交互；
4. spatial-temporal modeling：同时承担空间与时间混合。

2 G. Chen et al.

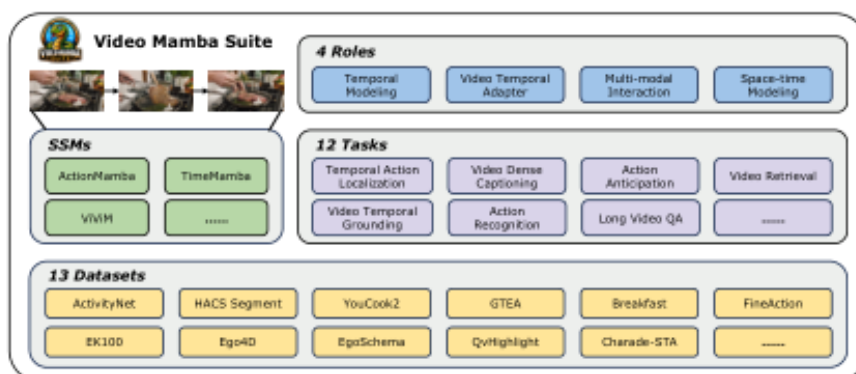


Fig. 1: We investigate SSMs exemplified by Mamba on video understanding. Our Video Mamba Suite comprises 14 SSM models/modules for 12 video understanding tasks. We

图 40: Video Mamba Suite 总览：4 类角色、12 类视频任务以及多种 SSM 模块。来源：Video Mamba Suite Fig.1.

这说明应用设计的关键不是“视频一定用一个 VideoMamba”，而是先确定 SSM 要替代哪一部分计算。若已有强空间 encoder，SSM 完全可以只处理时间轴；若 token 数极长，则再考虑全时空 state mixer。

来源：Video Mamba Suite, Introduction and Fig.1.

建议分页位置

第 18 页：RoboMamba——把 Mamba 用在 VLA 的 reasoning backbone，而动作头保持很小

RoboMamba 的路线非常清楚：视觉 encoder 将图像变为视觉 token，经 connector 对齐到 Mamba LLM 的 embedding space；Mamba 负责视觉-语言 reasoning；最终用一个很小的 policy head 输出机器人动作/位姿。

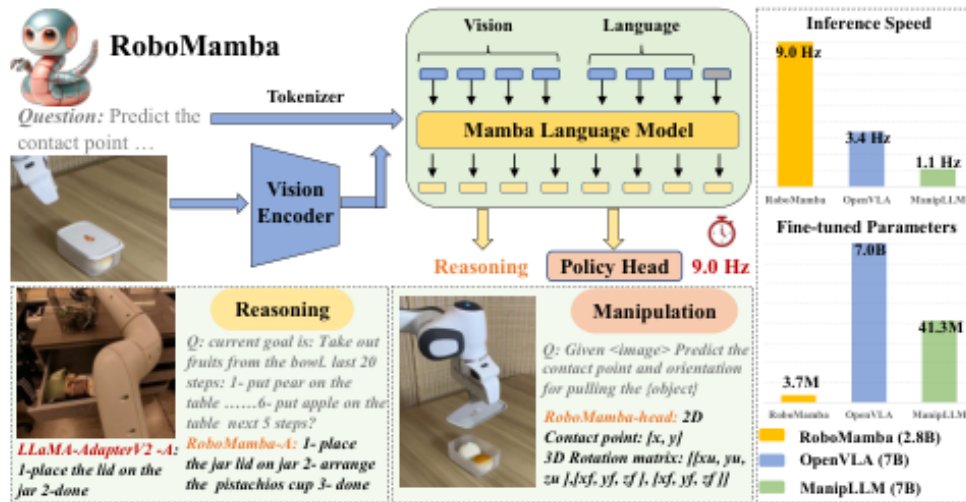


Figure 1: **Overview of RoboMamba.** RoboMamba is an efficient robotic VLA model that combines reasoning and manipulation capabilities. First, we integrate and align a vision encoder with the Mamba LLM, endowing our model with common sense and robotic-related reasoning abilities. Subsequently,

图 41: RoboMamba：视觉 encoder + Mamba language model + 小型 policy head。论文同时比较了推理控制频率与可训练 policy 参数量。来源：RoboMamba Fig.1。

因此它并没有声称“SSM 状态就是机器人真实状态”。更准确的课程表述是

$$\mathbf{h}_t^{\text{Mamba}} = F_\theta(\text{visual-language token history}), \quad \hat{\mathbf{a}}_t = g_\phi(\mathbf{h}_t^{\text{Mamba}}), \quad (330)$$

其中 g_ϕ 是小型 policy head。论文报告该 head 只占整体很小比例，并强调 Mamba backbone 带来的推理效率。

来源：RoboMamba, Fig.1 and Sec.3.

建议分页位置

F. NPU/Ascend 迁移的设备特定建议

第 14 页：如果要迁移到 NPU/Ascend，先保留“数学透明”的基础算子版本

硬件迁移建议分三层：

1. **Reference:** Python for-loop, 保证公式正确；
2. **Framework-native:** 只用 Linear、Conv1d、Softplus、exp/expm1、elementwise mul/add、sum、reshape；
3. **Fused:** 确认 profiler 瓶颈后再考虑自定义 scan/fusion。

不要一开始依赖 CUDA/Triton selective-scan extension，否则模型正确性和设备兼容性会绑在一起。最先建立三组一致性测试：

$$\begin{aligned} \text{CPU ref} &\leftrightarrow \text{GPU 框架原生 (framework-native)}, \\ \text{GPU framework-native} &\leftrightarrow \text{NPU framework-native}, \\ \text{framework-native} &\leftrightarrow \text{融合算子 (fused kernel)}. \end{aligned} \quad (331)$$

检查最大绝对误差、相对误差以及梯度误差。

建议分页位置

术语速查：第一次阅读时不需要死记英文

中文术语	英文 / 缩写	在本课程中的作用
线性时不变系统	linear time-invariant, LTI	参数不随时间/序列位置变化; 固定 SSM 的基本假设
零阶保持	zero-order hold, ZOH	把连续 SSM 精确离散成序列递推的一种输入保持假设
卷积核 / 核	kernel	$K_j = \mathbf{C}\bar{\mathbf{A}}^j\bar{\mathbf{B}}$; 描述距离为 j 的历史如何影响当前输出
勒让德平移 / 缩放	Legendre translated / scaled, LegT / LegS	HiPPO 中两种不同的历史坐标与测度设计
正规矩阵加低秩修正	Normal Plus Low-Rank, NPLR	将 HiPPO 矩阵拆成易稳定对角化的主体与低秩修正
对角矩阵加低秩修正	Diagonal Plus Low-Rank, DPLR	在正规部分的酉特征基下得到的快速计算形式
预解矩阵	resolvent	$(zI - A)^{-1}$ 一类矩阵逆; 生成函数与 S4 快速核计算的核心对象
柯西矩阵	Cauchy matrix	S4 将 DPLR 预解矩阵求值最终归约到的结构化矩阵
选择性扫描	selective scan	Mamba 对输入相关时变递推进行并行和硬件感知计算的方法
半可分矩阵	semiseparable matrix	Mamba-2 中把有限状态递推改写成结构化序列混合矩阵的数学语言
通用矩阵乘	General Matrix Multiplication, GEMM	GPU/TPU 最擅长的稠密矩阵运算; SSD 分块算法的主要计算形态
联想记忆	associative memory	用键、值和查询解释线性注意力、DeltaNet 与 GDN 的矩阵状态
豪斯霍尔德反射	Householder reflection	DeltaNet 秩一删除操作的几何原型
紧凑 WY 表示	compact WY representation	将一串秩一状态转移压缩成适合分块矩阵乘的表示
精确回忆	exact recall	从长历史中精确找回某个键值/片段; 主要受容量和索引能力限制
状态跟踪	state tracking	按顺序维护内部状态机变量; 主要受状态转移表达力限制

建议分页位置

全课程收束：最后只保留五句话

如果需要把整套课程压缩成最小认知骨架，可以只保留下面五句话：

1. **SSM 的基本价值是把历史压缩进有限状态。**固定 LTI 状态可以同时写成递推和卷积，这是后续一切高效算法的起点。
2. **长记忆和内容选择是两个不同问题。**HiPPO/S4 解决“怎样稳定地记得长”，Mamba 解决“当前 token 值不值得记、应该怎样写和读”。
3. **表达力每增强一次，都必须重新解决并行计算。**S4 用结构化卷积核，Mamba 用选择性扫描，Mamba-2 用 SSD，Delta/GDN/KDA 用 WY / 分块计算；算法结构与模型结构始终一起演化。
4. **有限状态不只要能写，还要能改。**Delta Rule 把 additive memory 升级为预测误差校正；GDN/KDA 再加入 adaptive/fine-grained 遗忘。
5. **有限状态不是万能记忆。**对精确回忆，显式 attention/检索仍有不可替代的索引价值，因此未来更自然的路线是“高效状态压缩 + 少量精确检索”的 Hybrid memory。

这五句话对应的最终方法链是

SSM → HiPPO/S4 → Mamba → Mamba-2/SSD → Delta/GDN/KDA → Hybrid.
--

应用和实践部分只是在回答最后一个问题：**面对具体任务时，哪些依赖适合压缩到状态，哪些依赖必须保留显式检索？**